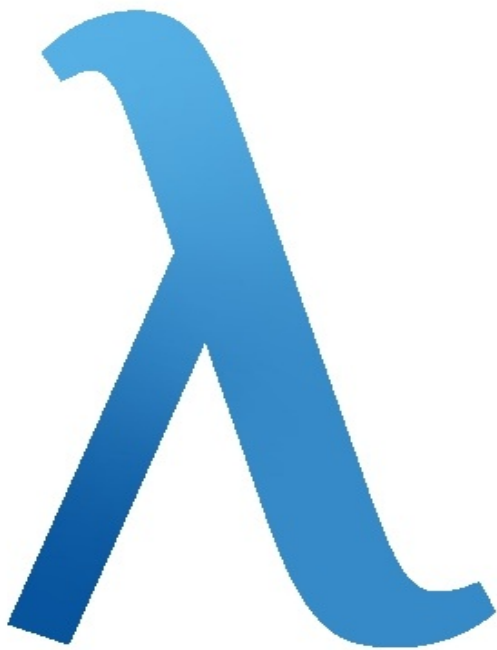


Marcin Moskała

Functional Kotlin



Functional Kotlin

Marcin Moskała

This book is for sale at http://leanpub.com/kotlin_functional

This version was published on 2023-06-26



* * * * *

This is a [Leanpub](#) book. Leanpub empowers authors and publishers with the Lean Publishing process. [Lean Publishing](#) is the act of publishing an in-progress ebook using lightweight tools and many iterations to get reader feedback, pivot until you have the right book and build traction once you do.

* * * * *

© 2022 - 2023 Marcin Moskała

Table of Contents

[Introduction](#)

[Who is this book for?](#)

[What will be covered?](#)

[The Kotlin for Developers series](#)

[Code conventions](#)

[Acknowledgments](#)

[Introduction to functional programming with Kotlin](#)

[Why do we need to use functions as objects?](#)

[Function types](#)

[Defining function types](#)

[Using function types](#)

[Named parameters](#)

[Type aliases](#)

[A function type is an interface](#)

[Anonymous functions](#)

[Lambda expressions](#)

[Tricky braces](#)

[Parameters](#)

[Trailing lambdas](#)

[Result values](#)

[Lambda expression examples](#)

[An implicit name for a single parameter](#)

[Closures](#)

[Lambda expressions vs anonymous functions](#)

[Function references](#)

[Top-level functions references](#)

[Method references](#)

[Extension function references](#)

[Method references and generic types](#)

[Bounded function references](#)

[Constructor references](#)

[Bounded object declaration references](#)

[Function overloading and references](#)

[Property references](#)

[SAM Interface support in Kotlin](#)

[Support for Java SAM interfaces in Kotlin](#)

[Functional interfaces](#)

[Inline functions](#)

[Inline functions](#)

[Inline functions with functional parameters](#)

[Non-local return](#)

[Crossinline and noinline](#)
[Reified type parameters](#)
[Inline properties](#)
[Costs of the inline modifier](#)
[Using inline functions](#)

[Collection processing](#)

[forEach and onEach](#)
[filter](#)
[map](#)
[flatMap](#)
[fold](#)
[reduce](#)
[sum](#)
[withIndex and indexed variants](#)
[take, takeLast, drop, dropLast and subList](#)
[Getting elements at certain positions](#)
[Finding an element](#)
[Counting: count](#)
[any, all and none](#)
[partition](#)
[groupBy](#)
[Associating: associate, associateBy and associateWith](#)
[distinct and distinctBy](#)
[Sorting: sorted, sortedBy and sortedWith](#)
[Sorting mutable collections](#)
[Maximum and minimum](#)
[shuffled and random](#)
[zip and zipWithNext](#)
[Windowing](#)
[joinToString](#)
[Map, Set and String processing](#)
[Using them all together](#)

[Sequences](#)

[What is a sequence?](#)
[Order is important](#)
[Sequences do the minimum number of operations](#)
[Sequences can be infinite](#)
[Sequences do not create collections at every processing step](#)
[When aren't sequences faster?](#)
[What about Java streams?](#)
[Kotlin Sequence debugging](#)
[Summary](#)

[Type Safe DSL Builders](#)

[A function type with a receiver](#)
[Simple DSL builders](#)
[Using apply](#)
[Multi-level DSLs](#)
[DslMarker](#)
[A more complex example](#)

[When should we use DSLs?](#)

[Summary](#)

[Scope functions](#)

[let](#)

[also](#)

[takeIf and takeUnless](#)

[apply](#)

[The dangers of careless receiver overloading](#)

[with](#)

[run](#)

[Using scope functions](#)

[Context receivers](#)

[Extension function problems](#)

[Introducing context receivers](#)

[Use cases](#)

[Classes with context receivers](#)

[Concerns](#)

[Summary](#)

[A birds-eye view of Arrow](#)

[Functions and Arrow Core](#)

[Testing higher-order functions](#)

[Error Handling](#)

[Data Immutability with Arrow Optics](#)

[Notes](#)

Introduction

At the beginning of the 21st century, Java mostly dominated commercial programming. Therefore, the object-oriented paradigm ruled in our discipline. Many thought that the holy war between the two biggest paradigms - object-oriented and functional programming - was resolved, but then Scala showed us that they had never needed to fight with each other in the first place. A programming language can have both functional and object-oriented features that complement each other. This has started a renaissance in functional programming, as many functional programming features have been introduced in many popular languages. Nowadays, most mainstream languages support both functional and object-oriented features, but the problem is that people often still don't know how to use them effectively and efficiently.

This book is about functional programming features in Kotlin. It first covers the essentials, and then it builds on them: it presents important and practical topics like collection processing, function references, scope functions, DSL usage and creation, and context receivers.

Who is this book for?

This book is dedicated to developers with basic experience in using Kotlin or who have read my other book *Kotlin Essentials*.

What will be covered?

This book focuses on Kotlin's functional features, including:

- function types,
- anonymous functions,
- lambda expressions,
- function references,
- functional interfaces,
- collection processing functions,
- sequences,
- DSL usage and creation,
- scope functions,

- context receivers.

This book is based on a workshop I conducted.

The Kotlin for Developers series

This book is a part of a series of books called *Kotlin for Developers*, which includes the following books:

- Kotlin Essentials, which covers all the basic Kotlin features.
- Functional Kotlin, which is dedicated to functional Kotlin features, including function types, lambda expressions, collection processing, DSLs, and scope functions.
- Kotlin Coroutines, which covers all the features of Kotlin Coroutines, including how to use and test them, using flow, best practices, and the most common mistakes.
- Advanced Kotlin, which is dedicated to advanced Kotlin features, including generic variance modifiers, delegation, multiplatform programming, annotation processing, KSP, and compiler plugins.
- Effective Kotlin, which is dedicated to the best practices of Kotlin programming.

In this book, I assume that a reader has the knowledge presented in *Kotlin Essentials*, which I reference explicitly. However, readers with experience in Kotlin, or at least in Java, should be perfectly fine starting their adventure from this book.

Code conventions

Most of the presented snippets are executable, so if you copy-paste them to a Kotlin file, you should be able to execute them. The source code of all the snippets is published in the following repository:

https://github.com/MarcinMoskala/functional_kotlin_sources

I often use comments to explain what will be printed by a particular line.

```
fun main() {  
    val cheer: () -> Unit = fun() {  
        println("Hello")  
    }  
}
```

```
    cheer.invoke() // Hello
    cheer() // Hello
}
```

Sometimes, I also move all such comments to the end of a snippet.

```
fun main() {
    val cheer: () -> Unit = fun() {
        println("Hello")
    }
    cheer.invoke()
    cheer()
}
// Hello
// Hello
```

Occasionally, some parts of code or a result are shortened with `/*...*/`. In such cases, you can read it as “there should be more here, but it is not relevant to the example”.

```
adapter.setOnSwipeListener { /*...*/ }
```

Acknowledgments

This book would not be so good without the reviewers' great suggestions and comments. I would like to thank all of them. Here is the whole list of reviewers, starting from those who influenced it most.



Owen Griffiths has been developing software since the mid 1990s and remembers the productivity of languages such as Clipper and Borland Delphi. Since 2001, He moved to Web, Server based Java and the Open Source revolution. With many years of commercial Java experience, He picked up on Kotlin in early 2015. After taking detours into Clojure and Scala, like Goldilocks, He thinks Kotlin is just right and tastes the best. Owen enthusiastically helps Kotlin developers continue to succeed.



Endre Deak is a software architect building AI infrastructure at Disco, a market leading legal tech company. He has 15 years of experience building complex, scalable systems, and he thinks Kotlin is one of the best programming languages ever created.

Piotr Prus is an Android developer and mobile technology enthusiast since the first Maemo and Android systems. Loves clean simple designs and readable code.

Shares knowledge with the community by writing articles and speaking at conferences. Currently, KMMing and Composing all the things.

Jacek Kotorowicz is an Android developer based in Lublin, graduated from UMCS. Wrote his Master's thesis in C++ in Vim and LaTeX. Later, fell in love-hate relationship with JVM languages and Android platform. First used Kotlin (or at least tried to do so) in versions before 1.0. Still learning how NOT to be a perfectionist and how to find time for learning and hobbies.

Anna Zharkova is a Lead Mobile developer with more than 8 years of experience. Kotlin GDE. Develop both native (iOS - Swift/Objective-c, Android - Kotlin/Java) and cross platform (Xamarin, Kotlin Multiplatform) applications.

Design architectural solution in mobile projects. Leading mobile team, mentorship. Public speaker on conferences and meetups (Droidcon, Android Worldwide, SwiftHero, Mobius). Tutor in Otus. Writing articles about mobile development (especially KMM and Swift)

Norbert Kiesel is a backend Kotlin and Java developer and architect who started using Kotlin 5 years ago as a “better Java” and never looked back. His initiative made Kotlin a recommended language in his company, and he helped its adoption by running a Kotlin user group.

Jana Jarolimova is an Android developer at Avast. She started her career teaching Java classes at Prague City University, before moving on to mobile development, which inevitably led to Kotlin and her love thereof.

Aasif Sheikh and **Sunny Aditya**.

I would also like to thank **Michael Timberlake**, our language reviewer, for his excellent corrections to the whole book.

Introduction to functional programming with Kotlin

What is functional programming? This is not an easy question to answer. There is a popular saying that if you ask two developers about what functional programming is, you will get at least three answers. I don't think there is a single definition that everyone will agree on. However, there are several concepts that are often associated with functional programming, including:

- treating functions as objects,
- higher-order functions,
- data immutability,
- using statements as expressions¹,
- lazy evaluation,
- pattern-matching,
- recursive function calls.

There is also a way of thinking that stands behind functional programming. In the object-oriented approach, we see the world as a set of objects; in contrast, in a functional approach, we see the world as a set of functions. Think of a bedroom: is it a room with a bed, a nightstand, a bedside lamp, etc., or is it just a place where we sleep?

Is Kotlin a functional language? One programmer will say “yes”, whereas another might say “no”. I am certain of two things:

1. Kotlin has powerful support for many features that are typical of functional programming languages.
2. Kotlin is not a **purely** functional language.

Kotlin has powerful support for many features that are typical to functional programming languages. Let's consider our previous list of concepts that are typical of functional programming and let's look at how Kotlin supports them.

Feature	Support
Treating functions as objects	Function types, lambda expressions, function references

Higher-order functions	Full support
Data immutability	<code>val</code> support, default collections are read-only, copy in data classes
Expressions	if-else, try-catch, when statements are expressions
lazy evaluation	lazy delegate.
Pattern-matching	when together with smart casting
Recursive function calls	<code>tailrec</code> modifier

Kotlin was designed to support functional programming (FP), but not as much as Haskell or Scala. However, there are many functional programming features that it does not support. We might mention currying, partial function application, etc. Kotlin's creators wanted to take the best features from FP that they believed are best for practical applications without taking features that might make programs harder to understand or modify. Did they do a good job? Who knows, but it seems that many developers like the final result.

Some developers miss some FP features that are not supported by Kotlin, so they have implemented external libraries like Arrow to make at least some of them available. This book concentrates on the functional features built into Kotlin, but the last chapter presents an overview of the essential Arrow features. It was written by Alejandro Serrano Mena, Simon Vergauwen, and Raúl Raja Martínez, who are Arrow maintainers and co-creators.

Kotlin is not a purely functional language. It has support for features that are typical of an Object-Oriented (OO) approach, and it is often used as a Java successor. Kotlin tries to take the best from both OOP and FP.

If you are reading this book, I assume that you already know the basic Kotlin features. No matter if you use Kotlin in your daily work as a developer, just learned the Kotlin basis, or finished my previous book *Kotlin Essentials*. I assume that you know what a data class is, what the difference is between `val` and `var`, how different statements can be used as expressions, etc. In this book, I will focus on what I believe is the essence of functional programming: using functions as objects. So, we will learn about function types, anonymous functions, lambda expressions, function references, etc. Then, I would like to focus on the most important practical application of using functions as objects: functional-style collections processing. Then, we will look at two other applications: type-safe DSL builders and scope functions. In my opinion, these

are the most important aspects of Kotlin's support for and application of functional programming.

For now, let's focus on what I find essential: using functions as objects in Kotlin. Why do we need this?

Why do we need to use functions as objects?

To understand why we need to use functions as objects, take a look at these functions:

```
fun sum(a: Int, b: Int): Int {
    var sum = 0
    for (i in a..b) {
        sum += i
    }
    return sum
}

fun product(a: Int, b: Int): Int {
    var product = 1
    for (i in a..b) {
        product *= i
    }
    return product
}

fun main() {
    sum(5, 10) // 45
    product(5, 10) // 151200
}
```

The first one calculates the sum of all the numbers in a range; the second one calculates a product. The bodies of these two functions are nearly identical: the only difference is in the initial value and the operation. Yet, without support for treating functions as objects, extracting the common parts would not make any sense. Just think about how it would look in Java before version 8. In such a case, we had to create classes to represent operations and interfaces to specify what you expect... It would be absurd.

```
// Java 7
public class RangeOperations {

    public static int sum(int a, int b) {
        return fold(a, b, 0, new Operation() {
            @Override
            public int invoke(int left, int right) {
                return a + b;
            }
        });
    }

    public static int product(int a, int b) {
        return fold(a, b, 1, new Operation() {
            @Override
            public int invoke(int left, int right) {
                return a * b;
            }
        });
    }
}
```

```

    }
    });
}

private interface Operation {
    int invoke(int left, int right);
}

private static int fold(
    int a,
    int b,
    int initial,
    Operation operation
) {
    int acc = initial;
    for (int i = a; i <= b; i++) {
        acc = operation.invoke(acc, i);
    }
    return acc;
}
}

```

This is where functional programming features come to the rescue. They allow us to easily create a function and pass it as an object. To create a function, we can use a lambda expression. To express what kind of function a parameter expects, we can use a function type. This is what our code might look like if we use lambda expressions and function types:

```

fun sum(a: Int, b: Int) =
    fold(a, b, 0, { acc, i -> acc + i })

fun product(a: Int, b: Int) =
    fold(a, b, 1, { acc, i -> acc * i })

fun fold(
    a: Int,
    b: Int,
    initial: Int,
    operation: (Int, Int) -> Int
): Int {
    var acc = initial
    for (i in a..b) {
        acc = operation(acc, i)
    }
    return acc
}

```

Functional programmers noticed long ago that many repetitive code patterns could be extracted into separated functions with the help of functional programming features. `fold` is a great example. Its more universal form was defined years ago and nowadays it is a part of the Kotlin Standard Library (`stdlib`). This is why we can define our `sum` and `product` in the following way:

```
fun sum(a: Int, b: Int) =  
    (a..b).fold(0) { acc, i -> acc + i }  
  
fun product(a: Int, b: Int) =  
    (a..b).fold(1) { acc, i -> acc * i }
```

However, if we use function references, they can also be defined in the following way:

```
fun sum(a: Int, b: Int) = (a..b).fold(0, Int::plus)  
fun product(a: Int, b: Int) = (a..b).fold(1, Int::times)
```

If you are acquainted with the collection processing functions well, you know that calculating the sum of all the numbers in an iterable can be done with the `sum` method:

```
fun sum(a: Int, b: Int) = (a..b).sum()  
fun product(a: Int, b: Int) = (a..b).fold(1, Int::times)
```

In this book, we will learn this and much more. Does it sound interesting? So, let's get started.

Function types

To represent functions as objects, we need a type to represent them. A type specifies what we can do with an object², for instance by specifying what methods³ and properties it has. A function type is a type that specifies that an object needs to be a function. We can call this function using the `invoke` method. However, functions can have different parameters and result types, so there are many possible function types.

Defining function types

A function type starts with a bracket, inside which it specifies the parameter types, separated with commas. After the bracket, there must be an arrow (`->`) and the result type. Since all functions in Kotlin need to have a result type, a function that does not return anything significant should declare `Unit`⁴ as its result type.

`(ParamType1, ParamType2, ParamType3) -> ResType`

Parameter types

Result type

Here are a few function types (in the next chapters, we will see them in use):

- `() -> Unit` - the simplest function type, representing a function that expects no arguments and returns nothing significant⁵.
- `(Int) -> Unit` - a function type representing a function that expects a single argument of type `Int` and returns nothing significant.
- `(String, String) -> Unit` - a function type representing a function that expects two arguments of type `String` and returns nothing significant.
- `() -> User` - a function type representing a function that expects no arguments and returns an object of type `User`.
- `(String, String) -> String` - a function type representing a function that expects two arguments of type `String` and returns an object of type `String`.

String.

- (String) -> Name - a function type representing a function that expects a single argument of type String and returns an object of type Name.

Functions that return Boolean, like (T) -> Boolean, are often named predicate. Functions that transform one value to another, like (T) -> R, are often called transformation. Functions that return Unit, like (T) -> Unit, are often called operation.

Using function types

A function type offers only one method: `invoke`. Its parameters and result type are the same as defined by the function type.

```
fun fetchText(
    onSuccess: (String) -> Unit,
    onFailure: (Throwable) -> Boolean
) {
    // ...
    onSuccess.invoke("Some text") // returns Unit
    // or
    val handled: Boolean =
        onFailure.invoke(Error("Some error"))
}
```

Since `invoke` is an operator⁶, we can “call an object” that has this method. This is an implicit `invoke` call.

```
fun fetchText(
    onSuccess: (String) -> Unit,
    onFailure: (Throwable) -> Boolean
) {
    // ...
    onSuccess("Some text") // returns Unit
    // or
    val handled: Boolean = onFailure(Error("Some error"))
}
```

You can decide for yourself what approach you prefer. Explicit `invoke` calls are more readable for less experienced developers. An implicit call is shorter and, from a conceptual perspective, it better represents calling an object.

If a function type is nullable (in such a case, wrap it with a bracket and add a question mark at the end), you can use a safe call only with an explicit `invoke`.

```
fun someOperations(  
    onStart: (() -> Unit)? = null,  
    onCompletion: (() -> Unit)? = null,  
) {  
    onStart?.invoke()  
    // ...  
    onCompletion?.invoke()  
}
```

A function type can be used wherever a type is expected. For example, in a class definition, a generic type argument, or a parameter definition.

```
class Button(val text: String, val onClick: () -> Unit)  
  
var listeners: List<(Action) -> Unit> = emptyList()  
  
fun setListener(listener: (Action) -> Unit) {  
    listeners = listeners + listener  
}
```

A function type can also be used as part of a function type definition.

- `() -> Unit) -> Unit` - a function type representing a function that expects a function type `() -> Unit` as an argument and returns nothing significant.
- `() -> () -> Unit` - a function type representing a function that expects no arguments and returns a function type `() -> Unit`.

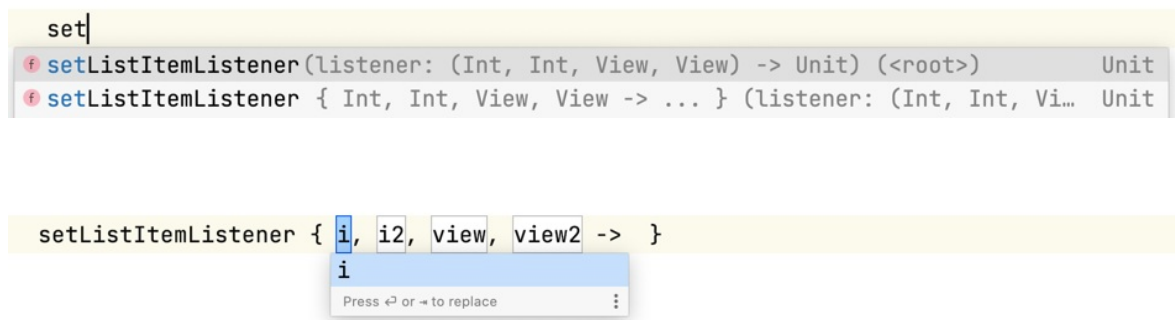
It is good to understand that a function type can include a function type, even though such function types are rarely useful.

Named parameters

Imagine a function type that expects many parameters, but it is unclear what every parameter means.

```
fun setListItemListener(  
    listener: (Int, Int, View, View) -> Unit  
) {  
    listeners = listeners + listener  
}
```

A user of such a function will likely be confused, and automatic name suggestions are not helpful at all.



That is why function types can suggest parameter names. We place a name before a parameter type and separate them with a colon from other declared parameters.

```
fun setListItemListener(  
    listener: (  
        position: Int,  
        id: Int,  
        child: View,  
        parent: View  
    ) -> Unit  
) {  
    listeners = listeners + listener  
}
```

Such names are visible in IntelliJ hints, and they have suggested names when we define a lambda expression for this type.


```

set
  f setListItemListener(listener: (Int, Int, View, View) -> Unit) (<root>) Unit
  f setListItemListener { position, id, child, parent -> ... } (listener: (Int, Int, View, View) -> Unit)

setListItemListener { position, id, child, parent -> ... }

```

Named parameters are only for developers' convenience, but they are not necessary from the technical point of view. However, it is good practice to add them if the parameters' meaning is unclear.

Type aliases

Function types can be long, especially when we use named arguments. In general, long types can be problematic, especially if they are repeated. Think of the `setListItemListener` example, where the same function type is repeated in the `listener` property and the `removeListItemListener` function.

```

private var listeners =
    emptyList<(Int, Int, View, View) -> Unit>()

fun setListItemListener(
    listener: (
        position: Int, id: Int,
        View, parent: View
    ) -> Unit
) {
    listeners = listeners + listener
}

fun removeListItemListener(
    listener: (Int, Int, View, View) -> Unit
) {
    listeners = listeners - listener
}

```

We define a type alias with the `typealias` keyword. We then specify a name, followed by the equals sign (`=`), and we then specify which type should stand behind this name. Defining a type alias is like giving someone a nickname. It is not really a new type: it's just a new way to reference the same type. Both types can be used interchangeably because types generated with type aliases are replaced with their definitions during compilation.

```
typealias Users = List<User>

fun updateUsers(users: Users) {}
// during compilation becomes
// fun updateUsers(users: List<User>) {}

fun main() {
    val users: Users = emptyList()
    // during compilation becomes
    // val users: List<User> = emptyList()

    val newUsers: List<User> = emptyList()
    updateUsers(newUsers) // acceptable
}
```

Type aliases can help us resolve name conflicts across libraries. For example, instead of the following code⁷:

```
import thirdparty.Name

class Foo {
    val name1: Name
    val name2: my.Name
}
```

We could use a type alias:

```
import my.Name

typealias ThirdPartyName = thirdparty.name

class Foo {
    val name1: ThirdPartyName
    val name2: Name
}
```

Be careful because type aliases do not protect our types from misuse. If you define different names for the same type, they can all be used interchangeably⁸.

```
// DON'T DO THAT! Misleading and false type safety
typealias Minutes = Int
typealias Seconds = Int

fun decideAboutTime(): Minutes = 10
fun setupTimer(time: Seconds) {
    /*...*/
}

fun main() {
    val time = decideAboutTime()
    setupTimer(time)
}
```

A function type is an interface

Under the hood, all function types are just interfaces with generic type parameters. This is why a class can implement a function type.

```
class OnClick : (Int) -> Unit {
    override fun invoke(viewId: Int) {
        // ...
    }
}

fun setListener(l: (Int) -> Unit) {
    /*...*/
}

fun main() {
    val onClick = OnClick()
    setListener(onClick)
}
```

We have learned something about function types, but we still do not know how to create objects of these types. This is what the following four chapters will be about, and we will start with the way that is the simplest, the oldest, and at the same time, the most forgotten: anonymous functions.

Anonymous functions

It's time to learn how to make an object that implements a function type. Readers who are familiar with Kotlin are now likely waiting for lambda expressions, but I will start with their predecessor: anonymous functions.

You can make an anonymous function by removing the name from a regular function definition. An anonymous function is an expression that returns an object of functional type. It does not define a regular function, so in the example below, to use the anonymous function I defined, I need to assign its result to a property.

```
// a regular function named `add1`  
fun add1(a: Int, b: Int) = a + b  
  
// an anonymous function stored in a property `add2`  
val add2 = fun(a: Int, b: Int): Int {  
    return a + b  
}
```

We can use single-expression or regular syntax when we define anonymous functions. Generic type parameters and default arguments are not supported.

```
val add2 = fun(a: Int, b: Int) = a + b
```

Generic type parameters and default arguments are not supported.

```
// Error! Generic anonymous functions are not supported  
val f = fun <T> (a: T): T = a // COMPILATION ERROR
```

The inferred type of add2 is (Int, Int) -> Int.

```
val add2 = fun(a: Int, b: Int) = a + b
```

add

add2 (Int, Int) -> Int
Press <?> to insert, => to replace Next Tip

In JavaScript, anonymous functions are also predecessors of lambda expressions (which are typically called arrow functions in the JavaScript community).

In the previous chapter, we presented a list of examples of function types. Here you can find an anonymous function for each of them:

```
data class User(val id: Int)
data class Name(val name: String)

fun main() {
    val cheer: () -> Unit = fun() {
        println("Hello")
    }
    cheer.invoke() // Hello
    cheer() // Hello

    val printNumber: (Int) -> Unit = fun(i: Int) {
        println(i)
    }
    printNumber.invoke(10) // 10
    printNumber(20) // 20

    val log: (String, String) -> Unit =
        fun(ctx: String, message: String) {
            println("[${ctx}] $message")
        }
    log.invoke("UserService", "Name changed")
    // [UserService] Name changed
    log("UserService", "Surname changed")
    // [UserService] Surname changed

    val makeAdmin: () -> User = fun() = User(id = 0)
    println(makeAdmin()) // User(id=0)

    val add: (String, String) -> String =
        fun(s1: String, s2: String): String {
            return s1 + s2
        }
    println(add.invoke("A", "B")) // AB
    println(add("C", "D")) // CD

    val toName: (String) -> Name =
        fun(name: String) = Name(name)
    val name: Name = toName("Cookie")
    println(name) // Name(name=Cookie)
}
```

An anonymous function specifies the type of the result and the parameters. It means that the variable type can be inferred, as in the examples below (we do not specify any type for cheer or printNumber variables because it is inferred).

```
val cheer = fun() {  
    println("Hello")  
}  
val printNumber = fun(i: Int) {  
    println(i)  
}  
val log = fun(ctx: String, message: String) {  
    println("[${ctx}] $message")  
}  
val makeAdmin = fun() = User(id = 0)  
val add = fun(s1: String, s2: String): String {  
    return s1 + s2  
}  
val toName = fun(name: String) = Name(name)
```

On the other hand, when parameter types can be inferred, anonymous functions do not need to define them:

```
val printNumber: (Int) -> Unit = fun(i) {  
    println(i)  
}  
val log: (String, String) -> Unit = fun(ctx, message) {  
    println("[${ctx}] $message")  
}  
val add: (String, String) -> String = fun(s1, s2): String {  
    return s1 + s2  
}  
val toName: (String) -> Name = fun(name) = Name(name)
```

Nowadays, anonymous functions are almost forgotten and rarely used. People prefer to use their convenient successor: lambda expressions. This is partly because lambda expressions are shorter and have better support and partly because only lambda expressions are suggested with hints in IntelliJ. So, let's finally talk about these famous lambda expressions.

Lambda expressions

Lambda expressions are a shorter alternative to anonymous functions. They are also used to define objects that represent functions. Both notations compile to the same result, but lambda expressions support more features (most of which will be presented in this chapter). In the end, lambda expressions are the most popular and idiomatic approach to create objects that represent functions, therefore understanding them is essential for using Kotlin's functional programming features.

An expression used to create an object representing a function is called a *function literal*, so both lambda expressions and anonymous functions are function literals.

Tricky braces

Lambda expressions are defined in braces (curly brackets). What is more, even just empty braces define a lambda expression.

```
fun main() {  
    val f: () -> Unit = {}  
    f()  
    // or f.invoke()  
}
```

But be careful because all braces that are not part of a Kotlin structure are lambda expressions (we can call them orphaned lambda expressions). This can lead to a lot of problems. Take a look at the following example: What does the following main function print?

```
fun main() {  
    {  
        println("AAA")  
    }  
}
```

The answer is nothing. It creates a lambda expression that is never invoked. Another question: What does the following produce function return?

```
fun produce() = { 42 }
```

```
fun main() {  
    println(produce()) // ???  
}
```

Counterintuitively, it is not 42. Braces are not a part of single-expression function notation. The produce function returns a lambda expression of type `() -> Int`, so the above code on JVM should print something like `Function0<java.lang.Integer>`, or just `() -> Int`. To fix this code, we should either call the produced function or remove the braces inside the single-expression function definition.

```
fun produceFun() = { 42 }  
fun produceNum() = 42  
  
fun main() {  
    val f = produceFun()  
    println(f()) // 42  
    println(produceFun()) // 42  
    println(produceFun().invoke()) // 42  
  
    println(produceNum()) // 42  
}
```

Parameters

If a lambda expression has parameters, we need to separate the content of the braces with an arrow `->`. Before the arrow, we specify parameter names and types, separated by commas. After the arrow, we specify the function body.

```
fun main() {  
    val printTimes = { text: String, times: Int ->  
        for (i in 1..times) {  
            print(text)  
        }  
    } // the type is (text: String, times: Int) -> Unit  
    printTimes("Na", 7) // NaNaNaNaNaNaN  
    printTimes.invoke("Batman", 2) // BatmanBatman  
}
```

Most often, we define lambda expressions as arguments to some functions. Regular functions need to define their parameter types, based on which lambda expression parameter types can be inferred.

```
fun setOnClickListener(listener: (View, Click) -> Unit) {}  
  
fun main() {  
    setOnClickListener({ view, click ->  
        println("Clicked")  
    })  
}
```

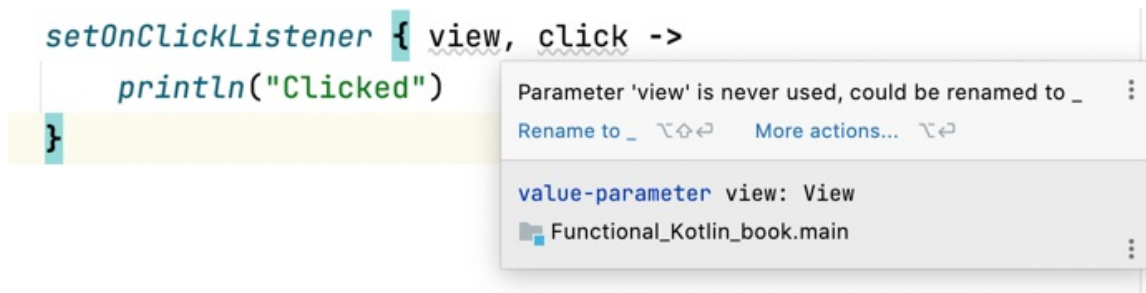


```
    })  
}
```

If we want to ignore a parameter, we can use underscore (`_`) instead of its name. This is a placeholder that shows that this parameter is ignored.

```
setOnClickListener({ _, _ ->  
    println("Clicked")  
})
```

IDEA IntelliJ suggests transforming unused parameters into underscores.



We can also use destructuring when defining a lambda expression's parameters⁹.

```
data class User(val name: String, val surname: String)  
data class Element(val id: Int, val type: String)  
  
fun setOnClickListener(listener: (User, Element) -> Unit) {}  
  
fun main() {  
    setOnClickListener({ (name, surname), (id, type) ->  
        println(  
            "User $name $surname clicked " +  
            "element $id of type $type"  
        )  
    })  
}
```

Trailing lambdas

Kotlin introduced a convention: if we call a function whose last parameter is of a functional type, we can define a lambda expression **outside** the parentheses. This feature is known as *trailing lambda*. If it is the only argument we define, we can skip the parameter bracket and just define a lambda expression. Take a look at these examples.

```
inline fun <R> run(block: () -> R): R = block()

inline fun repeat(times: Int, block: (Int) -> Unit) {
    for (i in 0 until times) {
        block(i)
    }
}

fun main() {
    run({ println("A") }) // A
    run() { println("A") } // A
    run { println("A") } // A

    repeat(2, { print("B") }) // BB
    println()
    repeat(2) { print("B") } // BB
}
```

In the example above, both `run` and `repeat` are simplified functions from the standard library.

This means that we can call our `setOnClickListener` in the following way:

```
setOnClickListener { _, _ ->
    println("Clicked")
}
```

Remember `sum` and `product` from the introduction? We have implemented them using the `fold` function with a trailing lambda.

```
fun sum(a: Int, b: Int) =
    (a..b).fold(0) { acc, i -> acc + i }

fun product(a: Int, b: Int) =
    (a..b).fold(1) { acc, i -> acc * i }
```

But be careful because this convention works only for the last parameter. Take a look at the snippet below and guess what will be printed.

```
fun call(before: () -> Unit = {}, after: () -> Unit = {}) {
    before()
    print("A")
    after()
}

fun main() {
    call({ print("C") })
    call { print("B") }
}
```

The answer is “CAAB”. Tricky, isn’t it? If you call a function with more than one functional parameter, use the *named argument convention*[10](#).

```
fun main() {
    call(before = { print("C") })
    call(after = { print("B") })
}
```

Result values

Lambda expressions were initially designed to implement short functions. Their bodies were designed to be minimalistic; therefore, inside them, instead of using an explicit return, the result of the last statement is returned. For example, { 42 } returns 42 because this number is the last statement. { 1; 2 } returns 2. { 1; 2; 3 } returns 3.

```
fun main() {
    val f = {
        10
        20
        30
    }
    println(f()) // 30
}
```

In most use cases, this is really convenient, but what can we do if we need to finish our function prematurely? A simple return will not help (for reasons we will cover later).

```
fun main() {
    onUserChanged { user ->
        if (user == null) return // compilation error
        cheerUser(user)
    }
}
```

To use return in the middle of a lambda expression, we need to use a label that marks this lambda expression. We specify a label before a lambda expression by using the label name followed by @. Then, we can return from this lambda expression calling return on the defined label.

```
fun main() {
    onUserChanged someLabel@{ user ->
        if (user == null) return@someLabel
        cheerUser(user)
    }
}
```

To simplify this process, there is a convention: if a lambda expression is used as an argument to a function, the name of this function becomes its default label. So, without specifying a label, we could return from the lambda using the onUserChanged label in the example above.

```
fun main() {
    onUserChanged { user ->
        if (user == null) return@onUserChanged
        cheerUser(user)
    }
}
```

This is how we typically return from a lambda expression prematurely. In theory, specifying custom labels might be useful for returning from outer lambda expressions.

```
fun main() {
    val magicSquare = listOf(
        listOf(2, 7, 6),
        listOf(9, 5, 1),
        listOf(4, 3, 8),
    )
    magicSquare.forEach line@ { line ->
        var sum = 0
        line.forEach { elem ->
            sum += elem
            if (sum == 15) {
                return@line
            }
        }
        print("Line $line not correct")
    }
}
```

However, in practice, this is not only rare but also considered a poor practice^{[11](#)}, because it violates the usual encapsulation rules. This is similar to throwing an exception from an inner function, but in this case the caller can at least decide to

catch and react. However, returning from an outer label completely ignores the intermediate callers.

Lambda expression examples

The previous chapter showed a set of functions implemented with *anonymous functions*. This is how they might be defined with *lambda expressions*:

```
fun main() {
    val cheer: () -> Unit = {
        println("Hello")
    }
    cheer.invoke() // Hello
    cheer() // Hello

    val printNumber: (Int) -> Unit = { i: Int ->
        println(i)
    }
    printNumber.invoke(10) // 10
    printNumber(20) // 20

    val log: (String, String) -> Unit =
        { ctx: String, message: String ->
            println("[${ctx}] $message")
        }
    log.invoke("UserService", "Name changed")
    // [UserService] Name changed
    log("UserService", "Surname changed")
    // [UserService] Surname changed

    data class User(val id: Int)

    val makeAdmin: () -> User = { User(id = 0) }
    println(makeAdmin()) // User(id=0)

    val add: (String, String) -> String =
        { s1: String, s2: String -> s1 + s2 }
    println(add.invoke("A", "B")) // AB
    println(add("C", "D")) // CD

    data class Name(val name: String)

    val toName: (String) -> Name =
        { name: String -> Name(name) }
    val name: Name = toName("Cookie")
    println(name) // Name(name=Cookie)
}
```

A lambda expression can specify the types of parameters, so the result type can be inferred:

```
val cheer = {
    println("Hello")
}
```

```
}  
val printNumber = { i: Int ->  
    println(i)  
}  
val log = { ctx: String, message: String ->  
    println("[${ctx}] $message")  
}  
val makeAdmin = { User(id = 0) }  
val add = { s1: String, s2: String -> s1 + s2 }  
val toName = { name: String -> Name(name) }
```

On the other hand, when parameter types can be inferred, lambda expressions do not need to define them:

```
val printNumber: (Int) -> Unit = { i ->  
    println(i)  
}  
val log: (String, String) -> Unit = { ctx, message ->  
    println("[${ctx}] $message")  
}  
val add: (String, String) -> String = { s1, s2 -> s1 + s2 }  
val toName: (String) -> Name = { name -> Name(name) }
```

An implicit name for a single parameter

When a lambda expression has **exactly one parameter**, we can reference it using the `it` keyword instead of specifying its name. Since the type of `it` cannot be specified explicitly, it needs to be inferred. Despite this, it is still a very popular feature.

```
val printNumber: (Int) -> Unit = { println(it) }  
val toName: (String) -> Name = { Name(it) }  
  
// Real-life example, functions will be explained later  
val newItemAdapters = news  
    .filter { it.visible }  
    .sortedByDescending { it.publishedAt }  
    .map { it.toNewsItemAdapter() }
```

Closures

A lambda expression can use and modify variables from the scope where it is defined.

```
fun makeCounter(): () -> Int {
    var i = 0
    return { i++ }
}

fun main() {
    val counter1 = makeCounter()
    val counter2 = makeCounter()

    println(counter1()) // 0
    println(counter1()) // 1
    println(counter2()) // 0
    println(counter1()) // 2
    println(counter1()) // 3
    println(counter2()) // 1
}
```

A lambda expression that refers to an object defined outside its scope, like the lambda expression in the above example that refers to the local variable `i`, is called a *closure*.

Lambda expressions vs anonymous functions

Let's compare lambda expressions to anonymous functions. They are both *function literals*, i.e., structures that create an object representing a function. Under the hood, their efficiency is the same. So, when should we choose one over the other? Take a look at the processor variable below, which is defined using both approaches.

```
val processor = label@{ data: String ->
    if (data.isEmpty()) {
        return@label null
    }

    data.uppercase()
}

val processor = fun(data: String): String? {
    if (data.isEmpty()) {
        return null
    }
}
```

```
    return data.uppercase()  
}
```

Lambda expressions are shorter but also less explicit. They return the last expression without an explicit return keyword. To use return we need to have a label.

Anonymous functions are longer, but it is clear that they define a function. They use an explicit return and must specify the result type.

Lambda expressions were mainly designed for single-expression functions, and the documentation suggests using anonymous functions for longer bodies. Although developers used to use lambda expressions practically everywhere, nowadays anonymous functions seem nearly forgotten.

The popularity of lambda expressions is supported by the additional features: trailing lambda, an implicit name for a single parameter, and non-local return (this will be explained later). So, I understand if you decide to forget about anonymous functions and use lambda expressions everywhere. Many developers have already done this.

However, before we close this discussion, we must introduce one more approach for creating objects representing functions. This will be a serious competitor to lambda expressions because it is shorter and has a good-looking, functional style. Let's talk about function references.

Function references

When we need a function as an object, we can create it with a lambda expression, but we can also reference an existing function. The second approach is often shorter and more convenient. In this chapter, we will learn about the different kinds of function references, and we will see how they might be used in practice.

In our examples, we will reference the functions from the following code. These will be the basic functions in this chapter.

```
data class Complex(val real: Double, val imaginary: Double) {
    fun doubled(): Complex =
        Complex(this.real * 2, this.imaginary * 2)
    fun times(num: Int) =
        Complex(real * num, imaginary * num)
}

fun zeroComplex(): Complex = Complex(0.0, 0.0)

fun makeComplex(
    real: Double = 0.0,
    imaginary: Double = 0.0
) = Complex(real, imaginary)

fun Complex.plus(other: Complex): Complex =
    Complex(real + other.real, imaginary + other.imaginary)
fun Int.toComplex() = Complex(this.toDouble(), 0.0)
```

Top-level functions references

We use `::` and a function name to reference a top-level function^{[12](#)}. Function references are part of the Kotlin reflection API and support introspection. If you include the `kotlin-reflect` dependency in your project, you can use a function reference to check if the referenced function has the `open` modifier, what annotation it has, etc.^{[13](#)}

```
fun add(a: Int, b: Int) = a + b

fun main() {
    val f = ::add // function reference
    println(f.isOpen) // false
    println(f.visibility) // PUBLIC
    // The above statements require `kotlin-reflect`
    // dependency
}
```

However, function references also implement function types and can be used as function literals. Such usages are not considered “real” reflection and introduce no performance overhead compared to lambda expressions¹⁴.

```
fun add(a: Int, b: Int) = a + b

fun main() {
    val f: (Int, Int) -> Int = ::add
    // an alternative to:
    // val f: (Int, Int) -> Int = { a, b -> add(a, b) }
    println(f(10, 20)) // 30
}
```

Notice that `add` is a function with two parameters of type `Int`, and result type `Int`, so its reference function type is `(Int, Int) -> Int`.

Let’s get back to our basic functions. Can you guess what the function type of `zeroComplex` and `makeComplex` should be?

A function type specifies the parameters and the result type. The function `zeroComplex` has no parameters, and its result type is `Complex`, so the function type of its function reference is `() -> Complex`. The function `makeComplex` has two parameters of type `Double`, and its result type is `Complex`, so the function type of its function reference is `(Double, Double) -> Complex`.

```
fun zeroComplex(): Complex = Complex(0.0, 0.0)

fun makeComplex(
    real: Double = 0.0,
    imaginary: Double = 0.0
) = Complex(real, imaginary)

data class Complex(val real: Double, val imaginary: Double)

fun main() {
    val f1: () -> Complex = ::zeroComplex
    println(f1()) // Complex(real=0.0, imaginary=0.0)

    val f2: (Double, Double) -> Complex = ::makeComplex
    println(f2(1.0, 2.0)) // Complex(real=1.0, imaginary=2.0)
}
```

Since the function `makeComplex` has default arguments for its parameters, it should also implement `(Double) -> Complex` and `() -> Complex`. Limited support for such behavior was introduced in Kotlin 1.4, but a reference must still be used as an argument.

```
fun produceComplex1(producer: ()->Complex) {}  
produceComplex1(::makeComplex)  
fun produceComplex2(producer: (Double)->Complex) {}  
produceComplex2(::makeComplex)
```

Method references

When you reference a method, you need to start with a type, followed by `::` and the method name. Every method needs a receiver, namely the object on which the function should be called. Function references expect it as the first parameter. Take a look at the example below.

```
data class Number(val num: Int) {  
    fun toFloat(): Float = num.toFloat()  
    fun times(n: Int): Number = Number(num * n)  
}  
  
fun main() {  
    val numberObject = Number(10)  
    // member function reference  
    val float: (Number) -> Float = Number::toFloat  
    // `toFloat` has no parameters, but its function type  
    // needs a receiver of type `Number`  
    println(float(numberObject)) // 10.0  
    val multiply: (Number, Int) -> Number = Number::times  
    println(multiply(numberObject, 4)) // Number(num = 40.0)  
    // `times` has one parameter of type `Int`, but its  
    // function type also needs a receiver of type `Number`  
}
```

The `toFloat` function has no explicit parameters, but its function reference requires a receiver of type `Number`. The `times` function has only one explicit parameter of type `Int`, but it also requires another one for the receiver.

Do you remember `sum` and `product` from the introduction? We implemented them using lambda expressions, but we could also have used method references.

```
fun sum(a: Int, b: Int) =  
    (a..b).fold(0, Int::plus)  
  
fun product(a: Int, b: Int) =  
    (a..b).fold(1, Int::times)
```

Getting back to our basic functions, can you deduce the function type of `Complex::doubled` and `Complex::times`?

`doubled` has no explicit parameters, a receiver of type `Complex`, and the result type is `Complex`; therefore, the function type of its function reference is `(Complex) -> Complex`. `times` has an explicit parameter of type `Int`, a receiver

of type Complex, and the result type is Complex; therefore, the function type of its function reference is (Complex, Int) -> Complex.

```
data class Complex(val real: Double, val imaginary: Double) {
    fun doubled(): Complex =
        Complex(this.real * 2, this.imaginary * 2)
    fun times(num: Int) =
        Complex(real * num, imaginary * num)
}

fun main() {
    val c1 = Complex(1.0, 2.0)

    val f1: (Complex) -> Complex = Complex::doubled
    println(f1(c1)) // Complex(real=2.0, imaginary=4.0)

    val f2: (Complex, Int) -> Complex = Complex::times
    println(f2(c1, 4)) // Complex(real=4.0, imaginary=8.0)
}
```

Extension function references

We can reference extension functions in the same way as member functions. Their function types are also analogous.

```
data class Number(val num: Int)

fun Number.toFloat(): Float = num.toFloat()
fun Number.times(n: Int): Number = Number(num * n)

fun main() {
    val num = Number(10)
    // extension function reference
    val float: (Number) -> Float = Number::toFloat
    println(float(num)) // 10.0
    val multiply: (Number, Int) -> Number = Number::times
    println(multiply(num, 4)) // Number(num = 40.0)
}
```

Can you now guess the function type of Complex::plus and Int::toComplex from our basic functions?

plus has a Complex parameter, a receiver of type Complex, and it returns Complex; therefore, the function type of its function reference is (Complex, Complex) -> Complex. The toComplex function has no parameters, a receiver of type Int, and it returns Complex; therefore, the function type of its function reference is (Int) -> Complex.

```
data class Complex(val real: Double, val imaginary: Double)
```

```

fun Complex.plus(other: Complex): Complex =
    Complex(real + other.real, imaginary + other.imaginary)

fun Int.toComplex() = Complex(this.toDouble(), 0.0)

fun main() {
    val c1 = Complex(1.0, 2.0)
    val c2 = Complex(4.0, 5.0)

    // extension function reference
    val f1: (Complex, Complex) -> Complex = Complex::plus
    println(f1(c1, c2)) // Complex(real=5.0, imaginary=7.0)

    val f2: (Complex, Int) -> Complex = Complex::times
    println(f2(c1, 4)) // Complex(real=4.0, imaginary=8.0)
}

```

Method references and generic types

We reference a method on a type, not a property. So, if you want to reference `sum`, which is an extension function on the type `List<Int>`, you need to use `List<Int>::sum`. If you want to reference `isNullOrBlank`, which is an extension property on the type `String?`, you should use `String?::isNullOrBlank`¹⁵.

```

class TeamPoints(val points: List<Int>) {
    fun <T> calculatePoints(operation: (List<Int>) -> T): T =
        operation(points)
}

fun main() {
    val teamPoints = TeamPoints(listOf(1, 3, 5))

    val sum = teamPoints
        .calculatePoints(List<Int>::sum)
    println(sum) // 9

    val avg = teamPoints
        .calculatePoints(List<Int>::average)
    println(avg) // 3.0

    val invalid = String?::isNullOrBlank
    println(invalid(null)) // true
    println(invalid(" ")) // true
    println(invalid("AAA")) // false
}

```

When you reference a method from a generic class, its type arguments need to be explicit. So, in the example below, to reference the `unbox` method, we need to use `Box<String>::unbox`, and the `Box::unbox` notation is not acceptable.

```

class Box<T>(private val value: T) {
    fun unbox(): T = value
}

```

```

}

fun main() {
    val unbox = Box<String>::unbox
    val box = Box("AAA")
    println(unbox(box)) // AAA
}

```

Bounded function references

We have learned how to reference a method on a type, but there is also another option: we can reference a method on an object instance. Such references are called **bounded function references**.

```

data class Number(val num: Int) {
    fun toFloat(): Float = num.toFloat()
    fun times(n: Int): Number = Number(num * n)
}

fun main() {
    val num = Number(10)
    // bounded function reference
    val getNumAsFloat: () -> Float = num::toFloat
    // There is no need for receiver type in function type,
    // because reference is already bound to an object
    println(getNumAsFloat()) // 10.0
    val multiplyNum: (Int) -> Number = num::times
    println(multiplyNum(4)) // Number(num = 40.0)
}

```

Notice that the function type of `num::toFloat` is `() -> Float` in the example above. We have previously learned that the function type of `Number::toFloat` is `(Number) -> Float`; therefore, in the regular method reference notation, the receiver type will be in the first position. In bounded function references, the receiver object is already provided in the reference, so there is no need to specify it additionally.

Getting back to our basic functions, can you deduce the type of the bounded references to `doubled`, `times`, `plus`, and `toComplex`? The answers can be found in the code below.

```

data class Complex(val real: Double, val imaginary: Double) {
    fun doubled(): Complex =
        Complex(this.real * 2, this.imaginary * 2)
    fun times(num: Int) =
        Complex(real * num, imaginary * num)
}

fun Complex.plus(other: Complex): Complex =

```

```

    Complex(real + other.real, imaginary + other.imaginary)
fun Int.toComplex() = Complex(this.toDouble(), 0.0)

fun main() {
    val c1 = Complex(1.0, 2.0)

    val f1: () -> Complex = c1::doubled
    println(f1()) // Complex(real=2.0, imaginary=4.0)

    val f2: (Int) -> Complex = c1::times
    println(f2(17)) // Complex(real=17.0, imaginary=34.0)

    val f3: (Complex) -> Complex = c1::plus
    println(f3(Complex(12.0, 13.0)))
    // Complex(real=13.0, imaginary=15.0)

    val f4: () -> Complex = 42::toComplex
    println(f4()) // Complex(real=42.0, imaginary=0.0)
}

```

Bounded function references also work on object expressions and object declarations^{[16](#)}.

```

object SuperUser {
    fun getId() = 0
}

fun main() {
    val myId = SuperUser::getId
    println(myId()) // 0

    val obj = object {
        fun cheer() {
            println("Hello")
        }
    }
    val f = obj::cheer
    f() // Hello
}

```

I find bounded function references especially useful when using libraries like RxJava or Reactor, where we often set handlers for different kinds of events. Small, simple handlers can be defined using lambda expressions. However, extracting them as member functions and setting bounded function references as handlers is a good idea for larger and more complicated handlers.

```

class MainPresenter(
    private val view: MainView,
    private val repository: MarvelRepository
) : BasePresenter() {

    fun onViewCreated() {
        subscriptions += repository.getAllCharacters()
    }
}

```



```

        .applySchedulers()
        .subscribeBy(
            onSuccess = this::show,
            onError = view::showError
        )
    }

    fun show(items: List<MarvelCharacter>) {
        // ...
        view.show(items)
    }
}

```

Using the bounded function reference is really convenient in this case because handlers need to have access to the `MainPresenter` properties, but `getAllCharacters` should not know anything about this.

A bounded function reference on the receiver (`this`) can be used implicitly, so `this::show` can also be replaced with `::show`.

Constructor references

A constructor is also considered a function in Kotlin. We call and reference it in the same way as all other functions. This means that to reference the `Complex` class constructor, we need to use `::Complex`. The constructor reference has the same parameters as the constructor it references, and its result type is the type of the class whose constructor it is.

```

data class Complex(val real: Double, val imaginary: Double)

fun main() {
    // constructor reference
    val produce: (Double, Double) -> Complex = ::Complex
    println(produce(1.0, 2.0))
    // Complex(real=1.0, imaginary=2.0)
}

```

I find constructor references useful when I map elements from one type to another using a constructor. This could be especially useful for mapping to wrapper classes. However, mapping using a constructor should not be used too often as we prefer factory functions (like conversion functions) instead of secondary constructors^{[17](#)}.

```

class StudentId(val value: Int)
class UserId(val value: Int) {
    constructor(studentId: StudentId) : this(studentId.value)
}

```

```
fun main() {  
    val ints: List<Int> = listOf(1, 1, 2, 3, 5, 8)  
    val studentIds: List<StudentId> = ints.map(::StudentId)  
    val userIds: List<UserId> = studentIds.map(::UserId)  
}
```

Bounded object declaration references

One of the motivations for the introduction of bounded function references was to make a simple way to reference object declaration methods^{[18](#)}. Every object declaration is a singleton, so its name serves as the only object reference. Thanks to the bounded function reference feature, we can reference object declaration methods using its name, followed by two colons (: :), then the method name.

```
object Robot {  
    fun moveForward() {  
        /*...*/  
    }  
    fun moveBackward() {  
        /*...*/  
    }  
}  
  
fun main() {  
    Robot.moveForward()  
    Robot.moveBackward()  
  
    val action1: () -> Unit = Robot::moveForward  
    val action2: () -> Unit = Robot::moveBackward  
}
```

Companion objects are also a form of object declaration. However, referencing their methods using the class name is not enough. We need to use the real companion name, which is Companion by default.

```
class Drone {  
    fun setOff() {}  
    fun land() {}  
  
    companion object {  
        fun makeDrone(): Drone = Drone()  
    }  
}  
  
fun main() {  
    val maker: () -> Drone = Drone.Companion::makeDrone  
}
```

Function overloading and references

Kotlin allows function overloading, which means defining multiple functions with the same name. During compilation, the Kotlin compiler decides which function should be used based on the types of arguments used.

```
fun foo(i: Int) = 1
fun foo(str: String) = "AAA"

fun main() {
    println(foo(123)) // 1
    println(foo("")) // AAA
}
```

The same logic is used when we use function references. The compiler determines which function should be chosen based on the expected type. Without a specified type, our code will not compile due to ambiguity.

```
fun foo(i: Int) = 1
fun foo(str: String) = "AAA"

fun main() {
    val f = ::foo
}
```

Overload resolution ambiguity. All these functions match.

- **public fun foo(i: Int): Int** defined in root package in file Playground.kt
- **public fun foo(str: String): String** defined in root package in file Playground.kt

No documentation found.

Therefore, when we eliminate ambiguity with a type, everything will be correctly determined and resolved.

```
fun foo(i: Int) = 1
fun foo(str: String) = "AAA"

fun main() {
    val fooInt: (Int) -> Int = ::foo
    println(fooInt(123)) // 1
    val fooStr: (String) -> String = ::foo
    println(fooStr("")) // AAA
}
```

The same is true when we have multiple constructors.

```
class StudentId(val value: Int)
data class UserId(val value: Int) {
    constructor(studentId: StudentId) : this(studentId.value)
}

fun main() {
    val intToUserId: (Int) -> UserId = ::UserId
    println(intToUserId(1)) // UserId(value=1)

    val studentId = StudentId(2)
    val studentIdToUserId: (StudentId) -> UserId = ::UserId
    println(studentIdToUserId(studentId)) // UserId(value=2)
}
```

Property references

A property can be considered as a getter or as a getter and a setter. That is why its reference implements the getter function type.

```
data class Complex(val real: Double, val imaginary: Double)

fun main() {
    val c1 = Complex(1.0, 2.0)
    val c2 = Complex(3.0, 4.0)

    // property reference
    val getter: (Complex) -> Double = Complex::real

    println(getter(c1)) // 1.0
    println(getter(c2)) // 3.0

    // bounded property reference
    val c1ImgGetter: () -> Double = c1::imaginary
    println(c1ImgGetter()) // 2.0
}
```

For var, you can reference the setter using the setter property from the property reference, but this requires `kotlin-reflect`; therefore, I recommend avoiding this approach because it might impact your code's performance.

There are many kinds of references. Some developers like using them, while others avoid them. Anyway, it is good to know how function references look and behave. It is worth practicing them as they can help make our code more elegant in applications where functional programming concepts are widely used.

SAM Interface support in Kotlin

Many languages do not support function types. Instead, they often use interfaces with a single method. Such interfaces are known as SAM (Single-Abstract Method) interfaces. Here is an example of a SAM interface that is used to express an object that specifies behavior that should be invoked when a view is clicked:

```
interface OnClickListener {  
    fun onClick(view: View)  
}
```

When a function expects a SAM interface, we must pass an object that implements this interface.

```
fun setOnClickListener(listener: OnClickListener) {  
    //...  
}  
  
setOnClickListener(object : OnClickListener {  
    override fun onClick(view: View) {  
        // ...  
    }  
})
```

Support for Java SAM interfaces in Kotlin

In Kotlin, we prefer to use function types instead of SAM interfaces. They are better conceptually and are more convenient in use. An object that implements a function type can be created with a lambda expression, an anonymous function, a function reference, etc. The problems start when we need to interoperate with other languages, like Java.

Java does not have a direct analog of Kotlin function types, so its libraries operate on SAM interfaces. This is very important because on Kotlin/JVM, we still highly depend on Java libraries. Creating an object for each SAM that is required by libraries (listeners, watchers, observers, etc.) would be a huge inconvenience, which is why Kotlin has special support for Java SAM interfaces:

- whenever a Java SAM interface is expected as an argument, a matching function type can be used instead,
- Java SAM interfaces have a fake constructor that lets us create them with lambda expressions.

Take a look at the code below. The function `setOnSwipeListener` expects an object of type `OnSwipeListener`, and `OnSwipeListener` is an interface with a single abstract method (SAM). Without any special support, we would need to create an instance of a class implementing this interface. Thanks to the support, we can pass a lambda expression as an argument instead. We can also create an object that implements `OnSwipeListener` using a fake constructor:
`OnSwipeListener` name and lambda expression.

```
// OnSwipeListener.java
public interface OnSwipeListener {
    void onSwipe();
}

// ListAdapter.java
public class ListAdapter {

    public void setOnSwipeListener(OnSwipeListener listener) {
        // ...
    }
}

// kotlin
val adapter = ListAdapter()
adapter.setOnSwipeListener { /*...*/ }
val listener = OnSwipeListener { /*...*/ }
adapter.setOnSwipeListener(listener)
adapter.setOnSwipeListener(fun() { /*...*/ })
adapter.setOnSwipeListener(::someFunction)
```

Notice that this convention works only for SAM interfaces defined in Java; by default, it will not work for SAM interfaces defined in Kotlin.

This support gives us a lot of convenience when we use Java libraries in Kotlin, but not the other way around. To better support using Kotlin code in Java, we need to use *functional interfaces*.

Functional interfaces

Creating Kotlin function types in Java is problematic. Under the hood, Kotlin function types are translated to `FunctionN` interfaces (where `N` is the number of

parameters). If these interfaces declare `Unit` as a result type, it needs to be returned explicitly, which is quite annoying.

```
// kotlin
fun setOnClickListener(listener: (Action) -> Unit) {
    //...
}

// Java before version 8
setOnClickListener(new Function1<Action, Unit>() {
    @Override
    public Unit invoke(Action action) {
        // Some actions
        return Unit.INSTANCE;
    }
});

// Java since version 8
setOnClickListener(action -> {
    // Some actions
    return Unit.INSTANCE;
});
```

Moreover, the generic `invoke` method is reasonable in Kotlin, where it is often called implicitly but might not be a good fit for all Java cases. For our listener, `onClick` would be a better name.

```
Function1<Action, Unit> listener = action -> {
    // Some actions
    return Unit.INSTANCE;
};
listener.invoke(new Action());
```

To solve these problems, Kotlin introduced *functional interfaces*. They are defined the same way as regular interfaces, but they are marked with the `fun` modifier and must have just a single abstract method.

```
fun interface OnClick {
    fun onClick(view: View)
}
```

They can be used like regular interfaces, which makes their Java usage more natural, also Kotlin supports automatic conversion from functional types to functional interfaces.

```
fun setOnClickListener(listener: OnClick) {
    //...
}

// Kotlin usage
```



```

setOnClickListener { /*...*/ }
val listener = OnClickListener { /*...*/ }
setOnClickListener(listener)
setOnClickListener(fun(view) { /*...*/ })
setOnClickListener(::someFunction)
// ...

// Java usage before version 8
setOnClickListener(new OnClickListener() {
    @Override
    public void onClick(@NotNull View view) {
        /*...*/
    }
});

// Java usage after version 8
setOnClickListener(view -> {
    /*...*/
});

```

Functional interfaces also allow non-abstract functions to be added and other interfaces to be implemented.

```

interface ElementListener<T> {
    fun invoke(element: T)
}

fun interface OnClickListener : ElementListener<View> {
    fun onClick(view: View)

    fun invoke(element: View) {
        onClick(element)
    }
}

```

Overall, the main reasons to prefer functional interfaces over function types are:

- Java interoperability,
- optimization for primitive types,
- when we need to not only represent a function but also to express a concrete contract.

If there is no good reason to use functional interfaces, prefer plain function types because they are the most basic way to express what we expect from a function in this position.

Inline functions

The idea of using functions like objects, which lies in the foundations of functional programming, has been known for years. It was one of the selling points of LISP, which was developed in the late 1950s.

Since the early days of the Java community, there have been discussions about supporting this. The opponents argued that using functions as objects should not be supported because it would lead to decreased efficiency. To understand this argument, look at the following code, and assume that `students` is a huge collection.

```
fun <T, R> Iterable<T>.fold(
    initial: R,
    operation: (acc: R, T) -> R
): R {
    var accumulator = initial
    for (element in this) {
        accumulator = operation(accumulator, element)
    }
    return accumulator
}

fun main() {
    val points = students.fold(0) { acc, s -> acc + s.points }
    println(points)
}
```

A lambda expression creates an object, so on JVM, it creates a class, while in JS, it creates a function, etc. Every function is a form of a boundary. In our case, with every student, our execution needs to jump inside it and then back to the `forEach`. This generates a small cost, but it's still a cost.

This led many developers to argue that they prefer Java not to support this so that developers are forced to make code that is (slightly) more efficient:

```
fun main() {
    var points = 0
    for (student in students) {
        points += student.points
    }
    println(points)
}
```

It might be efficient, but we often need to repeat the same algorithms again and again, so it is not effective.

However, this has always been a false dichotomy. We can have both maximum efficiency and the convenience of passing functions as arguments. We just need to use inline functions to avoid the overhead of invoking lambda expressions.

Inline functions

When we place the `inline` modifier before a function, this function will **not** be called like all others. Instead, its body will replace its usages (calls) during compilation.

The simplest example is the `print` function from Kotlin stdlib. In JVM, it calls `System.out.print`. Since `print` is an inline function, all its usages during compilation are replaced with its body, so the `print` call is replaced with a `System.out.print` call.

```
inline fun print(message: Any?) {  
    System.out.print(message)  
}  
  
fun main() {  
    print("A")  
    print("B")  
    print("C")  
}  
  
// under the hood becomes  
fun main() {  
    System.out.print("A")  
    System.out.print("B")  
    System.out.print("C")  
}
```

Inline function calls in Kotlin are replaced with the bodies of these functions. In these bodies, parameter usages are replaced with associated argument expressions. There are a few advantages of this behavior:

1. Functions with functional parameter calls are more efficient when they are inline.
2. Non-local return is allowed.
3. A type argument can be reified.

Inline functions with functional parameters

When an inline function has parameters with functional types, they are also inlined by default. For instance, if we specify them with lambda expressions, these parameters' calls are replaced with the lambda expressions' bodies during compilation. For example, think about this repeat function call:

```
inline fun repeat(times: Int, action: (Int) -> Unit) {  
    for (index in 0 until times) {  
        action(index)  
    }  
}  
  
fun main() {  
    repeat(10) {  
        print(it)  
    }  
}
```

Since repeat is replaced with its body, and the lambda expression's body is inlined into its usage, the compiled code will be the equivalent of the following:

```
fun main() {  
    for (index in 0 until 10) {  
        print(index)  
    }  
}
```

If we get back to our fold example, it is enough to mark this function as inline to have both the performance benefit and the convenience of using functions as arguments.

```
inline fun <T, R> Iterable<T>.fold(  
    initial: R,  
    operation: (acc: R, T) -> R  
) : R {  
    var accumulator = initial  
    for (element in this) {  
        accumulator = operation(accumulator, element)  
    }  
    return accumulator  
}  
  
fun main() {  
    val points = students.fold(0) { acc, s -> acc + s.points }  
    println(points)  
}  
  
// under the hood compiled to  
fun main() {  
    var accumulator = 0
```

```
    for (element in students) {  
        accumulator = accumulator + element.points  
    }  
    val points = accumulator  
    println(points)  
}
```

The result is not only more efficient, but also fewer objects are allocated.

It is like having your cake and eating it! So, it's no wonder that it has become standard practice to mark top-level functions with functional parameters as inline. Here are some examples:

```
public inline fun <T, R> Iterable<T>.map(  
    transform: (T) -> R  
) : List<R> {  
    return mapTo(  
        ArrayList<R>(collectionSizeOrDefault(10)),  
        transform  
    )  
}  
  
public inline fun <T> Iterable<T>.filter(  
    predicate: (T) -> Boolean  
) : List<T> {  
    return filterTo(ArrayList<T>(), predicate)  
}
```

However, this is not the only advantage of inline functions. Lambda expressions used in such function calls do not create an object; as a result, they have capabilities that non-inline functions do not.

Non-local return

As we have already seen, when you need to repeat some operations a certain number of times, you can use the `repeat` function from the standard library.

```
fun main() {
    repeat(7) {
        print("Na")
    }
    println(" Batman")
}
// NaNaNaNaNaNaNa Batman
```

This `repeat` function call reminds me of built-in control structures, like the `for`-loop or the `if`-condition. It is amazing that we can create custom structures that are so close to essential language structures. Thanks to that, `repeat` can be a function and does not need to be built into the language.

However, lambda expressions have some limitations that the control structure does not have. For instance, you can use `return` inside a `for`-loop to return from the outer function.

```
fun main() {
    for (i in 0 until 10) {
        if (i == 4) return // Returns from main
        print(i)
    }
}
// 0123
```

This cannot be done in regular lambda expressions, because the body of their functions is a different function (on JVM, it is placed in a class that is generated for this lambda expression). However, we do not have this problem when a lambda expression is inlined; therefore, since `repeat` is an inline function, you can use `return` inside its lambda. This is called *non-local return*.

```
fun main() {
    repeat(10) { index ->
        if (index == 4) return // Returns from main
        print(index)
    }
}
// 0123
```

This works because repeat is inlined during compilation, so its lambda expression is also inlined; as a result, our code is compiled to the following:

```
fun main() {
    for (index in 0 until 10) {
        if (index == 4) return // Returns from main
        print(index)
    }
}
// 0123
```

Collection processing functions, like `forEach`, `map`, or `filter`, are inline functions too, and so they also support non-local return.

```
fun main() {
    (0 until 19).forEach { index ->
        if (index == 4) return // Returns from main
        print(index)
    }
}
// 0123
```

Crossinline and noinline

There are situations where we want to inline a function but, for some reason, we cannot inline all functions used as arguments. In such cases, we can use the following modifiers:

- `crossinline` - means that the function should be inlined, but the non-local return is not allowed. We use it when this function is used in another scope where the non-local return is not allowed, for instance, in another lambda that is not inlined.
- `noinline` - means that this argument should not be inlined at all. It is used mainly when we use this function as an argument to another function that is not inlined.

```
inline fun requestNewToken(
    hasToken: Boolean,
    crossinline onRefresh: () -> Unit,
    noinline onGenerate: () -> Unit
) {
    if (hasToken) {
        httpCall("get-token", onGenerate) // We must use
        // noinline to pass function as an argument to a
        // function that is not inlined
    } else {
        httpCall("refresh-token") {
            onRefresh() // We must use crossinline to
        }
    }
}
```

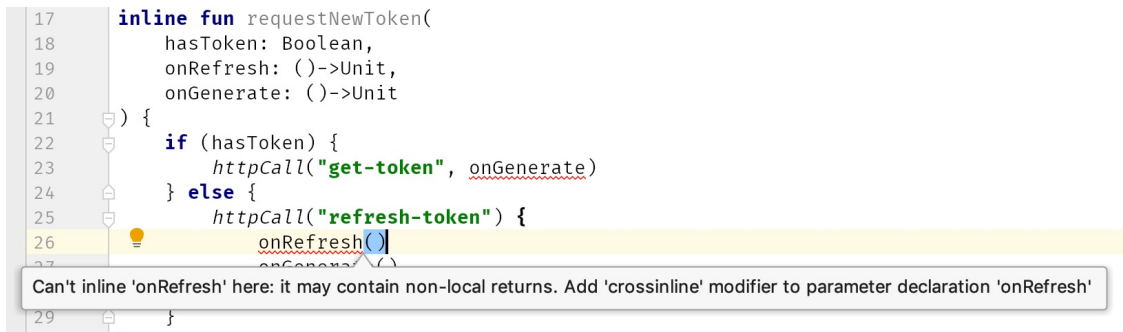
```

        // inline function in a context where
        // non-local return is not allowed
        onGenerate()
    }
}

fun httpCall(url: String, callback: () -> Unit) {
    /*...*/
}

```

It is good to know what the meanings of both modifiers are, but we can live without remembering them because IntelliJ IDEA suggests them when they are needed:



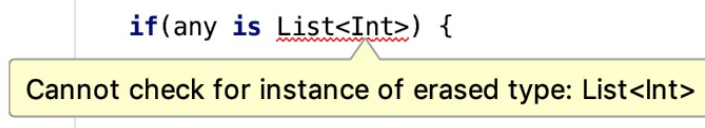
Reified type parameters

Older versions of Java do not have generics. They were introduced in 2004 in version J2SE 5.0, but they are still not present in the JVM bytecode. Therefore, generic types are erased during compilation. For instance, `List<Int>` compiles to `List` on JVM. This is why we cannot check if an object is `List<Int>`; we can only check if it is a `List` (which we express with `List<*>`).

```

any is List<Int> // Error
any is List<*> // OK

```



For the same reason, we cannot operate on a type argument:


```
fun <T> printTypeName() {
    print(T::class.simpleName) // ERROR
}

fun <T> isOfType(value: Any): Boolean =
    value is T // ERROR
```

We can overcome this limitation by making a function inline and marking type parameters with the reified modifier. An inline function call is replaced with its body, so usages of reified type parameters are replaced with type arguments¹⁹.

```
inline fun <reified T> printTypeName() {
    print(T::class.simpleName)
}

fun main() {
    printTypeName<Int>() // Int
    printTypeName<Char>() // Char
    printTypeName<String>() // String
}
```

During compilation, the body of printTypeName replaces the usages, and the type arguments (Int, Char and String) replace the reified type parameter T:

```
fun main() {
    print(Int::class.simpleName) // Int
    print(Char::class.simpleName) // Char
    print(String::class.simpleName) // String
}
```

reified is a useful modifier. For instance, it is used in the stdlib's filterIsInstance to filter only elements of a certain type:

```
class Worker
class Manager

val employees: List<Any> =
    listOf(Worker(), Manager(), Worker())

val workers: List<Worker> =
    employees.filterIsInstance<Worker>()
```

The reified modifier is also used in many libraries and util functions we define ourselves. The example below presents a common implementation of fromJsonOrNull that uses the Gson library.

```
inline fun <reified T : Any> String.fromJsonOrNull(): T? =
    try {
        gson.fromJson(this, T::class.java)
    } catch (e: JsonSyntaxException) {
```

```
        null
    }

// usage
val user: User? = userAsText.fromJsonOrNull()
```

Below are examples of how the Koin library uses reified functions to simplify both dependency injection and module declaration.

```
// Koin module declaration
val myModule = module {
    single { Controller(get()) } // get is reified
    single { BusinessService() }
}

// Koin injection
val service: BusinessService by inject()
// inject is reified
```

Reified parameters are really powerful; library creators should know them well because they can truly simplify passing or returning type parameters from generic functions.

Inline properties

Properties defined by accessors^[^07_2] are considered to be functions. In the end, such properties are compiled into functions.

```
val User.fullName: String
    get() = "$name $surname"

var User.birthday: Date
    get() = Date(birthdayMillis)
    set(value) {
        birthdayMillis = value.time
    }

// Under the hood is similar to:

fun getFullName(user: User) =
    "${user.name} ${user.surname}"

fun getBirthday(user: User) =
    Date(user.birthdayMillis)

fun setBirthday(user: User, value: Date) {
    user.birthdayMillis = value.time
}
```

This is why such properties can be marked with the `inline` modifier, which results in inlining the body of these properties into their usages.

```
class User(val name: String, val surname: String) {
    inline val fullName: String get() = "$name $surname"
}

fun main() {
    val user = User("A", "B")
    println(user.fullName) // A B
    // during compilation changes to
    println("${user.name} ${user.surname}")
}
```

Inline properties are not very popular as using them rarely has any impact on our code, however some library creators treat them as a low-level performance optimization.

Costs of the inline modifier

Inline is a useful modifier, but it should not be used everywhere due to its costs and limitations:

- Inline functions cannot use elements with more restrictive visibility.
- Inline functions cannot be recursive.
- Inline functions make our code grow.

In practice, the first one is the biggest problem. We cannot use private or internal functions or properties in public inline functions. In fact, an inline function cannot use anything with more restrictive visibility:

```
internal inline fun read() {
    val reader = Reader() // Error
    // ...
}

private class Reader {
    // ...
}
```

This is why inner classes cannot be used to hide implementation, therefore they are rarely used in classes.

Using inline functions

There are two main reasons for using inline functions:

- To improve the performance of functions with functional parameters; as a bonus, we also have support for non-local return.
- To support reified type parameters.

Inline functions are best suited to helper functions: either top-level functions or redundant methods that are used to simplify the use of other class methods.

Collection processing

One of the most useful applications of functional programming is collection processing: operations on collections of elements. This is generally one of the most common tasks in programming. This should come as no surprise. Just look at any advanced programming project, and you will likely see plenty of collections. An online shop? Products, sellers, delivery methods, payment methods... A bank application? Accounts, transactions, contacts, offers... it goes on and on. Consider internet search results, folder structures, task managers, topics, and answers on forums... Collections are everywhere in nearly all the services we use.

These collections often need to be transformed, either to other collections or to some aggregate results. This is what we need collection processing methods for: to transform collections.

Collection processing is not a small deal. For years, it has been a primary selling point of Functional Programming^{[20](#)}. Even the name of the Lisp programming language^{[21](#)} stands for “list processing”. Likewise, Haskell is famous for its powerful collection processing methods. These amazing capabilities are also a selling point of Scala, where even `Option`, a type used for null safety, can be viewed as a collection of zero or one element to be processed as a part of a list comprehension structure. Scala has strongly influenced the Java community and promoted a functional style, especially for processing collections. This is one of the biggest reasons why so many previously Object-Oriented languages introduced support for Functional Programming features: they wanted to support functional-style collection processing. Nowadays, most modern languages support such processing. This includes Kotlin, which has a huge library of collection processing methods that help us make processing effective and efficient.

To see the power of collection processing methods in a practical case, consider a situation in which we need to fetch a list of news items but we need to show only those that are visible, have the correct order, and are mapped to the proper view elements. Without functional-style collection processing, this is how these transformations look like:

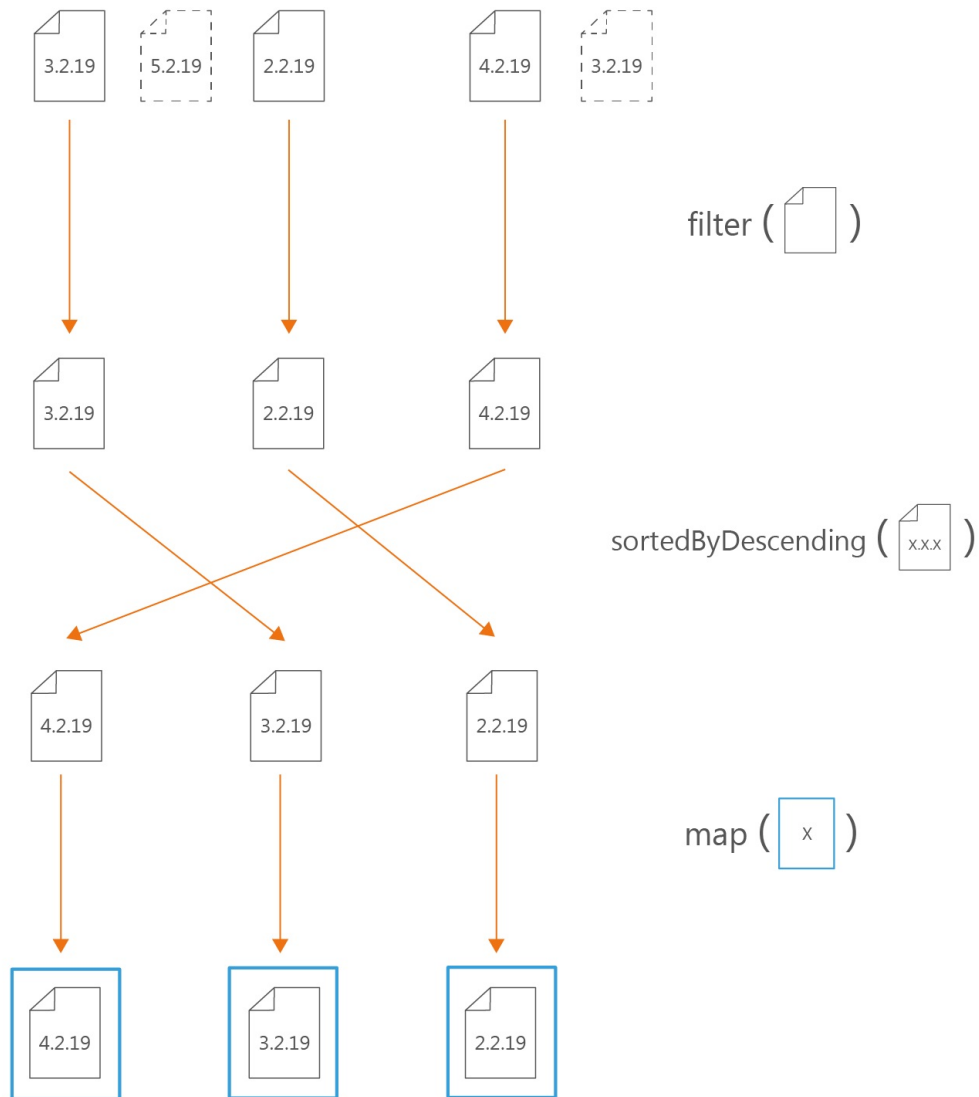
```
val visibleNews = mutableListOf<News>()
for (n in news) {
    if (n.visible) {
        visibleNews.add(n)
    }
}

Collections.sort(visibleNews) { n1, n2 ->
    n2.publishedAt - n1.publishedAt
}
val newItemAdapters = mutableListOf<NewsItemAdapter>()
for (n in visibleNews) {
    newItemAdapters.add(NewsItemAdapter(n))
}
```

With collection processing^{[22](#)}, this can be replaced with the following code:

```
val newItemAdapters = news
    .filter { it.visible }
    .sortedByDescending { it.publishedAt }
    .map(::NewsItemAdapter)
```

Such notation is not only shorter but also more readable. Every step performs a concrete transformation on the list of elements. Here is a visualization of the above process:



Being proficient in using functional-style collection processing is one of the hallmarks of a good Kotlin developer. It requires knowing useful methods and having experience in using them for a variety of problems. In this chapter, we

will learn about the methods I find most useful, and then we will look at how they can be used together to achieve powerful collection processing.

Most collection processing functions are very simple under the hood. For the simplest ones, I will show their simplified implementations before their explanations so that you can enjoy figuring out how these functions work before learning about them.

forEach and onEach

```
// `forEach` implementation from Kotlin stdlib
inline fun <T> Iterable<T>.forEach(action: (T) -> Unit) {
    for (element in this) action(element)
}

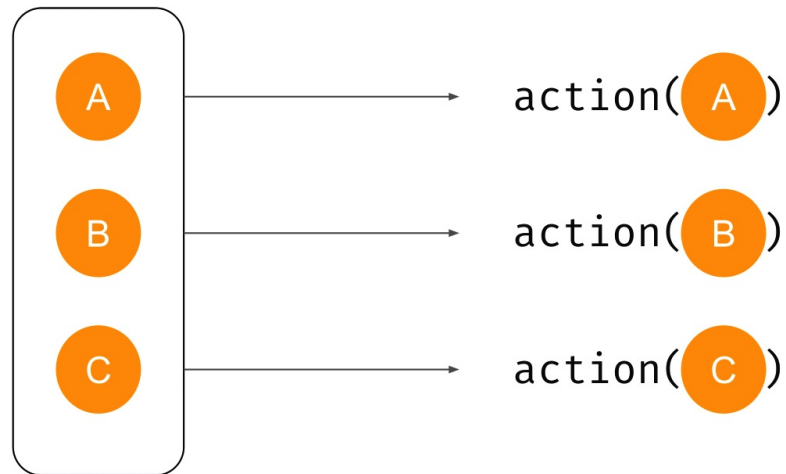
// simplified `onEach` implementation from Kotlin stdlib
inline fun <T, C : Iterable<T>> C.onEach(
    action: (T) -> Unit
): C {
    for (element in this) action(element)
    return this
}
```

The `forEach` function is an alternative to a simple for-loop - both invoke an operation on every element. Choosing between these two is often a matter of personal preference. The advantage of `forEach` is that it can be called conditionally with a safe-call (`?.`) and is better suited to multiline expressions. For-loop is generally considered more intuitive for less experienced developers.

```
// Without variable, this code would be hard to read
val messagesToSend = users.filter { it.isActive }
    .flatMap { it.remainingMessages }
    .filter { it.isToBeSent }
for (message in messagesToSend) {
    sendMessage(message)
}

// better
users.filter { it.isActive }
    .flatMap { it.remainingMessages }
    .filter { it.isToBeSent }
    .forEach { sendMessage(it) }
```

`.forEach(action)`

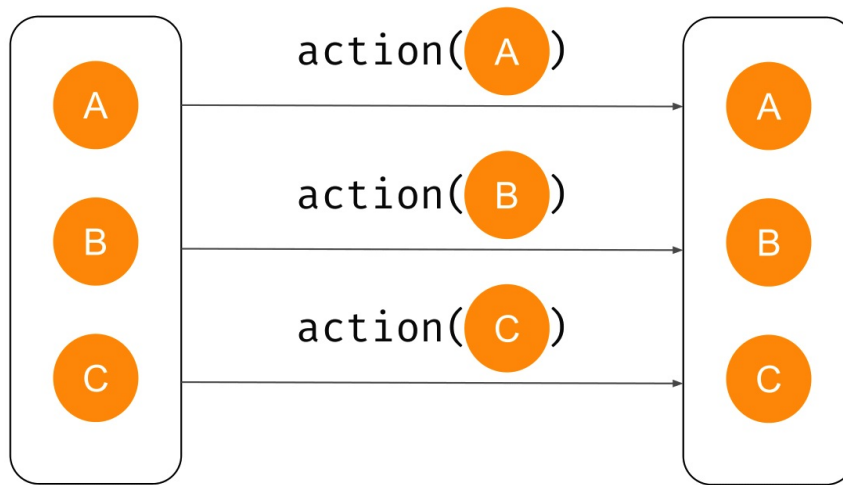


Methods like `filter` or `flatMap` will be covered later.

`forEach` returns `Unit`, so it is a terminal operation. This means no further steps are possible in the pipeline. However, in some situations, we need to invoke an operation on each element in the middle of collection processing. In such cases, we use `onEach`, which also invokes an operation on each element, but it returns the same collection it is invoked on.

```
users
  .filter { it.isActive }
  .onEach { log("Sending messages for user $it") }
  .flatMap { it.remainingMessages }
  .filter { it.isToBeSent }
  .forEach { sendMessage(it) }
```

`.onEach(action)`



filter

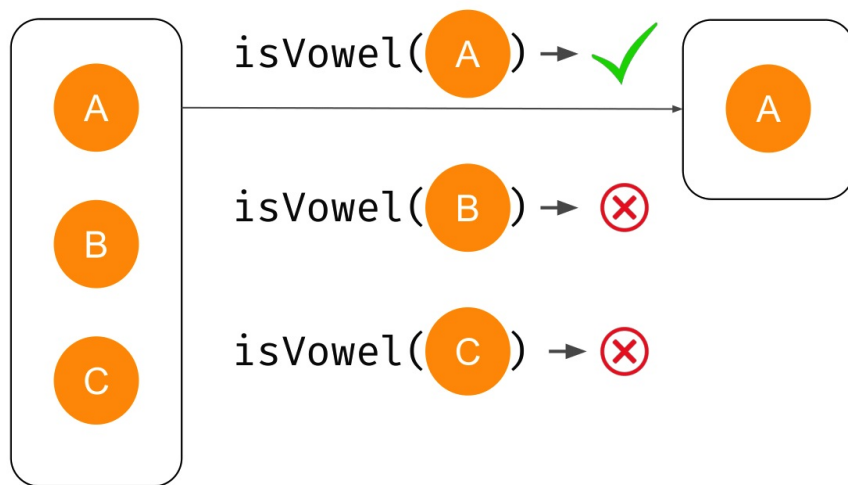
```
// simplified `filter` implementation from Kotlin stdlib
inline fun <T> Iterable<T>.filter(
    predicate: (T) -> Boolean
): List<T> {
    val destination = ArrayList<T>()
    for (element in this) {
        if (predicate(element)) {
            destination.add(element)
        }
    }
    return destination
}
```

Very often, we are interested in only certain elements in a collection. For instance, when we have a list of all users but are interested only in those that are active. Alternatively, we have a list of articles but we want to show only those that are public. In such cases, we use the `filter` method, which returns a collection of only the elements that satisfy its predicate.

```
val activeUsers = users
    .filter { it.isActive }

val publicArticles = articles
    .filter { it.visibility == PUBLIC }
```

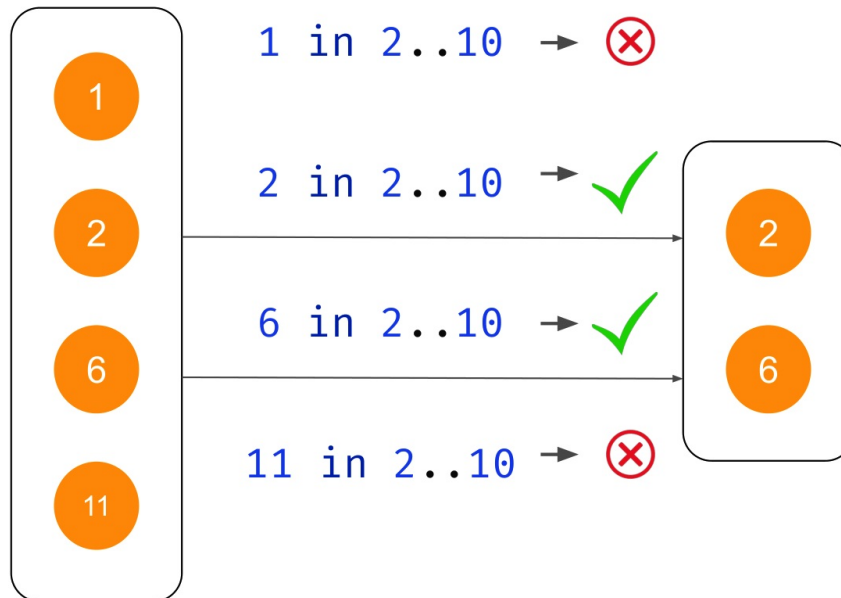
`.filter(isVowel)`



The `filter` method can limit the number of elements; therefore, the new collection might be smaller or even empty, but the elements in it are the same elements as in the original one.

```
fun main() {  
    val old = listOf(1, 2, 6, 11)  
    val new = old.filter { it in 2..10 }  
    println(new) // [2, 6]  
}
```

```
.filter { it in 2..10 }
```



The name “filter” is a bit tricky because in English, we often use it in the meaning “filter out” (like “sediment filter” or “UV filter”). When we use a `filter` in programming, we are interested not in what is filtered out but in what is retained. I understand the `filter` function as “filter to keep the elements that...”. For instance, in the above example, I would read “filter to keep the elements that are in the range from 2 to 10”. You can also think of filtering water - when you do that, you want to get clear water as a result.

There is also `filterNot`, which works similarly but keeps the elements that **do not** satisfy its predicate. So, `filterNot(op)` gives the same result as `filter { !op(it) }`.

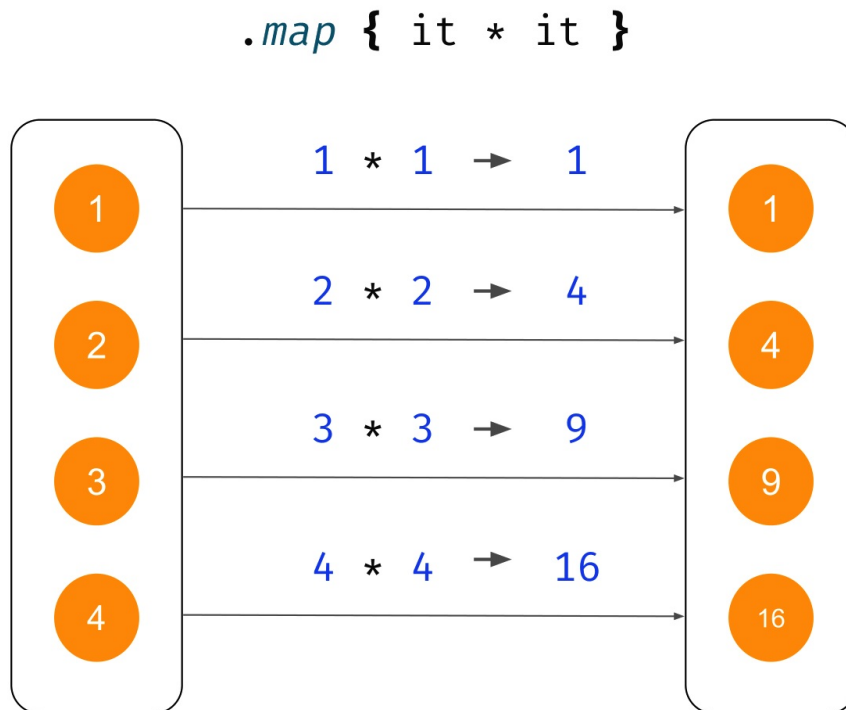
```
fun main() {  
    val old = listOf(1, 2, 6, 11)  
    val new = old.filterNot { it in 2..10 }  
    println(new) // [1, 11]  
}
```

map

```
// simplified `map` implementation from Kotlin stdlib
inline fun <T, R> Iterable<T>.map(
    transform: (T) -> R
): List<R> {
    val size = if (this is Collection<*>) this.size else 10
    val destination = ArrayList<R>(size)
    for (element in this) {
        destination.add(transform(element))
    }
    return destination
}
```

One of the most popular collection processing functions is `map`, which we use to transform all elements in a collection.

```
fun main() {
    val old = listOf(1, 2, 3, 4)
    val new = old.map { it * it }
    println(new) // [1, 4, 9, 16]
}
```



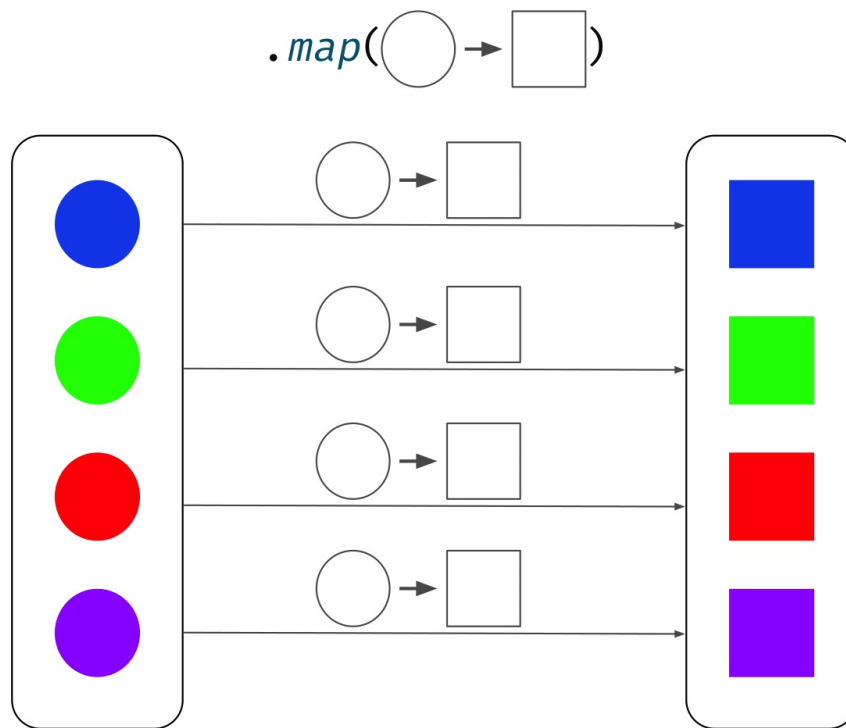
map produces a collection of the same size, but the elements might be transformed and their type might be different from the original collection.

```
fun main() {  
    val names: List<String> = listOf("Alex", "Bob", "Carol")  
    val nameSizes: List<Int> = names.map { it.length }  
    println(nameSizes) // [4, 3, 5]  
}
```

This transformation might be a simple modification, but often it is a transformation from one type to another. For instance, let's say that you are implementing an online shop: you have a list of offers to display, but you need to transform these simple data holders into some view elements that you can display.

```
// Make users that are 1 year older than before  
val olderUsers = users  
    .map { it.copy(age = it.age + 1) }  
  
// Transform offers into offer views
```

```
val offerViews = offers  
  .map { OfferView(it) }
```



flatMap

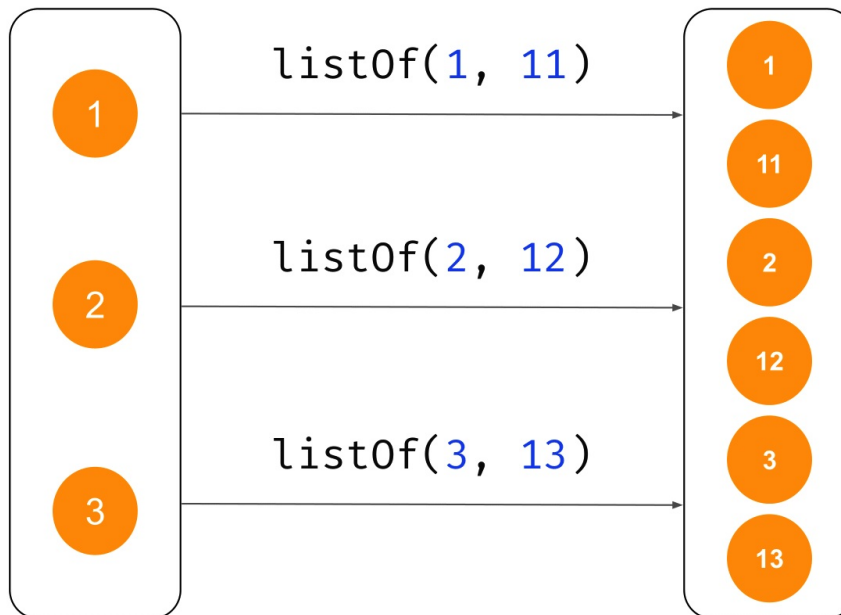
```
// simplified `flatMap` implementation from Kotlin stdlib
inline fun <T, R> Iterable<T>.flatMap(
    transform: (T) -> Iterable<R>
): List<R> {
    val size = if (this is Collection<*>) this.size else 10
    val destination = ArrayList<R>(size)
    for (element in this) {
        destination.addAll(transform(element))
    }
    return destination
}
```

Among collection processing functions, there is a famous quartet of functions every developer should know: `forEach`, `filter`, `map` and... `flatMap`. These are as idiomatic to functional collection processing as `for` and `while` loops are to imperative programming

`flatMap` first maps elements into another collection of elements, then it flattens them. To make it possible to flatten elements, `flatMap` requires its transformation to return something that is iterable, for instance a list or a set.

```
fun main() {
    val old = listOf(1, 2, 3)
    val new = old.flatMap { listOf(it, it + 10) }
    println(new) // [1, 11, 2, 12, 3, 13]
}
```

```
.flatMap { listOf(it, it + 10) }
```



In practice, the only difference between `flatMap` and `map` is this flattening. So, if `map` returns a collection of collections, `flatMap` returns a collection. This difference can be eliminated with the `flatten` method on `Iterable<Iterable<T>>` (so `flatMap(tr)` gives the same result as `map(tr).flatten()`).

```
fun main() {
    val names = listOf("Ann", "Bob", "Cale")
    val chars1: List<Char> = names.flatMap { it.toList() }
    println(chars1) // [A, n, n, B, o, b, C, a, l, e]
    val mapRes: List<List<Char>> = names.map { it.toList() }
    println(mapRes) // [[A, n, n], [B, o, b], [C, a, l, e]]
    val chars2 = mapRes.flatten()
    println(chars2) // [A, n, n, B, o, b, C, a, l, e]
    println(chars1 == chars2) // true
}
```

`String.toList()` transforms a string into a list of characters.

We typically use `flatMap` to extract elements from an object that holds a list of elements. For instance, we have a list of schools, each of which has a list of students, but we are interested in all the students. Another example might be if we have a list of departments, each of which has a list of employees, but we're interested in the employees.

```
val allStudents = schools
    .flatMap { it.students }

val allEmployees = department
    .flatMap { it.employees }
```

fold

```
// `fold` implementation from Kotlin stdlib
inline fun <T, R> Iterable<T>.fold(
    initial: R,
    operation: (acc: R, T) -> R
): R {
    var accumulator = initial
    for (element in this) {
        accumulator = operation(accumulator, element)
    }
    return accumulator
}
```

`fold` is the most universal method in our collection processing toolbox. We use it rarely because Kotlin standard library has already provided most important aggregate operations for us, but if we are missing a method for a specific task, `fold` is at our service.

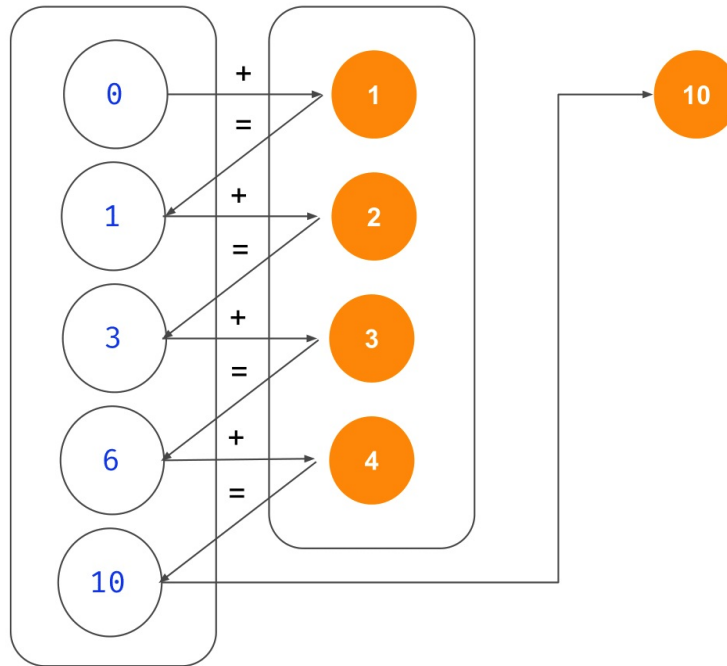
Let's see it practice. `fold` is a method that accumulates all elements into a single variable (called an "accumulator") using a defined operation. For instance, let's say that our collection contains the numbers from 1 to 4, our initial accumulator value is 0, and our operation is addition. So `fold` will:

- add the first value 1 to the initial accumulator value 0,
- then it will add the result 1 to the next value 2,
- then it will add the result 3 to the next value 3,
- then it will add the result 6 to the next value 4,
- and the result is 10.

As you can see, `fold(0) { acc, i -> acc + i }` calculates the sum of all the numbers.

```
list.fold(0) { acc, i -> acc + i }
```

Accumulator values



Since you can specify the initial value, you can decide the result type. If your initial value is an empty string and your operation is addition, then the result will be a “1234” string.

```
fun main() {  
    val numbers = listOf(1, 2, 3, 4)  
    val sum = numbers.fold(0) { acc, i -> acc + i }  
    println(sum) // 10  
    val joinedString = numbers.fold("") { acc, i -> acc + i }  
    println(joinedString) // 1234  
    val product = numbers.fold(1) { acc, i -> acc * i }  
    println(product) // 24  
}
```

fold is very universal. Nearly all collection processing methods can be implemented using it.

```
// simplified `filter` implemented with `fold`  
inline fun <T> Iterable<T>.filter(  
    predicate: (T) -> Boolean  
) : List<T> =  
    fold(emptyList()) { acc, e ->
```

```
        if (predicate(e)) acc + e else acc
    }

// simplified `map` implemented with `fold`
inline fun <T, R> Iterable<T>.map(
    transform: (T) -> R
): List<R> =
    fold(emptyList()) { acc, e -> acc + transform(e) }

// simplified `flatMap` implemented with `fold`
inline fun <T, R> Iterable<T>.flatMap(
    transform: (T) -> Iterable<R>
): List<R> =
    fold(emptyList()) { acc, e -> acc + transform(e) }
```

On the other hand, thanks to the fact that the Kotlin standard library has so many collection processing functions, we rarely need to use `fold`. Even the functions we presented before that calculate a sum and join elements into a string have dedicated methods.

```
fun main() {
    val numbers = listOf(1, 2, 3, 4, 5)
    println(numbers.sum()) // 15
    println(numbers.joinToString(separator = "")) // 12345
}
```

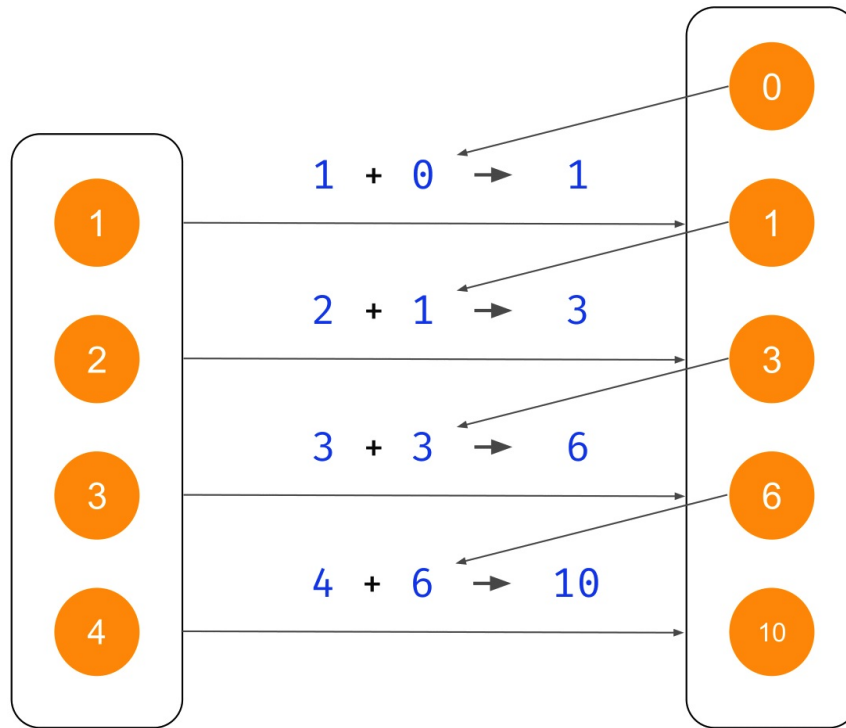
There is currently no standard library method to calculate the product of all the numbers in a collection, so this is where `fold` can be used. We might use it directly, or we might use it to implement the product method ourselves.

```
fun Iterable<Int>.product(): Int =
    fold(1) { acc, i -> acc * i }
```

If you want to reverse the order of accumulation (to start from the end of the collection), use `foldRight`.

In some situations, you might want to have not only the result of `fold` accumulations but also all the intermediate values. For that, you can use the `runningFold` method or its alias^{[23](#)} `scan`.

```
list.scan(0) { acc, i -> acc + i }
```



```
fun main() {  
    val numbers = listOf(1, 2, 3, 4)  
    println(numbers.fold(0) { acc, i -> acc + i }) // 10  
    println(numbers.scan(0) { acc, i -> acc + i })  
    // [0, 1, 3, 6, 10]  
    println(numbers.runningFold(0) { acc, i -> acc + i })  
    // [0, 1, 3, 6, 10]  
  
    println(numbers.fold("") { acc, i -> acc + i }) // 1234  
    println(numbers.scan("") { acc, i -> acc + i })  
    // [, 1, 12, 123, 1234]  
    println(numbers.runningFold("") { acc, i -> acc + i })  
    // [, 1, 12, 123, 1234]  
  
    println(numbers.fold(1) { acc, i -> acc * i }) // 24  
    println(numbers.scan(1) { acc, i -> acc * i })  
    // [1, 1, 2, 6, 24]  
    println(numbers.runningFold(1) { acc, i -> acc * i })  
    // [1, 1, 2, 6, 24]  
}
```

`runningFold(init, oper).last()` and `scan(init, oper).last()` always give the same result as `fold(init, oper)`.

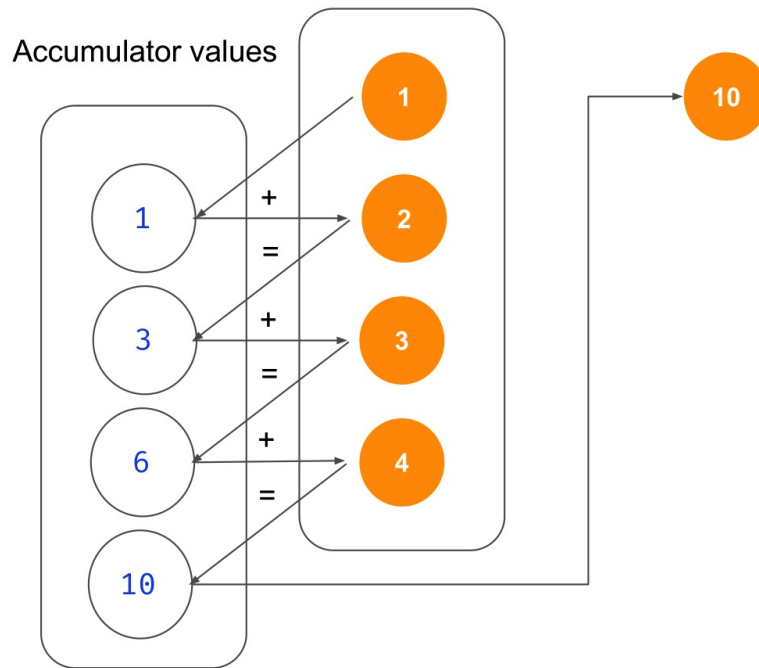
reduce

```
// simplified `reduce` implementation from Kotlin stdlib
public inline fun <S, T : S> Iterable<T>.reduce(
    operation: (acc: S, T) -> S
): S {
    val iterator = this.iterator()
    if (!iterator.hasNext())
        throw UnsupportedOperationException(
            "Empty collection can't be reduced."
        )
    var accumulator: S = iterator.next()
    while (iterator.hasNext()) {
        accumulator = operation(accumulator, iterator.next())
    }
    return accumulator
}
```

reduce is a very similar function to fold: it also accumulates all elements using a defined transformation. The difference is that in reduce we do not define the initial value, and so reduce uses the first element as the initial value. There are two consequences of this fact:

- If a collection is empty, reduce throws an exception. If we are not certain that a collection has elements, we should use `reduceOrNull`, which returns `null` for an empty collection.
- The result from reduce must be of the same type as its elements.
- reduce is slightly faster than fold because it has one operation less to do.

```
list.reduce { acc, i -> acc + i }
```



```
fun main() {
    val numbers = listOf(1, 2, 3, 4, 5)
    println(numbers.fold(0) { acc, i -> acc + i }) // 15
    println(numbers.reduce { acc, i -> acc + i }) // 15

    println(numbers.fold("") { acc, i -> acc + i }) // 12345
    // Here `reduce` cannot be used instead of `fold`

    println(numbers.fold(1) { acc, i -> acc * i }) // 120
    println(numbers.reduce { acc, i -> acc * i }) // 120
}
```

`list.reduce(oper)` is a lot like `list.drop(1).fold(list[0], oper)`.

In general, I prefer using `fold` whenever there is a “zero” value because `fold` does not face the risk of an empty collection and it is able to control the result type.

Just like for `fold`, there is `runningReduce` and `reduceRight`.

sum

```
// simplified sample `sum` implementation from Kotlin stdlib
fun Iterable<Int>.sum(): Int {
    var sum: Int = 0
    for (element in this) {
        sum += element
    }
    return sum
}

// simplified sample `sumOf` implementation from Kotlin stdlib
inline fun <T> Iterable<T>.sumOf(
    selector: (T) -> Int
): Int {
    var sum: Int = 0.toInt()
    for (element in this) {
        sum += selector(element)
    }
    return sum
}
```

I mentioned that there is already a function to calculate the sum of all the numbers in a collection, and its name is `sum`. It is implemented for all the basic ways of representing numbers, like `Int`, `Long`, `Double`, etc.

```
fun main() {
    val numbers = listOf(1, 6, 2, 4, 7, 1)
    println(numbers.sum()) // 21

    val doubles = listOf(0.1, 0.6, 0.2, 0.4, 0.7)
    println(doubles.sum()) // 1.9999999999999998
    // It is not 2, due to limited JVM double representation

    val bytes = listOf<Byte>(1, 4, 2, 4, 5)
    println(bytes.sum()) // 16
}
```

When you have a list of elements and you want to calculate the sum of one of their properties, you could first map the elements onto the values of these properties, but it is more efficient to use `sumOf`, which extracts a countable value for each element and then sums these values.

```
import java.math.BigDecimal

data class Player(
    val name: String,
    val points: Int,
    val money: BigDecimal,
)

fun main() {
    val players = listOf(
```

```
        Player("Jake", 234, BigDecimal("2.30")),
        Player("Megan", 567, BigDecimal("1.50")),
        Player("Beth", 123, BigDecimal("0.00")),
    )

    println(players.map { it.points }.sum()) // 924
    println(players.sumOf { it.points }) // 924

    // Works for `BigDecimal` as well
    println(players.sumOf { it.money }) // 3.80
}
```

withIndex and indexed variants

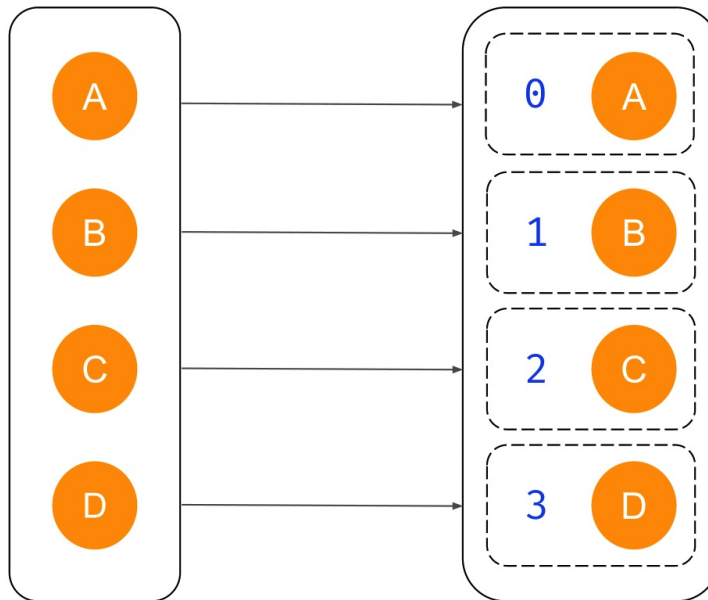
```
// `withIndex` implementation from Kotlin stdlib
fun <T> Iterable<T>.withIndex(): Iterable<IndexedValue<T>> =
    IndexingIterable { iterator() }

data class IndexedValue<out T> (
    val index: Int,
    val value: T
)
```

Sometimes we are not only interested in elements but also in their positions in a collection. Let's say that in one of your collection processing functions you need to depend not only on an element's value but also on its index in the collection. The generic way is to use the `withIndex` function, which lazily transforms a list of elements into a list of indexed elements. These elements can be then destructured^{[24](#)} into an index and a value.

```
fun main() {
    listOf("A", "B", "C", "D") // List<String>
        .withIndex() // List<IndexedValue<String>>
        .filter { (index, value) -> index % 2 == 0 }
        .map { (index, value) -> "[$index] $value" }
        .forEach { println(it) }
}
// [0] A
// [2] C
```

`.withIndex()`



This is a universal iterator function, but many collection processing functions do not need it because they have “indexed” variants. For instance, there are the `filterIndexed`, `mapIndexed`, `flatMapIndexed`, `foldIndexed`, and `scanIndexed` functions, which work the same as `filter`, `map`, `flatMap`, `fold`, and `scan`, but they also have an index in the first position of their operation.

```
fun main() {
    val chars = listOf("A", "B", "C", "D")

    val filtered = chars
        .filterIndexed { index, value -> index % 2 == 0 }
    println(filtered) // [A, C]

    val mapped = chars
        .mapIndexed { index, value -> "$index $value" }
    println(mapped) // [[0] A, [1] B, [2] C, [3] D]
}
```

Notice that using `withIndex` adds the current index to each element, and this index stays the same for all steps, while the indexed function operates on the

current index for each step.

```
fun main() {
    val chars = listOf("A", "B", "C", "D")

    val r1 = chars.withIndex()
        .filter { (index, value) -> index % 2 == 0 }
        .map { (index, value) -> "[$index] $value" }
    println(r1) // [[0] A, [2] C]

    val r2 = chars
        .filterIndexed { index, value -> index % 2 == 0 }
        .mapIndexed() { index, value -> "[$index] $value" }
    println(r2) // [[0] A, [1] C]
}
```

take, takeLast, drop, dropLast and subList

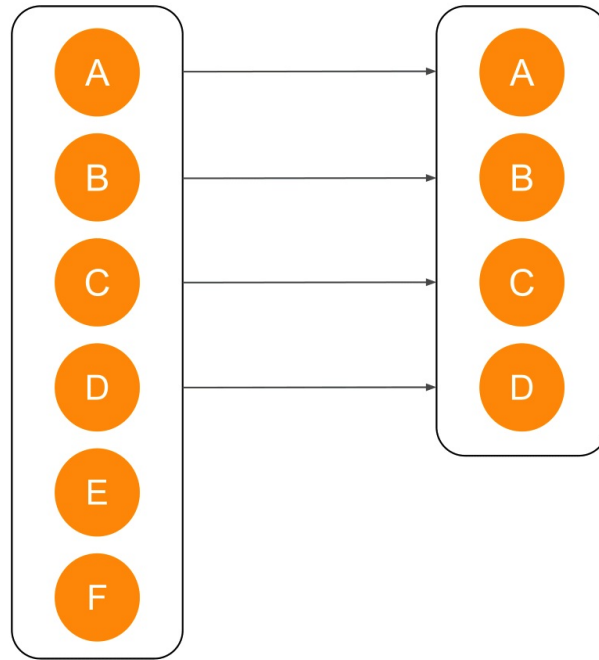
When you need to take or get rid of a certain number of elements, the take, takeLast, drop and dropLast functions are at your service:

- take(n) - returns a collection with only the first n elements (or returns the unchanged collection if it has less than n elements).
- takeLast(n) - returns a collection with only the last n elements (or returns the unchanged collection if it has less than n elements).
- drop(n) - returns a collection without the first n elements.
- dropLast(n) - returns a collection without the last n elements.

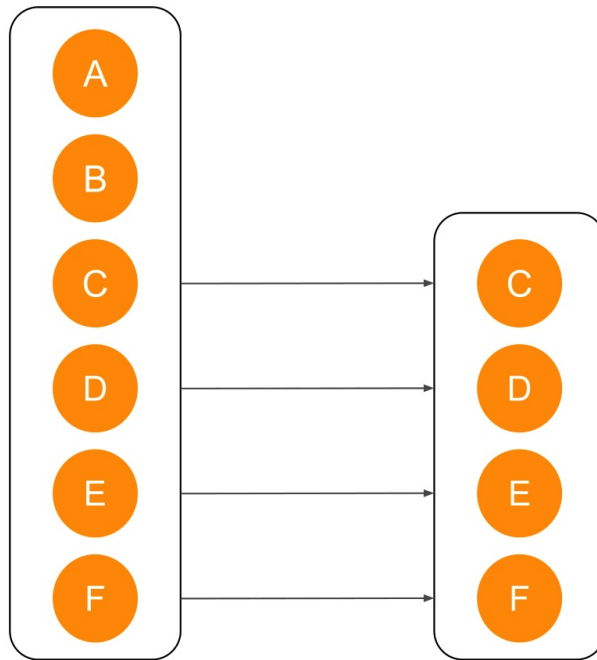
```
fun main() {
    val chars = ('a'..'z').toList()

    println(chars.take(10))
    // [a, b, c, d, e, f, g, h, i, j]
    println(chars.takeLast(10))
    // [q, r, s, t, u, v, w, x, y, z]
    println(chars.drop(10))
    // [k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z]
    println(chars.dropLast(10))
    // [a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p]
}
```

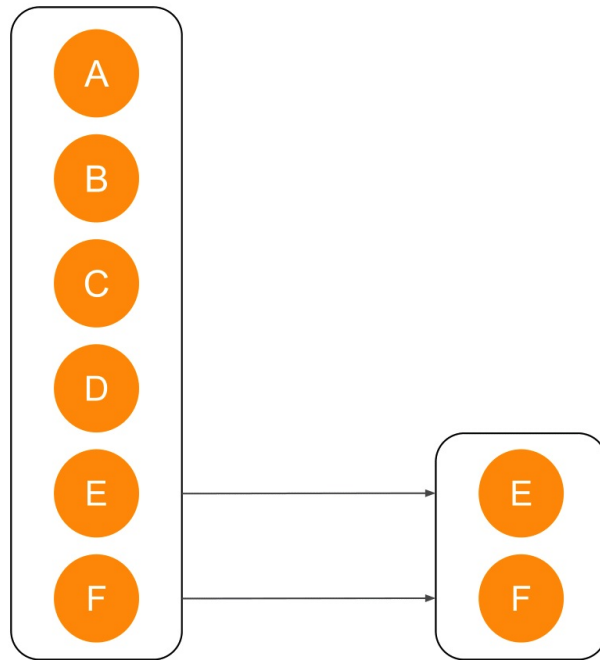
`.take(4)`



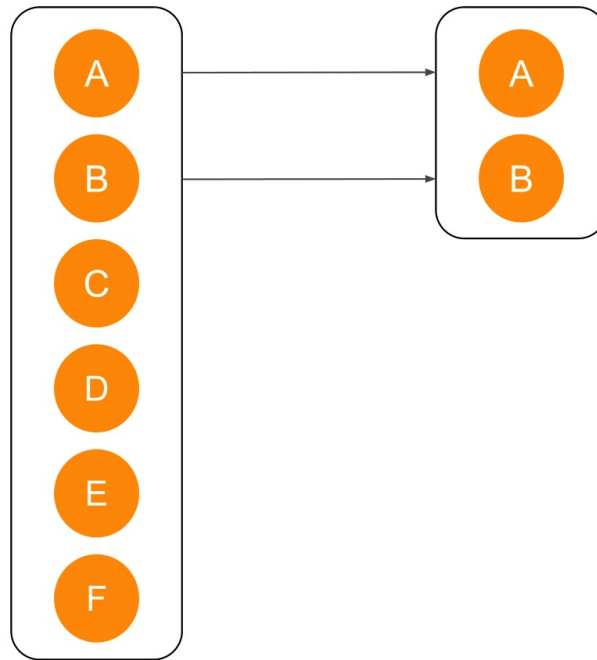
`.takeLast(4)`



`.drop(4)`



`.dropLast(4)`



Kotlin by design doesn't have aliases for `head` (to take the first element) or `tail` (to drop the first element) methods, that are well known in other functional languages. Instead, we use `first()` and `drop(1)`.

Most collection processing functions, including `take` and `drop`, are extension functions on the `Iterable` interface, but `takeLast` and `dropLast` are extension functions on `List`. Such design is needed for efficiency.

If we know the size of our collection, these methods can be used interchangeably:

- `l.take(n)` gives the same result as `l.dropLast(l.size - n)`,
- `l.takeLast(n)` gives the same result as `l.drop(l.size - n)`,
- `l.drop(n)` gives the same result as `l.takeLast(l.size - n)`,
- `l.dropLast(n)` gives the same result as `l.take(l.size - n)`,

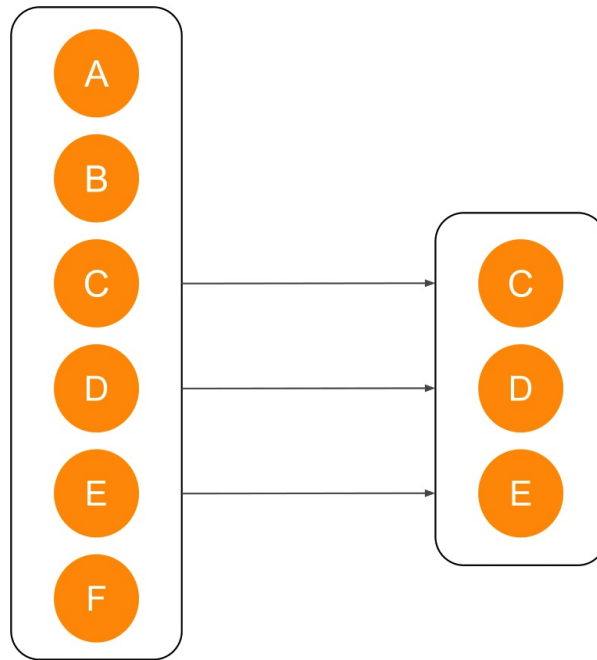
```
fun main() {  
    val c = ('a'..'z').toList()  
  
    println(c.take(10) == c.dropLast(c.size - 10)) // true  
    println(c.takeLast(10) == c.drop(c.size - 10)) // true  
    println(c.drop(10) == c.takeLast(c.size - 10)) // true  
    println(c.dropLast(10) == c.take(c.size - 10)) // true  
}
```

If we are operating on a `List`, all these methods can be replaced with the more universal `subList`, which expects as arguments the start index (inclusive) and the end index (exclusive), so:

- `l.take(n)` gives the same result as `l.subList(0, n)`,
- `l.takeLast(n)` gives the same result as `l.subList(l.size - n, l.size)`,
- `l.drop(n)` gives the same result as `l.subList(n, l.size)`,
- `l.dropLast(n)` gives the same result as `l.subList(0, l.size - n)`,

```
fun main() {  
    val c = ('a'..'z').toList()  
  
    val n = 10  
    val s = c.size  
    println(c.take(n) == c.subList(0, n)) // true  
    println(c.takeLast(n) == c.subList(s - n, s)) // true  
    println(c.drop(n) == c.subList(n, s)) // true  
    println(c.dropLast(n) == c.subList(0, s - n)) // true  
}
```

`.subList(2, 5)`



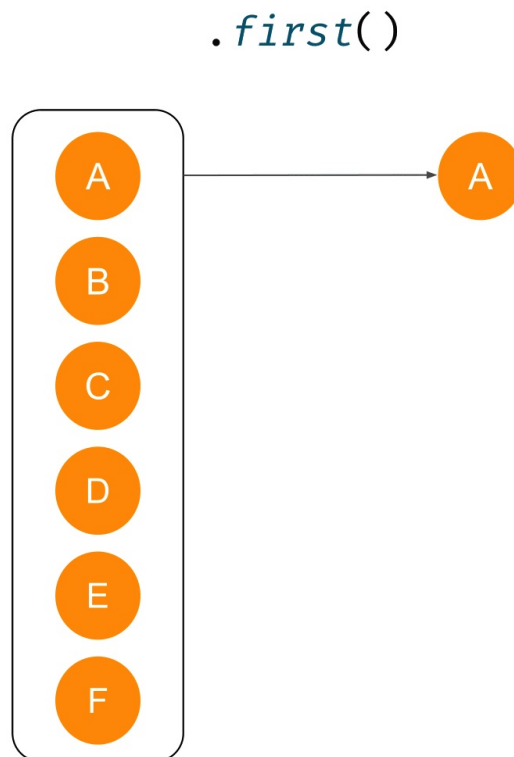
I find `take`, `takeLast`, `drop` and `dropLast` much more readable than `subList`, which requires unintuitive operations on indexes. They are also safer - when we ask to drop more elements than there are in the collection, the result is an empty collection, when we try to take more than there is the result is the collection with as many elements as possible, when we call `subList` with an incorrect value, it throws an exception.

```
fun main() {  
    val letters = listOf("a", "b", "c")  
    println(letters.take(100)) // [a, b, c]  
    println(letters.takeLast(100)) // [a, b, c]  
    println(letters.drop(100)) // []  
    println(letters.dropLast(100)) // []  
    letters.subList(0, 4) // throws IndexOutOfBoundsException  
}
```

Getting elements at certain positions

If you need to get the first element of a collection, use the `first` method. To get the last one, use the `last` method. To find an element at a concrete index, use the `get` function, which is also an operator and can be replaced with box brackets. You can also destructure a list into elements, starting at the first position.

```
fun main() {  
    val c = ('a'..'z').toList()  
  
    println(c.first()) // a  
    println(c.last()) // z  
    println(c.get(3)) // d  
    println(c[3]) // d  
    val (c1, c2, c3) = c  
    println(c1) // a  
    println(c2) // b  
    println(c3) // c  
}
```



A problem arises when a collection is empty. In such a case, all the functions above throw an exception (`NoSuchElementException` or

IndexOutOfBoundsException). To prevent this, use the variants of these functions with the “OrNull” suffix.

```
fun main() {  
    val c = listOf<Char>()  
  
    println(c.firstOrNull()) // null  
    println(c.lastOrNull()) // null  
    println(c.getOrNull(3)) // null  
}
```

Finding an element

Out of a whole collection of elements, we often want to find a single one that fulfills a predicate. It might be a user with a certain id, or a configuration with a concrete name. The most basic method of finding an element in a collection is `find`.

```
fun getUser(id: String): User? =
    users.find { it.id == id }

fun findConfiguration(name: String): Configuration? =
    configurations.find { it.name == name }
```

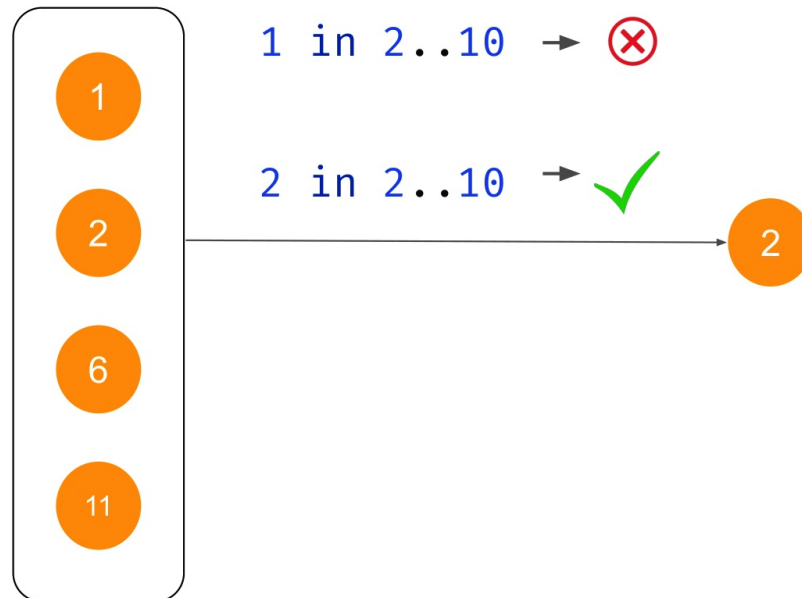
`find` is just an alias for `firstOrNull`. They both return the first element that fulfills the predicate, or `null` if no such element is found.

```
fun main() {
    val names = listOf("Cookie", "Figa")

    println(names.find { it.first() == 'A' }) // null
    println(names.firstOrNull { it.first() == 'A' }) // null
    println(names.find { it.first() == 'C' }) // Cookie
    println(names.firstOrNull { it.first() == 'C' }) // Cookie

    println(listOf(1, 2, 6, 11).find { it in 2..10 }) // 2
}
```

```
.find { it in 2..10 }
```



If you prefer to start searching from the end, you can use `findLast` or `lastOrNull`.

```
fun main() {  
    val names = listOf("C1", "C2")  
  
    println(names.find { it.first() == 'C' }) // C1  
    println(names.firstOrNull { it.first() == 'C' }) // C1  
    println(names.findLast { it.first() == 'C' }) // C2  
    println(names.lastOrNull { it.first() == 'C' }) // C2  
}
```

Counting: count

Counting the number of elements in a list is easy as we can always use the `size` property. However, some collections that implement the `Iterable` interface might require iterating over elements to count how many elements they have. The universal method of counting the number of elements in a collection is `count`.

```
fun main() {  
    val range = (1..100 step 3)  
    println(range.count()) // 34  
}
```

We can also add a predicate to `count` in order to count the number of elements that satisfy this predicate. For instance, we could count the number of users with a premium account, or the number of students that qualify for an internship.

```
val premiumUsersCount = users  
    .count { it.hasPremium }  
  
val qualifiedNum = students  
    .count { qualifiesForInternship(it) }
```

The `count` method returns the number of elements for which the predicate returned true.

```
fun main() {  
    val range = (1..100 step 3)  
    println(range.count { it % 5 == 0 }) // 7  
}
```

any, all and none

To check if a condition is true for all, any, or none of the elements in a collection, we use respectively `all`, `any` and `none`. They all return a `Boolean`. Let's see some examples.

```
data class Person(  
    val name: String,  
    val age: Int,  
    val male: Boolean  
)  
  
fun main() {  
    val people = listOf(  

```



```

    Person("Alice", 31, false),
    Person("Bob", 29, true),
    Person("Carol", 31, true)
)

fun isAdult(p: Person) = p.age > 18
fun isChild(p: Person) = p.age < 18
fun isMale(p: Person) = p.male
fun isFemale(p: Person) = !p.male

// Is there an adult?
println(people.any(::isAdult)) // true
// Are they all adults?
println(people.all(::isAdult)) // true
// Is none of them an adult?
println(people.none(::isAdult)) // false

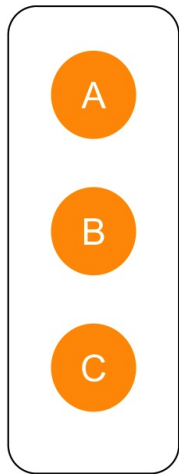
// Is there any child?
println(people.any(::isChild)) // false
// Are they all children?
println(people.all(::isChild)) // false
// Are none of them children?
println(people.none(::isChild)) // true

// Are there any males?
println(people.any { isMale(it) }) // true
// Are they all males?
println(people.all { isMale(it) }) // false
// Is none of them a male?
println(people.none { isMale(it) }) // false

// Are there any females?
println(people.any { isFemale(it) }) // true
// Are they all females?
println(people.all { isFemale(it) }) // false
// Is none of them a female?
println(people.none { isFemale(it) }) // false
}

```

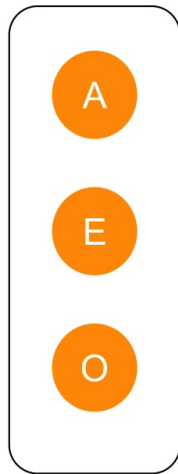
`.all(isVowel)`



`isVowel(A)` → ✓

`isVowel(B)` → ⊗ → ⊗

`.all(isVowel)`



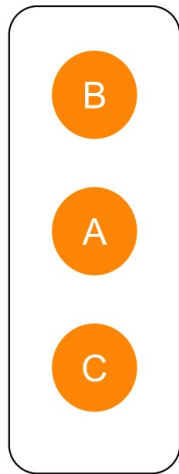
`isVowel(A)` → ✓

`isVowel(E)` → ✓

`isVowel(O)` → ✓



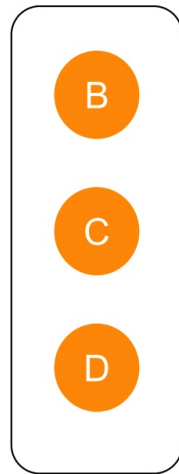
`.any(isVowel)`



`isVowel(B)` → ❌

`isVowel(A)` → ✔️ → ✔️

`.any(isVowel)`



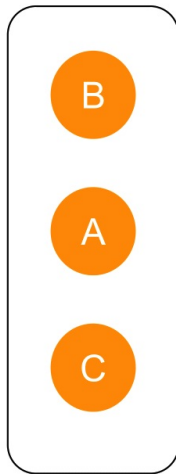
`isVowel(B) →`

`isVowel(C) →`

`isVowel(D) →`

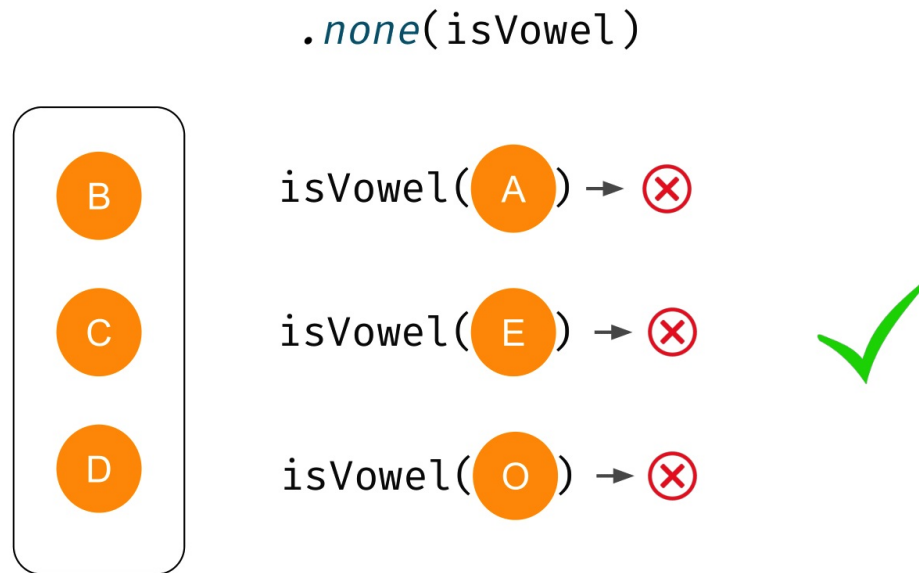


`.none(isVowel)`



`isVowel(B)` → ❌

`isVowel(A)` → ✓ → ❌



Beware: Developers often confuse the methods for finding elements, like `find` or `last`, with methods for checking a condition on elements, like `any`.

For empty collections, the predicate is never called. `any` returns `false`, while `all` and `none` return `true`. These values come from the mathematical definitions of these functions²⁵.

```
fun main() {  
    val emptyList = emptyList<String>()  
    println(emptyList.any { error("Ignored") }) // false  
    println(emptyList.all { error("Ignored") }) // true  
    println(emptyList.none { error("Ignored") }) // true  
}
```

partition

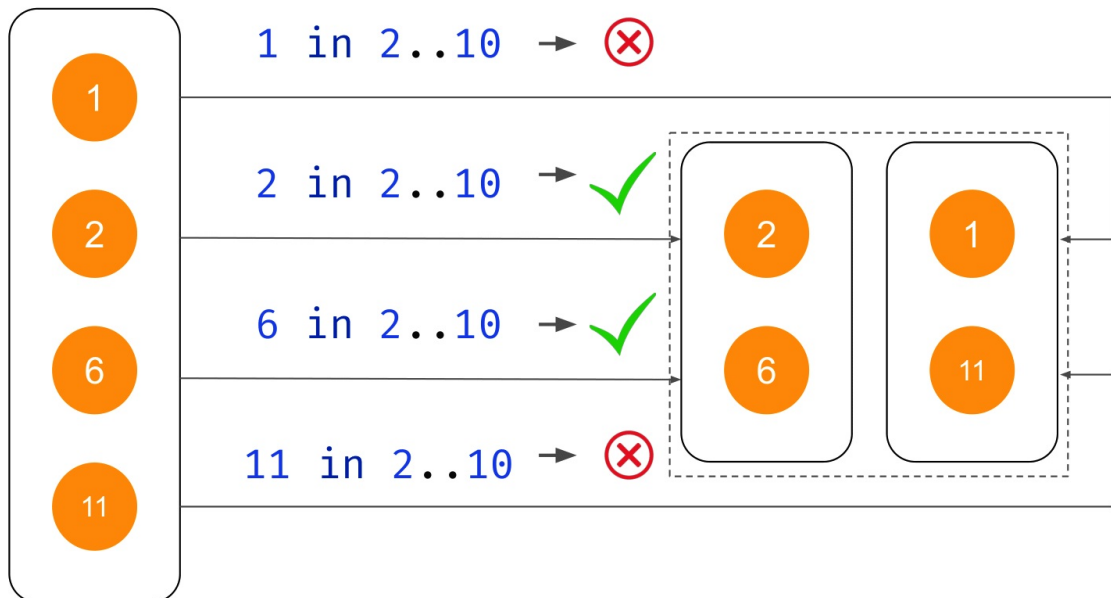
```
// `partition` implementation from Kotlin stdlib
inline fun <T> Iterable<T>.partition(
    predicate: (T) -> Boolean
): Pair<List<T>, List<T>> {
    val first = ArrayList<T>()
    val second = ArrayList<T>()
    for (element in this) {
        if (predicate(element)) {
            first.add(element)
        } else {
            second.add(element)
        }
    }
    return Pair(first, second)
}
```

We have learned about the `filter` function, which returns a list of elements that satisfy a predicate, but what if we are interested in the elements that satisfy it as well as those that do not? In such a case, we use the `partition` method, which returns a pair of lists. The first list contains all elements that satisfy its predicate, and the second list contains those that do not. This pair can be then destructured into separate collections.

```
fun main() {
    val nums = listOf(1, 2, 6, 11)
    val partitioned: Pair<List<Int>, List<Int>> =
        nums.partition { it in 2..10 }
    println(partitioned) // ([2, 6], [1, 11])

    val (inRange, notInRange) = partitioned
    println(inRange) // [2, 6]
    println(notInRange) // [1, 11]
}
```

`.partition { it in 2..10 }`



```
fun main() {  
    val nums = (1..10).toList()  
  
    val (smaller, bigger) = nums.partition { it <= 5 }  
    println(smaller) // [1, 2, 3, 4, 5]  
    println(bigger)  // [6, 7, 8, 9, 10]  
  
    val (even, odd) = nums.partition { it % 2 == 0 }  
    println(even)   // [2, 4, 6, 8, 10]  
    println(odd)    // [1, 3, 5, 7, 9]  
  
    data class Student(val name: String, val passing: Boolean)  
  
    val students = listOf(  
        Student("Alex", true),  
        Student("Ben", false),  
    )  
    val (passing, failed) = students.partition { it.passing }  
    println(passing) // [Student(name=Alex, passing=true)]  
    println(failed)  // [Student(name=Ben, passing=false)]  
}
```

groupBy

```
// `groupBy` implementation from Kotlin stdlib
inline fun <T, K> Iterable<T>.groupBy(
    keySelector: (T) -> K
): Map<K, List<T>> {
    val destination = LinkedHashMap<K, MutableList<T>>()
    for (element in this) {
        val key = keySelector(element)
        val list = destination.getOrPut(key) {
            ArrayList<T>()
        }
        list.add(element)
    }
    return destination
}
```

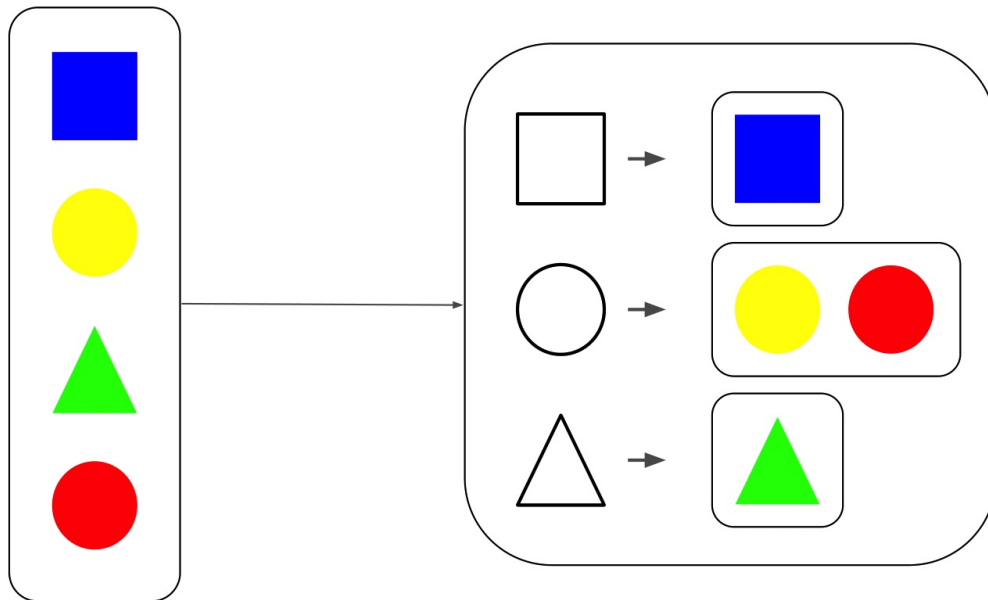
After presenting partition, I am often asked what we can do if we want to divide our collection into more than two groups. In such situations, we use `groupBy`, which groups elements by keys and returns a map from each key into a list of elements with this key (`Map<K, List<E>>`).

```
fun main() {
    val names = listOf("Marcin", "Maja", "Cookie")

    val byCapital = names.groupBy { it.first() }
    println(byCapital)
    // {M=[Marcin, Maja], C=[Cookie]}

    val byLength = names.groupBy { it.length }
    println(byLength)
    // {6=[Marcin, Cookie], 4=[Maja]}
}
```

`.groupBy(shape)`



From my experience, when my colleagues ask me for help with more complex collection processing, pretty often what they are missing is `groupBy`. Here are a few tasks that require this operation²⁶:

- Count the number of users in each city, based on a list of users.
- Find the number of points received by each team, based on a list of players.
- Find the best option in each category, based on a list of options.

```
// Count the number of users in each city
val usersCount: Map<City, Int> = users
    .groupBy { it.city }
    .mapValues { (_, users) -> users.size }

// Find the number of points received by each team
val pointsPerTeam: Map<Team, Int> = players
    .groupBy { it.team }
    .mapValues { (_, players) ->
        players.sumOf { it.points }
    }

// Find the best resolution in each category
val bestResolutionPerQuality: Map<Quality, Resolution> =
```

```
formats.groupBy { it.quality }  
    .mapValues { (_, formats) ->  
        formats.maxOf { it.resolution }  
    }
```

There is also the `groupingBy` method, which can be used as an alternative to `groupBy`. `groupingBy` is more efficient but also harder to use^{[27](#)}.

You can reverse the `groupBy` method using `flatMap`. If you first use `groupBy` and then `flatMap` the values, you will have the same elements you started with (but possibly in a different order).

```
data class Player(val name: String, val team: String)  
  
fun main() {  
    val players = listOf(  
        Player("Alex", "A"),  
        Player("Ben", "B"),  
        Player("Cal", "A"),  
    )  
    val grouped = players.groupBy { it.team }  
    println(grouped)  
    // {A=[Player(name=Alex, team=A),  
    //     Player(name=Cal, team=A)],  
    // B=[Player(name=Ben, team=B)]}  
    println(grouped.flatMap { it.value })  
    // [Player(name=Alex, team=A), Player(name=Cal, team=A),  
    // Player(name=Ben, team=B)]  
}
```

Associating: `associate`, `associateBy` and `associateWith`

```
// `associate` implementation from Kotlin stdlib  
inline fun <T, K, V> Iterable<T>.associate(  
    transform: (T) -> Pair<K, V>  
) : Map<K, V> {  
    val capacity = mapCapacity(collectionSizeOrDefault(10))  
        .coerceAtLeast(16)  
    val destination = LinkedHashMap<K, V>(capacity)  
    for (element in this) {  
        destination += transform(element)  
    }  
    return destination  
}
```

```
// `associateBy` implementation from Kotlin stdlib  
inline fun <T, K> Iterable<T>.associateBy(  
    keySelector: (T) -> K  
) : Map<K, T> {  
    val capacity = mapCapacity(collectionSizeOrDefault(10))  
        .coerceAtLeast(16)  
    val destination = LinkedHashMap<K, V>(capacity)
```

```

        for (element in this) {
            destination.put(keySelector(element), element)
        }
        return destination
    }
}

// `associateWith` implementation from Kotlin stdlib
public inline fun <K, V> Iterable<K>.associateWith(
    valueSelector: (K) -> V
): Map<K, V> {
    val capacity = mapCapacity(collectionSizeOrDefault(10))
        .coerceAtLeast(16)
    val destination = LinkedHashMap<K, V>(capacity)
    for (element in this) {
        destination.put(element, valueSelector(element))
    }
    return destination
}

```

To transform an iterable^{[28](#)} into a map, we use the `associate` method. In maps, elements are represented by both a key and a value, therefore the `associate` method needs to return a pair. If you want to use the elements of your list as the **keys** of your new map, a better alternative to `associate` is `associateWith`. On its lambda expression, you should specify what the **value** should be for each key. If you want to use elements of your list as **values** of your new map, a better alternative to `associate` is `associateBy`. On its lambda expression, specify what the **key** should be for each value.

```

fun main() {
    val names = listOf("Alex", "Ben", "Cal")
    println(names.associate { it.first() to it.drop(1) })
    // {A=lex, B=en, C=al}
    println(names.associateWith { it.length })
    // {Alex=4, Ben=3, Cal=3}
    println(names.associateBy { it.first() })
    // {A=Alex, B=Ben, C=Cal}
}

```

`associateWith(op)` works the same as `associate { it to op(it) }`.
`associateBy(op)` works the same as `associate { op(it) to it }`.

Be careful because keys on maps need to be unique, and a new value with the same key replaces the previous one. If you want to keep instead of replace previous values, use the `groupBy` or `groupingBy` method instead of the `associateBy` method.

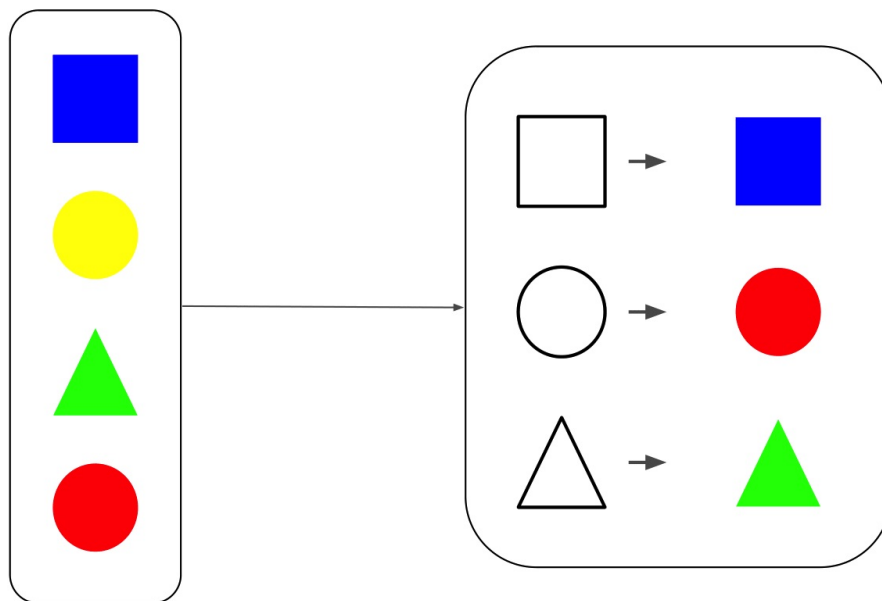
```

fun main() {
    val names = listOf("Alex", "Aaron", "Ada")
}

```

```
println(names.associateBy { it.first() })  
// {A=Ada}  
println(names.groupBy { it.first() })  
// {A=[Alex, Aaron, Ada]}  
}
```

`.associate(shape)`



When keys are unique, `associateWith` can be reversed using the `keys` property, and `associateBy` can be reversed using the `values` property.

```
fun main() {  
    val names = listOf("Alex", "Ben", "Cal")  
    val aW = names.associateWith { it.length }  
    println(aW.keys.toList() == names) // true  
    val aB = names.associateBy { it.first() }  
    println(aB.values.toList() == names) // true  
}
```

`toList` is required before comparison because `keys` returns a set, and `values` returns a dedicated collection.

Finding an element in a list requires iterating over the elements one by one. Finding a value by a key is much more efficient thanks to the hash table that is used under the hood. That is why `associateBy` is used to optimize searching for elements^{[29](#)}.

```
fun produceUserOffers(
    offers: List<Offer>,
    users: List<User>
): List<UserOffer> {
    //
    val usersById = users.associateBy { it.id }
    return offers
        .map { createUserOffer(it, usersById[it.buyerId]) }
}
```

distinct and distinctBy

```
// `distinct` implementation from Kotlin stdlib
fun <T> Iterable<T>.distinct(): List<T> {
    return this.toMutableSet().toList()
}

inline fun <T, K> Iterable<T>.distinctBy(
    selector: (T) -> K
): List<T> {
    val set = HashSet<K>()
    val list = ArrayList<T>()
    for (e in this) {
        val key = selector(e)
        if (set.add(key))
            list.add(e)
    }
    return list
}
```

So, we now know that we can use `associate` to transform a list to a map. Transforming it to a set is much easier: we can just use the `toSet` function. A set is much more similar to a list than a map, and the key difference is that sets do not allow duplicates^{[30](#)}.

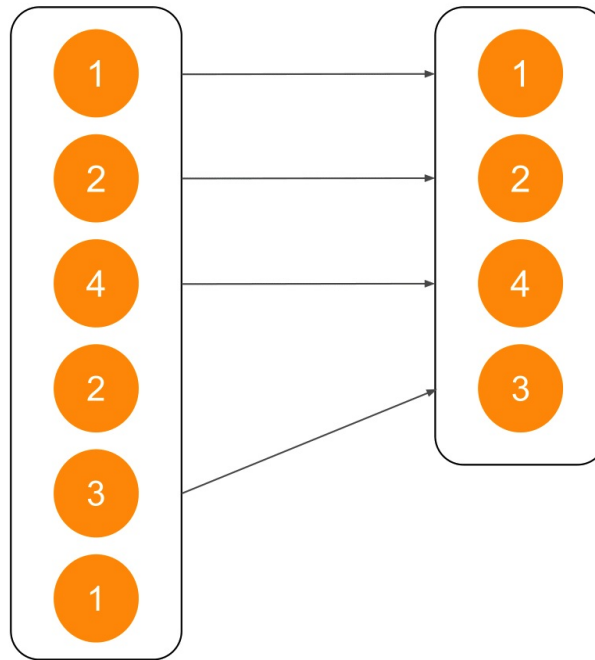
```
fun main() {
    val list: List<Int> = listOf(1, 2, 4, 2, 3, 1)
    val set: Set<Int> = list.toSet()
    println(set) // [1, 2, 4, 3]
}
```

If you want to keep operating on a list but at the same time eliminate duplicates, use the `distinct` method. Under the hood, it transforms a list into a set and then back to a list. So, it eliminates elements that are equal to each other.

```
fun main() {
    val numbers = listOf(1, 2, 4, 2, 3, 1)
    println(numbers) // [1, 2, 4, 2, 3, 1]
    println(numbers.distinct()) // [1, 2, 4, 3]

    val names = listOf("Marta", "Maciek", "Marta", "Daniel")
    println(names) // [Marta, Maciek, Marta, Daniel]
    println(names.distinct()) // [Marta, Maciek, Daniel]
}
```

`.distinct()`



We can also use `distinctBy`, which uses a selector and keeps only the elements with the distinct values returned by this selector. This way, it gives us full control over the criteria used to decide if two values are distinct.

```
fun main() {  
    val names = listOf("Marta", "Maciek", "Marta", "Daniel")  
    println(names) // [Marta, Maciek, Marta, Daniel]  
    println(names.distinctBy { it[0] }) // [Marta, Daniel]  
    println(names.distinctBy { it.length }) // [Marta, Maciek]  
}
```

Be aware that `distinct` keeps the first element of the list, while `associateBy` keeps the last element.

```
fun main() {  
    val names = listOf("Marta", "Maciek", "Daniel")  
    println(names)  
    // [Marta, Maciek, Daniel]  
    println(names.distinctBy { it.length })  
    // [Marta, Maciek]  
    println(names.associateBy { it.length }.values)
```

```
// [Marta, Daniel]
}
```

These functions are often used when we suspect that we accidentally have some kind of duplicates.

```
data class Person(val id: Int, val name: String) {
    override fun toString(): String = "$id: $name"
}

fun main() {
    val people = listOf(
        Person(0, "Alex"),
        Person(1, "Ben"),
        Person(1, "Carl"),
        Person(2, "Ben"),
        Person(0, "Alex"),
    )
    println(people.distinct())
    // [0: Alex, 1: Ben, 1: Carl, 2: Ben]
    println(people.distinctBy { it.id })
    // [0: Alex, 1: Ben, 2: Ben]
    println(people.distinctBy { it.name })
    // [0: Alex, 1: Ben, 1: Carl]
}
```

Sorting: sorted, sortedBy and sortedWith

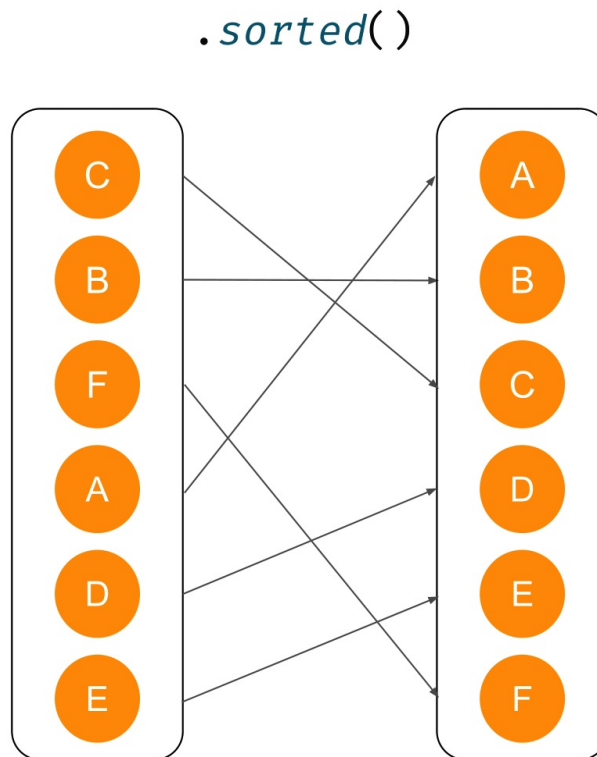
To have your collection elements organized in a concrete order, we can use sorting functions: `sorted`, `sortedBy` and `sortedWith`.

`sorted` can only be used on a list of elements with natural order for elements that implement the `Comparable` interface. The most important types with natural order are:

- `Int`, `Long`, `Double` and other basic classes representing numbers that are sorted from the lowest number to the highest.
- `Char` is treated as a number in UTF-16 code under the hood, so comparing two characters is like comparing their codes. Letters are organized in alphabetical order, but capital letters always come before lowercase letters. A space comes before all letters.
- `String`, whose natural order is lexicographical (this is a generalization of the alphabetical order that is used in dictionaries), where we start from comparing the first character (according to `Char` order); whenever two characters are equal, we are shifting the burden of the decision to the next character.

- Boolean places false before true. This is because false and true are typically represented by 0 and 1, respectively, and the natural order for numbers places 0 before 1.

```
fun main() {  
    println(listOf(4, 1, 3, 2).sorted())  
    // [1, 2, 3, 4]  
  
    println(listOf('b', 'A', 'a', ' ', 'B').sorted())  
    // [ , A, B, a, b]  
  
    println(listOf("Bab", "AAZ", "Bza", "A").sorted())  
    // [A, AAZ, Bab, Bza]  
  
    println(listOf(true, false, true).sorted())  
    // [false, true, true]  
}
```



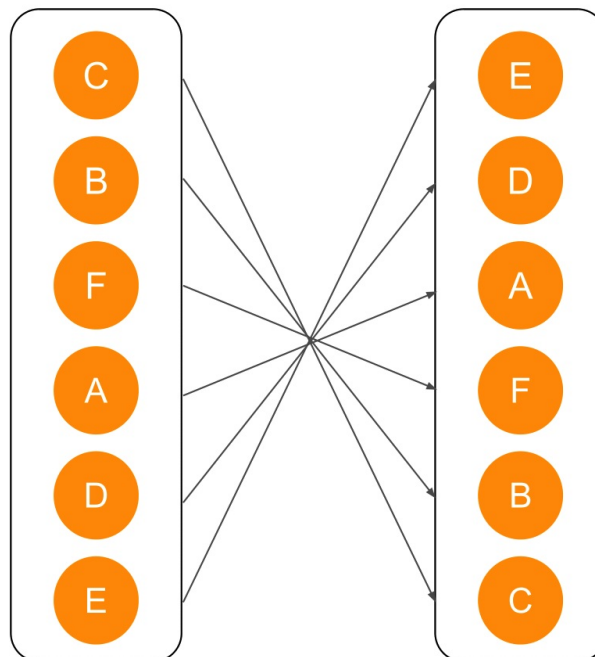
Kotlin standard library sorting functions are implemented in the way, so that equal elements remain in the same order (so we say that a stable sorting algorithm is being used).

```
fun main() {  
    val names = listOf("Ben", "Bob", "Bass", "Alex")  
    val sorted = names.sortedBy { it.first() }  
    println(sorted) // [Alex, Ben, Bob, Bass]  
}
```

To reverse the order of the elements in the list, use the `reversed` method.

```
fun main() {  
    println(listOf(4, 1, 3, 2).reversed())  
    // [2, 3, 1, 4]  
    println(listOf('C', 'B', 'F', 'A', 'D', 'E').reversed())  
    // [E, D, A, F, B, C]  
}
```

`.reversed()`



To reverse the sorting order, we can use the `sortedDescending` function, which gives the same result as first using `sorted` and then `reversed`.

```
fun main() {  
    println(listOf(4, 1, 3, 2).sortedDescending())  
    // [4, 3, 2, 1]  
    println(listOf(4, 1, 3, 2).sorted().reversed())  
}
```

```

// [4, 3, 2, 1]

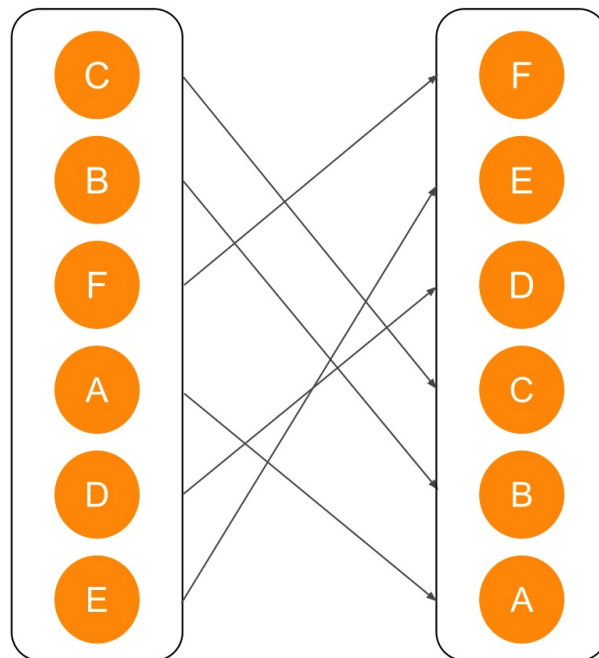
println(
  listOf('b', 'A', 'a', ' ', 'B')
    .sortedDescending()
)
// [b, a, B, A,  ]

println(
  listOf("Bab", "AAZ", "Bza", "A")
    .sortedDescending()
)
// [Bza, Bab, AAZ, A]

println(listOf(true, false, true).sortedDescending())
// [true, true, false]
}

```

.sortedDescending()



If we want to sort elements by one of their properties, we should use `sortedBy`, which sorts elements by the value its selector returns. For instance, if we have a list of students and we want to sort them by the semester, we can use `sortedBy` with a selector that reads the semester value.

```
// Sort students by the semester
students.sortedBy { it.semester }

// Sort students by surname
students.sortedBy { it.surname }
```

In other words, in `sortedBy`, the selector decides what value should be compared when we sort elements. This value needs to be comparable to itself (implement `Comparable<T>` interface).

```
fun main() {
    val names = listOf("Alex", "Bob", "Celine")

    // Sort by name length
    println(names.sortedBy { it.length })
    // [Bob, Alex, Celine]

    // Sort by last letter
    println(names.sortedBy { it.last() })
    // [Bob, Celine, Alex]
}
```

`sortedBy` also has a descending alternative called `sortedByDescending`.

```
fun main() {
    val names = listOf("Alex", "Bob", "Celine")

    // Sort by name length
    println(names.sortedByDescending { it.length })
    // [Celine, Alex, Bob]

    // Sort by last letter
    println(names.sortedByDescending { it.last() })
    // [Alex, Celine, Bob]
}
```

We might use `sortedBy` or `sortedByDescending` to sort users by their login, news by publication date, or tasks by priority.

```
// Users sorted by login
val usersSorted = users
    .sortedBy { it.login }

// News sorted starting from the newest
val newsFromLatest = news
    .sortedByDescending { it.publicationDate }

// News sorted starting from the oldest
val newsFromOldest = news
    .sortedBy { it.publicationDate }

// Tasks from the highest priority to the lowest
```

```
val tasksInOrder = tasks
    .sortedByDescending { it.priority }
```

The selectors of `sortedBy` and `sortedByDescending` accept `null`, which is considered less than all other values.

```
fun main() {
    val people = listOf(
        Person(1, "Alex"),
        Person(null, "Ben"),
        Person(2, null),
    )
    println(people.sortedBy { it.id })
    // [null: Ben, 1: Alex, 2: null]
    println(people.sortedBy { it.name })
    // [2: null, 1: Alex, null: Ben]
}
```

It gets more complicated when we need to sort by more than one property. For example, a typical governmental order of people's names requires sorting them by their surnames, and then people with the same surnames should be sorted by their first names. How can we implement this? Sorting by name first and then by surname would give us the correct result, but would be terribly inefficient. A much better solution is using `sortedWith`.

`sortedWith` is a function that returns a collection sorted according to a comparator it receives as an argument. The comparator is an object that implements the `Comparator` interface.

```
fun interface Comparator<T> {
    fun compare(a: T, b: T): Int
}
```

In many languages, it is popular to make an object that implements a comparator.

```
names.sortedWith(Comparator { o1, o2 ->
    when {
        o1.surname < o2.surname -> -1
        o1.surname > o2.surname -> 1
        o1.name < o2.name -> -1
        o1.name > o2.name -> 1
        else -> 0
    }
}))
```

We can do that in Kotlin too, but in most cases it is better to use one of the top-level functions from the standard library. For instance, we can use `compareBy` to create a comparator that first compares using one selector; then, if it considers two objects equal, it compares values using the next selector. This way, we can make a comparator with multiple sorting selectors, used lexicographically.

```
data class FullName(val name: String, val surname: String) {
    override fun toString(): String = "$name $surname"
}

fun main() {
    val names = listOf(
        FullName("B", "B"),
        FullName("B", "A"),
        FullName("A", "A"),
        FullName("A", "B"),
    )

    println(names.sortedBy { it.name })
    // [A A, A B, B B, B A]
    println(names.sortedBy { it.surname })
    // [B A, A A, B B, A B]
    println(names.sortedWith(compareBy(
        { it.surname },
        { it.name }
    )))
    // [A A, B A, A B, B B]
    println(names.sortedWith(compareBy(
        { it.name },
        { it.surname }
    )))
    // [A A, A B, B A, B B]
}
```

`sortedBy(selector)` under the hood is just
`sortedWith(compareBy(selector))`.

`sortedWith` and `compareBy` can be used for as many selectors as we want, which makes them really universal for complex sorting.

```
return recommendations.sortedWith(
    compareBy(
        { it.blocked }, // blocked to the end
        { !it.favourite }, // favorite at the beginning
        { calculateScore(it) },
    )
)
```

When we need to construct a different comparator, we have a variety of standard library functions. We can create a new comparator using:

- `compareBy`,
- `naturalOrder` (sorts with natural order),
- `reverseOrder` (sorts with the reverse of natural order),
- `nullsFirst` and `nullsLast` (both use natural order, but they also place nulls first or last).

Then, when we have a comparator, we can modify it using functions on `Comparator`, such as:

- `then` or `thenComparator`, both of which add another comparator that is used when the previous comparator considers elements equal;
- `thenBy`, which compares values using a selector when the previous comparator considers elements equal;
- `reversed`, which reverses the comparator order.

```
class Student(
    val name: String,
    val surname: String,
    val score: Double,
    val year: Int,
) {

    companion object {
        val ALPHABETICAL_ORDER =
            compareBy<Student>({ it.surname }, { it.name })
        val BY_SCORE_ORDER =
            compareByDescending<Student> { it.score }
        val BY_YEAR_ORDER =
            compareByDescending<Student> { it.year }
    }
}

fun presentStudentsForYearBook() = students
    .sortedWith(
        Student.BY_YEAR_ORDER.then(Student.ALPHABETICAL_ORDER)
    )

fun presentStudentsForTopScores() = students
    .sortedWith(
        Student.BY_YEAR_ORDER.then(Student.BY_SCORE_ORDER)
    )
```

Sorting mutable collections

If you want to sort a mutable collection, you can use the `sort` function. This is a part of classic collection processing as it modifies a mutable list instead of returning a processed one. The `sort` method is often confused with `sorted`. The `sort` method is an extension function on `MutableList` that, in contrast to

sorted, sorts a list and returns Unit. The sorted method is an extension function on Iterable that does not modify its receiver and returns a sorted collection.

```
fun main() {
    val list = listOf(4, 2, 3, 1)
    val sortedRes = list.sorted()
    // list.sort() is illegal
    println(list) // [4, 2, 3, 1]
    println(sortedRes) // [1, 2, 3, 4]

    val mutableList = mutableListOf(4, 2, 3, 1)
    val sortRes = mutableList.sort()
    println(mutableList) // [1, 2, 3, 4]
    println(sortRes) // kotlin.Unit
}
```

There are also sortBy, sortByDescending and sortWith, which respectively work similarly to sortedBy, sortedByDescending and sortedWith, but they modify a mutable collection instead of returning a new one.

Maximum and minimum

Another common situation is that we need to find extremes in a collection: the biggest or the smallest element. We could first sort the elements and then take the first or the last one, but such a solution would be far from optimal. Instead, we should use functions that start with the “max” or “min” prefix.

If we want to find an extreme using the natural order of the elements, use maxOrNull or minOrNull, both of which return null when a collection is empty.

```
fun main() {
    val numbers = listOf(1, 6, 2, 4, 7, 1)
    println(numbers.maxOrNull()) // 7
    println(numbers.minOrNull()) // 1
}
```

If we want to find an extreme according to a selector (similar to sortBy), use maxByOrNull or minByOrNull.

```
data class Player(val name: String, val score: Int)

fun main() {
    val players = listOf(
        Player("Jake", 234),
        Player("Megan", 567),
        Player("Beth", 123),
    )
}
```

```
println(players.maxByOrNull { it.score })
// Player(name=Megan, score=567)
println(players.minByOrNull { it.score })
// Player(name=Beth, score=123)
}
```

You can also find an extreme according to a comparator. In such a case, use `maxWithOrNull` or `minWithOrNull`.

```
data class FullName(val name: String, val surname: String)

fun main() {
    val names = listOf(
        FullName("B", "B"),
        FullName("B", "A"),
        FullName("A", "A"),
        FullName("A", "B"),
    )

    println(
        names
            .maxWithOrNull(compareBy(
                { it.surname },
                { it.name }
            ))
    )
    // FullName(name=B, surname=B)
    println(
        names
            .minWithOrNull(compareBy(
                { it.surname },
                { it.name }
            ))
    )
    // FullName(name=A, surname=A)
}
```

Another case is when you want to find an extreme value of a property: not the element that contains the extreme value but the value itself. For example, you have a list of students and you want to find their highest score. You could map the students to scores and then find the maximal value, or you could find the student with the highest score and get this score. However, both of these options do a lot of unnecessary operations. Instead, we should use the `maxOfOrNull` or `minOfOrNull` method with a selector that extracts a score (or `maxOf`/`minOf` if you are sure that your collection is not empty).

```
data class Player(val name: String, val score: Int)

fun main() {
    val players = listOf(
        Player("Jake", 234),
    )
}
```

```

        Player("Megan", 567),
        Player("Beth", 123),
    )

    println(players.map { it.score }.maxOrNull()) // 567
    println(players.maxByOrNull { it.score }?.score) // 567
    println(players.maxOfOrNull { it.score }) // 567
    println(players.maxOf { it.score }) // 567

    println(players.map { it.score }.minOrNull()) // 123
    println(players.minByOrNull { it.score }?.score) // 123
    println(players.minOfOrNull { it.score }) // 123
    println(players.minOf { it.score }) // 123
}

```

shuffled and random

We have learned how to sort elements, but we might also want to shuffle them. To get a random number from a collection, use `random` (or `randomOrNull` for possibly empty lists). To shuffle an iterable (to make its order random), use `shuffled`. For these functions, you can pass a custom `Random` object as an argument.

```

import kotlin.random.Random

fun main() {
    val range = (1..100)
    val list = range.toList()

    // `random` requires a collection
    println(list.random()) // random number from 1 to 100
    println(list.randomOrNull())
    // random number from 1 to 100

    println(list.random(Random(123))) // 7
    println(list.randomOrNull(Random(123))) // 7

    println(range.shuffled())
    // List with numbers in a random order
}

```

```

data class Character(val name: String, val surname: String)

fun main() {
    val characters = listOf(
        Character("Tamara", "Kurczak"),
        Character("Earl", "Gey"),
        Character("Ryba", "Luna"),
        Character("Cookie", "DePies"),
    )
    println(characters.random())
    // A random character,
    // like Character(name=Ryba, surname=Luna)
    println(characters.shuffled())
}

```

```
    // List with characters in a random order
}
```

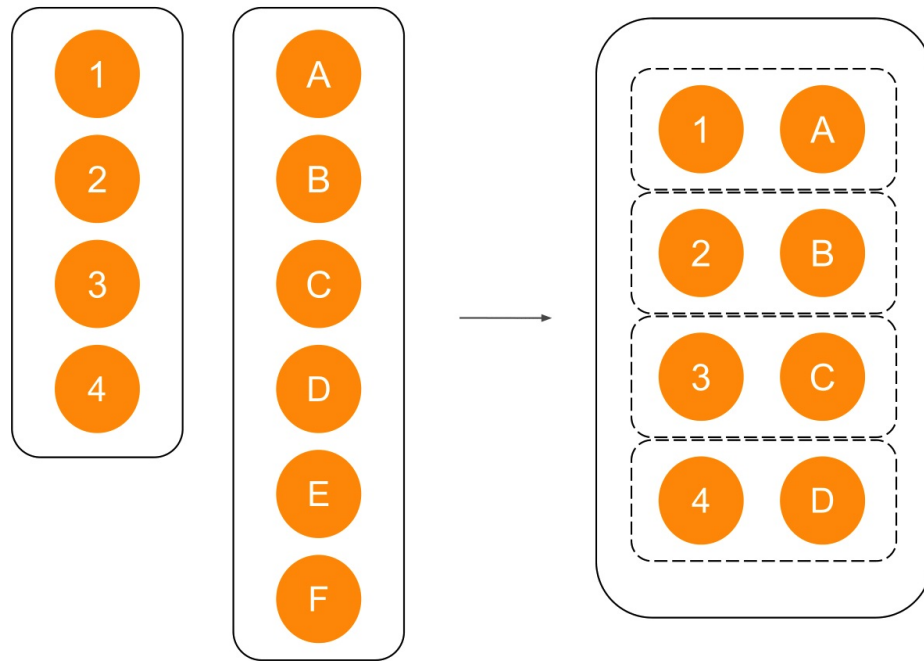
zip and zipWithNext

zip is used to connect two collections into one in a way that forms pairs of elements that are in the same positions. So, zip between List<T1> and List<T2> returns List<Pair<T1, T2>>. The result list ends when the shortest zipped collection ends.

```
fun main() {
    val nums = 1..4
    val chars = 'A'..'F'
    println(nums.zip(chars))
    // [(1, A), (2, B), (3, C), (4, D)]

    val winner = listOf(
        "Ashley",
        "Barbara",
        "Cyprian",
        "David",
    )
    val prices = listOf(5000, 3000, 1000)
    val zipped = winner.zip(prices)
    println(zipped)
    // [(Ashley, 5000), (Barbara, 3000), (Cyprian, 1000)]
    zipped.forEach { (person, price) ->
        println("$person won $price")
    }
    // Ashley won 5000
    // Barbara won 3000
    // Cyprian won 1000
}
```

`nums.zip(chars)`



The `zip` function reminds me of polonaise - a traditional Polish dance. One feature of this dance is that a line of pairs is separated down the middle, then these pairs reform when they meet again.



A still from the movie Pan Tadeusz, directed by Andrzej Wajda, presenting the polonaise dance.

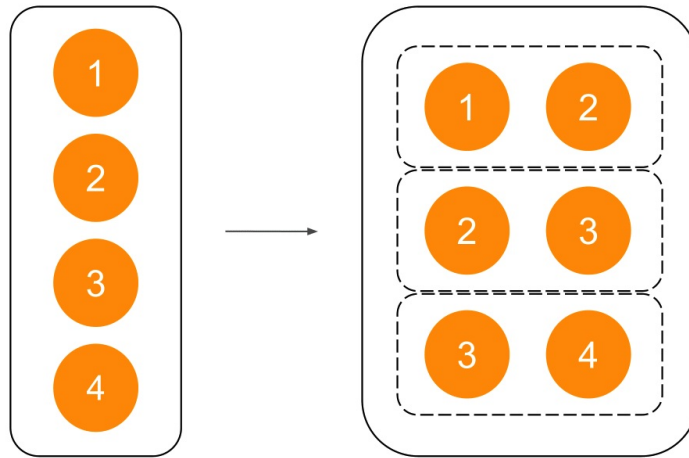
We can reverse zip operation using unzip, that transform a list of pairs into a pair of lists.

```
fun main() {  
    // zip can be used with infix notation  
    val zipped = (1..4) zip ('a'..'d')  
    println(zipped) // [(1, a), (2, b), (3, c), (4, d)]  
    val (numbers, letters) = zipped.unzip()  
    println(numbers) // [1, 2, 3, 4]  
    println(letters) // [a, b, c, d]  
}
```

When we need to connect adjacent elements of a collection into pairs, there is zipWithNext.

```
fun main() {  
    println((1..4).zipWithNext())  
    // [(1, 2), (2, 3), (3, 4)]  
  
    val person = listOf(  
        "Ashley",  
        "Barbara",  
        "Cyprian",  
    )  
    println(person.zipWithNext())  
    // [(Ashley, Barbara), (Barbara, Cyprian)]  
}
```

`.zipWithNext()`



There is also a variant of `zipWithNext`, that produces a list of results from a transformation, instead of a list of pairs.

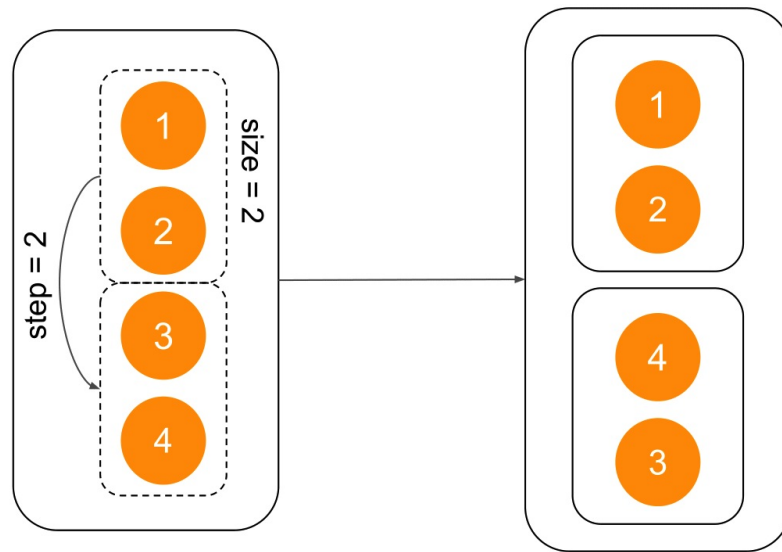
```
fun main() {  
    val person = listOf("A", "B", "C", "D", "E")  
    println(person.zipWithNext { prev, next -> "$prev$next" })  
    // [AB, BC, CD, DE]  
}
```

Windowing

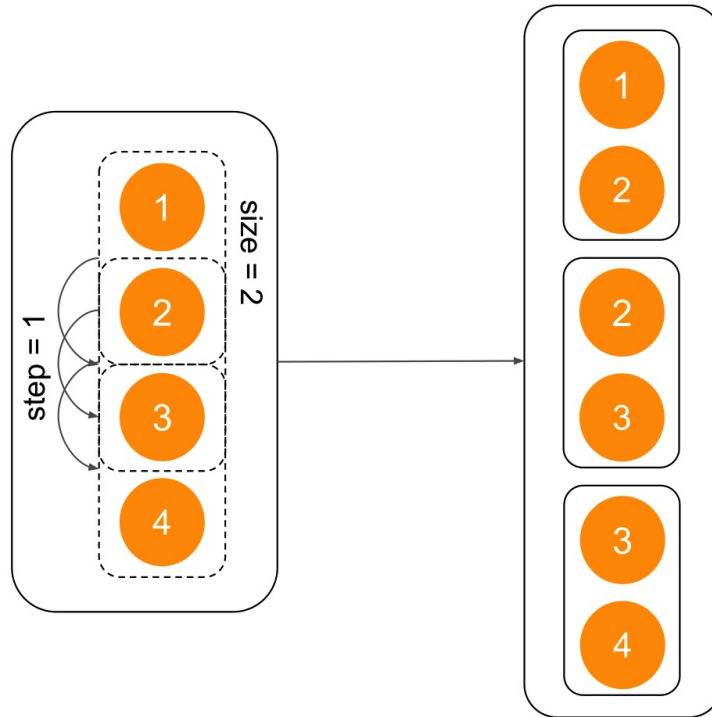
To connect adjacent elements into collections, the universal method is `windowed`, which returns a list of sublists of our list, where each is the next window of a given size. These sublists are made by sliding along this collection with the given step. In simpler words, you might imagine that `windowed` has a trolley of size `size` that makes a snapshot (a copy) of the elements below it and then makes a step of size `step`. When the end of the trolley falls off the collection, the process ends. However, suppose `partialWindows` is set to `true`. In that case, our

trolley needs to fully fall off the collection for the process to stop (with `partialWindows` for the process to stop, our trolley can extend past the end of the collection to include any remaining elements).

```
.windowed(size = 2, step = 2)
```



`.windowed(size = 2, step = 1)`



```
fun main() {  
    val person = listOf(  
        "Ashley",  
        "Barbara",  
        "Cyprian",  
        "David",  
    )  
    println(person.windowed(size = 1, step = 1))  
    // [[Ashley], [Barbara], [Cyprian], [David]]  
    // so similar to map { listOf(it) }  
  
    println(person.windowed(size = 2, step = 1))  
    // [[Ashley, Barbara], [Barbara, Cyprian],  
    // [Cyprian, David]]  
    // so similar to zipWithNext().map { it.toList() }  
  
    println(person.windowed(size = 1, step = 2))  
    // [[Ashley], [Cyprian]]  
  
    println(person.windowed(size = 2, step = 2))  
    // [[Ashley, Barbara], [Cyprian, David]]  
  
    println(person.windowed(size = 3, step = 1))  
    // [[Ashley, Barbara, Cyprian], [Barbara, Cyprian, David]]  
  
    println(person.windowed(size = 3, step = 2))  
}
```

```

// [[Ashley, Barbara, Cyprian]]

println(
    person.windowed(
        size = 3,
        step = 1,
        partialWindows = true
    )
)
// [[Ashley, Barbara, Cyprian], [Barbara, Cyprian, David],
// [Cyprian, David], [David]]

println(
    person.windowed(
        size = 3,
        step = 2,
        partialWindows = true
    )
)
// [[Ashley, Barbara, Cyprian], [Cyprian, David]]
}

```

The windowed method is really universal but also complicated. So, one function that builds on it is chunked.

```

// `chunked` implementation from Kotlin stdlib
fun <T> Iterable<T>.chunked(size: Int): List<List<T>> =
    windowed(size, size, partialWindows = true)

```

chunked divides our collection into chunks that are sub-collections of a certain size. It does not lose elements, so the last chunk might be smaller than the argument value.

```

fun main() {
    val person = listOf(
        "Ashley",
        "Barbara",
        "Cyprian",
        "David",
    )
    println(person.chunked(1))
    // [[Ashley], [Barbara], [Cyprian], [David]]

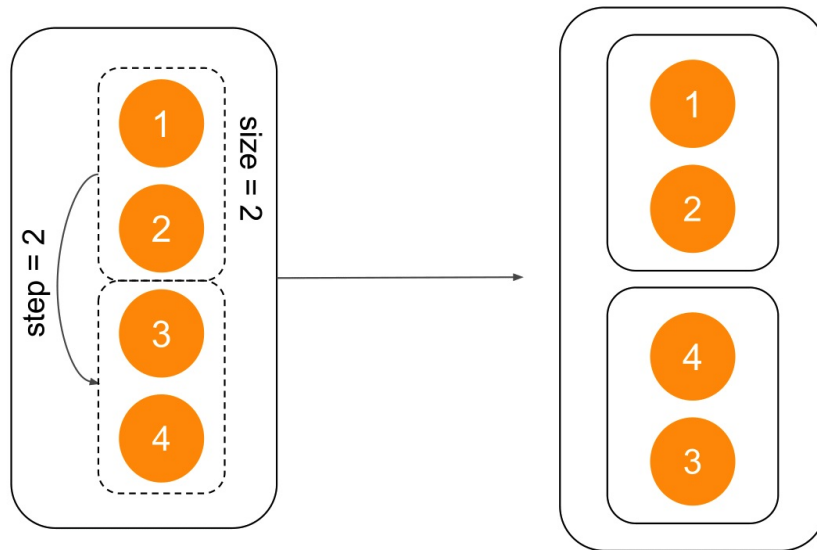
    println(person.chunked(2))
    // [[Ashley, Barbara], [Cyprian, David]]

    println(person.chunked(3))
    // [[Ashley, Barbara, Cyprian], [David]]

    println(person.chunked(4))
    // [[Ashley, Barbara, Cyprian, David]]
}

```

`.chunked(2)`



joinToString

When we need to transform an iterable into a string, and `toString` is not enough, we use the `joinToString` function. In its simplest form, it just presents elements one after another, separated with commas. However, `joinToString` is highly customisable with optional arguments:

- `separator` (", " by default) - decides what should be between the values in the produced string.
- `prefix` (" by default) and `postfix` (" by default) - decide what should be at the beginning and at the end of the string. `prefix` and `postfix` are also displayed for an empty collection.
- `limit` (-1 by default, which means no limit) and `truncated` ("..." by default) - `limit` decides how many elements can be displayed. Once the limit is reached, `truncated` is shown instead of the rest of the elements.

- transform (toString by default) - decides how each element should be transformed to String.

```
fun main() {  
    val names = listOf("Maja", "Norbert", "Ola")  
    println(names.joinToString())  
    // Maja, Norbert, Ola  
    println(names.joinToString { it.uppercase() })  
    // MAJA, NORBERT, OLA  
    println(names.joinToString(separator = "; "))  
    // Maja; Norbert; Ola  
    println(names.joinToString(limit = 2))  
    // Maja, Norbert, ...  
    println(names.joinToString(limit = 2, truncated = "etc."))  
    // Maja, Norbert, etc.  
    println(  
        names.joinToString(  
            prefix = "{names=[",  
            postfix = "}"  
        )  
    )  
    // {names=[Maja, Norbert, Ola]}  
}
```

Map, set and string processing

Most of the presented functions are extensions on either `Collection` or on `Iterable`, therefore they can be used not only on lists but also on sets. However, in addition to `List` and `Set`, there is also the third most important data structure: `Map`. It does not implement `Collection` or `Iterable`, so it needs custom collection processing functions. It has them! Most of the functions we have covered so far are also defined for the `Map` interface.

```

val names = mapOf("A" to "Alex", "B" to "Ben")
names.

```

Press ^ to choose the selected (or first) suggestion and insert a dot afterwards [Next Tip](#)

The biggest difference between collection and map processing methods stems from the fact that elements in maps are represented by both a key and a value. So, in functional arguments (predicates, transformations, selectors), instead of operating on values we operate on entries (the `Map.Entry` interface represents both a key and a value). When values are transformed (like in `map` or `flatMap`), the result type is `List`, unless we explicitly transform just keys or values (like in `mapValues` or `mapKeys`).

```

data class User(val id: Int, val name: String)

fun main() {
    val names: Map<Int, String> =
        mapOf(0 to "Alex", 1 to "Ben")
    println(names)
    // {0=Alex, 1=Ben}

    val users: List<User> = names
        .map { User(it.key, it.value) }
    println(users)
    // [User(id=0, name=Alex), User(id=1, name=Ben)]

    val usersById: Map<Int, User> = users
        .associateBy { it.id }
    println(usersById)
    // {0=User(id=0, name=Alex), 1=User(id=1, name=Ben)}
}

```

```

val namesById: Map<Int, String> = usersById
    .mapValues { it.value.name }
println(names)
// {0=Alex, 1=Ben}

val usersByName: Map<String, User> = usersById
    .mapKeys { it.value.name }
println(usersByName)
// {Alex=User(id=0, name=Alex), Ben=User(id=1, name=Ben)}
}

```

String is another important type. It is considered a collection of characters, but it does not implement Iterable or Collection. However, to support string processing, most collection processing functions are also implemented for String. However, String also supports many other operations, but these are better explained in the third part of the *Kotlin for developers* series: *Advanced Kotlin*.

```

val name: String = "Norbert"
name.

```

all {...} (predicate: (Char) -> Boolean) for CharSequence in kotlin.text	Boolean
any() for CharSequence in kotlin.text	Boolean
any {...} (predicate: (Char) -> Boolean) for CharSequence in kotlin.text	Boolean
asIterable() for CharSequence in kotlin.text	Iterable<Char>
asSequence() for CharSequence in kotlin.text	Sequence<Char>
associate {...} (transform: (Char) -> Pair<K, V>) for CharSequence in kotlin.text	Map<K, V>
associateBy {...} (keySelector: (Char) -> K) for CharSequence in kotlin.text	Map<K, Char>
associateBy(keySelector: (Char) -> K, valueTransform: (Char) -> V) for CharSequence in...	Map<K, V>
associateByTo(destination: M, keySelector: (Char) -> K) for CharSequence in kotlin.text	M
associateByTo(destination: M, keySelector: (Char) -> K, valueTransform: (Char) -> V) for CharS...	M
associateTo(destination: M, transform: (Char) -> Pair<K, V>) for CharSequence in kotlin.text	M
associateWith {...} (valueSelector: (Char) -> V) for CharSequence in kotlin.text	Map<Char, V>
associateWithTo(destination: M, valueSelector: (Char) -> V) for CharSequence in kotlin.text	M
chunked(size: Int) for CharSequence in kotlin.text	List<String>
chunked(size: Int, transform: (CharSequence) -> R) for CharSequence in kotlin.text	List<R>

Using them all together

Collection processing functions are often connected together, thus forming a flow that explains how a collection is processed step by step. Let's see a few practical examples. I will assume that we are writing an application for a university.

Let's assume that we have a list of students, and we need to find those who deserve internships. For this, students need to pass each semester and have an average grade above 4.0. Out of these students, we need to find the 10 with the

highest grade and sort them in official order. In the end, we need to form a list that can be printed. This is how this processing could be implemented:

```
students.filter { it.passing && it.averageGrade > 4.0 }
    .sortByDescending { it.averageGrade }
    .take(10)
    .sortedWith(compareBy({ it.surname }, { it.name }))
    .joinToString(separator = "\n") {
        "${it.name} ${it.surname}"
    }
}
```

Let's complicate this example a little by assuming that we need to assign the students to the appropriate internship amount. Once the students are sorted, we can zip them with the internships we prepared for the best students.

```
students.filter { it.passing && it.averageGrade > 4.0 }
    .sortedByDescending { it.averageGrade }
    .zip(INTERNSHIPS)
    .sortedWith(
        compareBy(
            { it.first.surname },
            { it.first.name }
        )
    )
    .joinToString(separator = "\n") { (student, internship) ->
        "${student.name} ${student.surname}, $$internship"
    }
}
```

```
private val INTERNSHIPS =
    List(5) { 5_000 } + List(10) { 3_000 }
```

To randomly divide the students into groups, you can use `shuffled` and `chunked`.

```
students.shuffled()
    .chunked(GROUP_SIZE)
```

To find the student with the highest result in each group, you can use `groupBy` and `maxByOrNull`.

```
students.groupBy { it.group }
    .map { it.values.maxByOrNull { it.result } }
```

These are just a few examples, but I'm sure you can find lots of great examples of collection processing in most bigger Kotlin projects. The collection processing operations have expanded the language capabilities such that Data Science, traditionally the realm of Python, and competitive coding challenges are

very approachable and natural in Kotlin. Its usage is universal and inter-domain, and I hope you will find the methods we have covered in this chapter useful.

Sequences

The way how collections are processed in Kotlin is not suitable for all use cases. Collections are loaded into memory to provide efficient and direct access to elements. That also means that collection processing functions, such as `map` or `filter`, each create a new collection. This is convenient in many use cases because the result is a collection, ready to be stored or used. However, it is not well-suited for more complex processing of large collections. In such cases, it is more efficient to describe all the processing steps in a single structure responsible for this whole process. Such a structure can optimize processing in terms of memory and the number of operations. This is what we use sequences for³¹.

I would like to demonstrate an extreme example. Let's say that we want to count the characters in a really large file. We could try to do this with collection processing:

```
val size = File("huge.file")
    .readLines()
    .sumOf { it.length }
```

The `readLines` function returns a list with all the lines. If the file is heavy, then this will be a heavy list. Allocating it in memory is not only a cost but it also leads to the risk of an `OutOfMemoryError`. A better option is to use `useLines`, which reads and processes the file line by line. This solution will be faster, and it's safer for our memory:

```
val size = File("huge.file").useLines {
    s.sumOf { it.length }
}
```

This is just an example of how a sequence can be used. Since this is a very important concept in Kotlin, let's analyze it.

What is a sequence?

People often miss the difference between `Iterable` and `Sequence`. This is understandable since even their definitions are nearly identical:

```
interface Iterable<out T> {
    operator fun iterator(): Iterator<T>
}

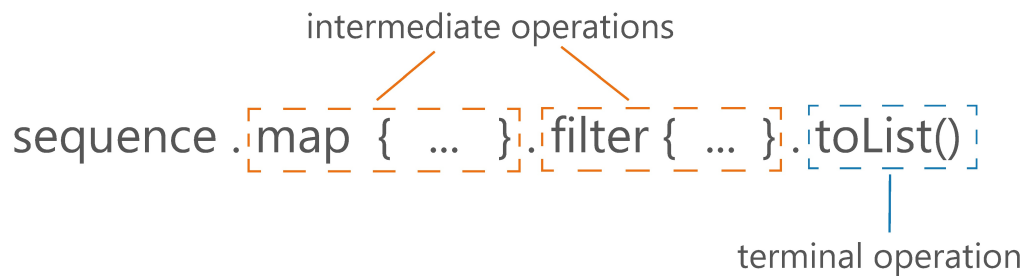
interface Sequence<out T> {
    operator fun iterator(): Iterator<T>
}
```

You could say that the only formal difference between them is the name. Both are Iterator types that allow the object to be used with a “for” loop. Although Iterable and Sequence are associated with totally different behaviors (have different contracts), nearly all their processing functions work differently. Sequences are lazy, so intermediate functions for Sequence processing (like filter or map) don’t do any calculations. Instead, they return a new Sequence that decorates the previous one with a new operation. All these computations are evaluated during terminal operations like `toList()` or `count()`. On the other hand, collection processing functions (those called on Iterable) are eager: they immediately perform all operations and return a new collection (typically a List).

```
public inline fun <T> Iterable<T>.filter(
    predicate: (T) -> Boolean
): List<T> {
    return filterTo(ArrayList<T>(), predicate)
}

public fun <T> Sequence<T>.filter(
    predicate: (T) -> Boolean
): Sequence<T> {
    return FilteringSequence(this, true, predicate)
}
```

As a result, collection processing operations are invoked as soon as they are used. Sequence processing functions are not invoked until a terminal operation (an operation that returns something else other than Sequence) is invoked. For instance, for Sequence, filter is an intermediate operation, so it doesn’t do any calculations; instead, it decorates the sequence with the new processing step. The calculations are done in a terminal operation like `toList()`. Thanks to that, sequence operations can be lazy.



Sequence processing consists of two types of operations: intermediate and terminal. Intermediate operations are those that return a new sequence. They decorate the previous step with a new action. All the processing happens in the terminal operation, which returns something different than a sequence.

```
fun main() {  
    val seq = sequenceOf(1, 2, 3)  
    val filtered = seq.filter { print("f$it "); it % 2 == 1 }  
    println(filtered) // FilteringSequence@...  
  
    val asList = filtered.toList() // terminal operation  
    // f1 f2 f3  
    println(asList) // [1, 3]  
  
    val list = listOf(1, 2, 3)  
    val listFiltered = list  
        .filter { print("f$it "); it % 2 == 1 }  
    // f1 f2 f3  
    println(listFiltered) // [1, 3]  
}
```

The fact that sequences are lazy in Kotlin is advantageous for a few reasons:

- They keep the natural order of operations.
- They do a minimal number of operations.
- They can have infinite number of elements.
- They are more memory efficient.

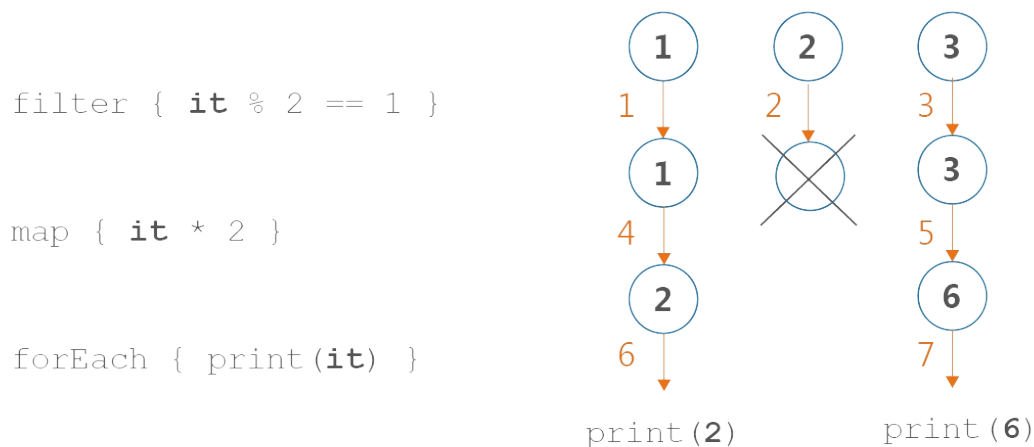
Let's talk about these advantages one by one.

Order is important

Because of how iterable and sequence processing are implemented, the ordering of their operations is different. In iterable processing, we take the first operation

and apply it to the whole collection, then we move to the next operation, etc. We will call this step-by-step or eager order.

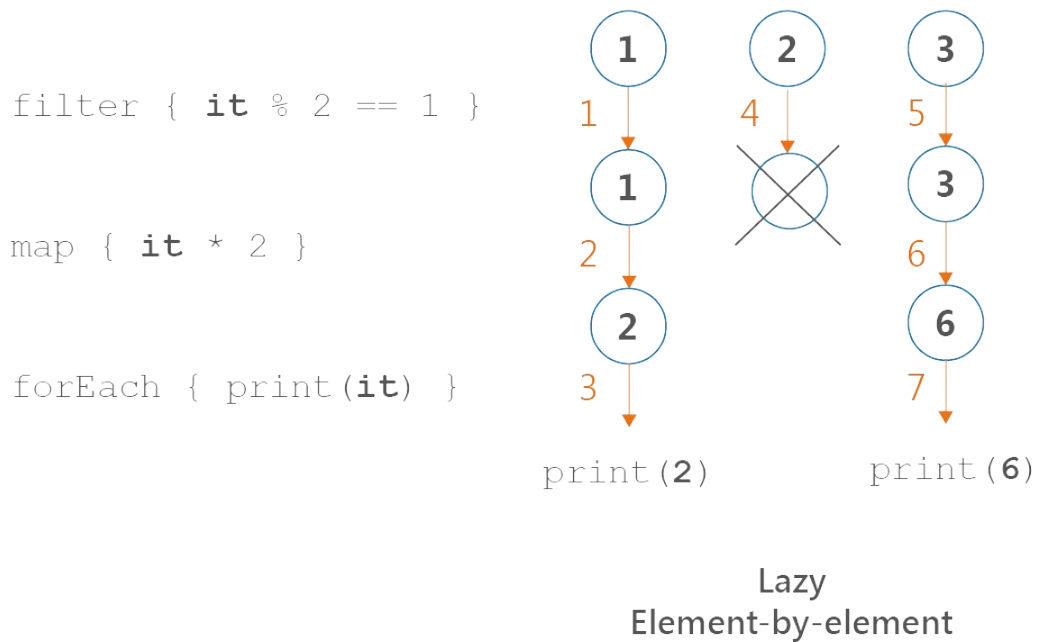
```
fun main() {
    listOf(1, 2, 3)
        .filter { print("F$it, "); it % 2 == 1 }
        .map { print("M$it, "); it * 2 }
        .forEach { print("E$it, ") }
    // Prints: F1, F2, F3, M1, M3, E2, E6,
}
```



The comparison between lazy processing (typical to Sequence) and eager processing (typical to Iterable) in terms of operations order (the numbers next to operations signalize in what order those operations are executed).

During sequence processing, we take the first element and apply all the operations to it, then we process the next element, and so on. We will call this element-by-element or lazy order.

```
fun main() {
    sequenceOf(1, 2, 3)
        .filter { print("F$it, "); it % 2 == 1 }
        .map { print("M$it, "); it * 2 }
        .forEach { print("E$it, ") }
    // Prints: F1, M1, E2, F2, F3, M3, E6,
}
```



Notice that if we were to implement these operations without any collection processing functions (using classic loops and conditions instead), we would have an element-by-element order, like in sequence processing:

```

fun main() {
    for (e in listOf(1, 2, 3)) {
        print("F$e, ")
        if (e % 2 == 1) {
            print("M$e, ")
            val mapped = e * 2
            print("E$mapped, ")
        }
    }
    // Prints: F1, M1, E2, F2, F3, M3, E6,
}

```

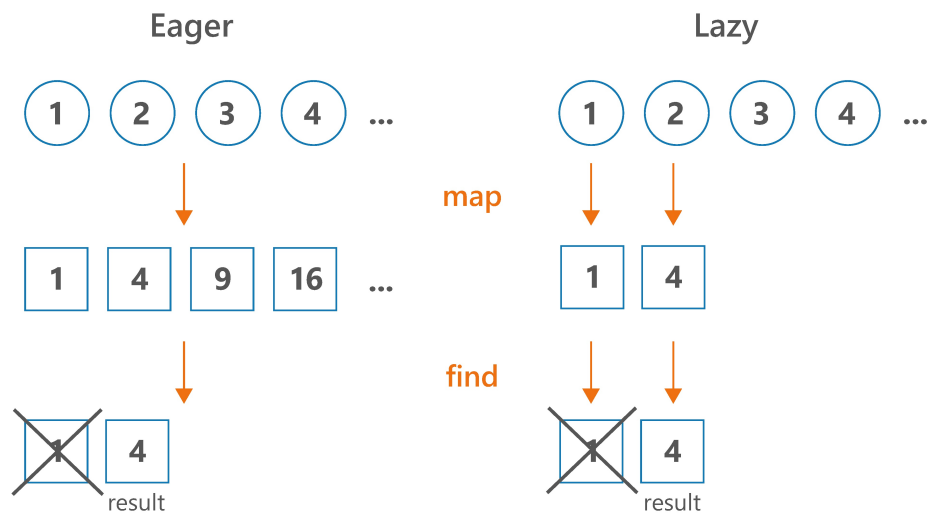
Therefore, the element-by-element order that is used in sequence processing is more natural. It also opens the door for low-level compiler optimizations as sequence processing can be optimized to basic loops and conditions (Haskell compiler actually does that with list fusion optimizations). I do not know anything about such optimizations at the time of writing this book, but maybe they will be introduced in the future.

Sequences do the minimum number of operations

Often we do not need to process a whole collection at every step to produce the result. Let's say that we have a collection with millions of elements, but, after processing, we only need to take the first 10. Why process all the other elements? Iterable processing doesn't have the concept of intermediate operations, so a processed collection is returned from every operation. Sequences do not need this, therefore they can do the minimum number of operations required to get the result.

Let's consider processing where we first map the items and then find one according to some criteria. An iterable will always map all the items first. A sequence will map the minimum number of items necessary.

```
fun main() {  
    val resI = (1..10).asIterable()  
        .map { print("M$it "); it * it }  
        .find { print("F$it "); it > 3 }  
    println(resI) // M1 M2 M3 M4 M5 M6 M7 M8 M9 M10 F1 F4 4  
  
    val resS = (1..10).asSequence()  
        .map { print("M$it "); it * it }  
        .find { print("F$it "); it > 3 }  
    println(resS) // M1 F1 M2 F4 4  
}
```



The difference between eager (characteristic of iterables) and lazy (characteristic of sequences or streams) processing.

Take a look at this example, where we have a few processing steps and finish our processing with `find`:

```
fun main() {
    (1..10).asSequence()
        .filter { print("F$it, "); it % 2 == 1 }
        .map { print("M$it, "); it * 2 }
        .find { it > 5 }
    // Prints: F1, M1, F2, F3, M3,

    (1..10)
        .filter { print("F$it, "); it % 2 == 1 }
        .map { print("M$it, "); it * 2 }
        .find { it > 5 }
    // Prints: F1, F2, F3, F4, F5, F6, F7, F8, F9, F10,
    // M1, M3, M5, M7, M9,
}
```

When we have some intermediate processing steps and our terminal operation does not necessarily need to iterate over all elements, using a sequence will most likely be better for your processing performance. We achieve all this easily, because sequence processing uses the same functions as iterable processing. Examples of operations that do not necessarily need to process all the elements are `first`, `find`, `take`, `any`, `all`, `none`, and `indexOf`.

Sequences perform the minimum number of operations, but only in case when they are used for processing. They do not store any data because they are not designed to do so. Instead, a sequence should be considered as a definition of the operations that will be used in the terminal operation. Whenever we call another terminal operation on this sequence, the elements are processed³².

```
fun main() {
    val s = (1..6).asSequence()
        .filter { print("F$it, "); it % 2 == 1 }
        .map { print("M$it, "); it * 2 }

    s.find { it > 3 } // F1, M1, F2, F3, M3,
    println()
    s.find { it > 3 } // F1, M1, F2, F3, M3,
    println()
    s.find { it > 3 } // F1, M1, F2, F3, M3,
    println()

    val l = (1..6)
        .filter { print("F$it, "); it % 2 == 1 }
        .map { print("M$it, "); it * 2 }
    // F1, F2, F3, F4, F5, F6, M1, M3, M5,

    l.find { it > 3 } // prints nothing
    l.find { it > 3 } // prints nothing
    l.find { it > 3 } // prints nothing
}
```

Sequences can be infinite

Thanks to the fact that sequences perform processing on demand, we can have infinite sequences. The two most important functions that are used to create an infinite sequence are `generateSequence` and `sequence`.

`generateSequence` takes as arguments the first element (seed) and a function that specifies how to calculate the next element.

```
fun main() {
    generateSequence(1) { it + 1 }
        .map { it * 2 }
        .take(10)
        .forEach { print("$it, ") }
    // Prints: 2, 4, 6, 8, 10, 12, 14, 16, 18, 20,
}
```

The second mentioned sequence generator, `sequence`, uses a suspending function³³ that generates the next number on demand. Whenever we ask for the next number, the sequence builder runs until a value is yielded with `yield`. The

execution then will be suspended until we ask for another number. Here is an infinite list of Fibonacci numbers, implemented using sequence:

```
import java.math.BigInteger

val fibonacci: Sequence<BigInteger> = sequence {
    var current = 1.toBigInteger()
    var prev = 0.toBigInteger()
    yield(prev)
    while (true) {
        yield(current)
        val temp = prev
        prev = current
        current += temp
    }
}

fun main() {
    print(fibonacci.take(10).toList())
    // [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
}
```

Notice the way that infinite sequences need to be processed, so their number of elements is limited. We cannot consume a sequence infinitely.

```
print(fibonacci.toList()) // Runs forever
```

Therefore, we either need to limit them using an operation such as `take`, or we need to use a terminal operation that does not need to perform all elements, such as `first`, `find` or `indexOf`. There are also operations for which sequences could be more efficient because they do not need to process all elements. However, notice that `any`, `all`, and `none` should not be used without being limited at first. `any` without a predicate can only return `true` or run forever. Similarly, `all` and `none` can only return `false`.

Sequences do not create collections at every processing step

Standard collection processing functions return a new collection at every step. In most cases it is a `List`. There could be benefits, because we have something ready to be used or stored after every step, but this comes at an extra cost: such collections need to be created and filled with data at every step.

```
val numbers = List(1_000_000) { it }

numbers
    .filter { it % 10 == 0 } // 1 collection here
```

```

        .map { it * 2 } // 1 collection here
        .sum()
// In total, 2 collections created under the hood

numbers
    .asSequence()
    .filter { it % 10 == 0 }
    .map { it * 2 }
    .sum()
// No collections created

```

This is especially a problem when we are dealing with big or heavy collections. Let's start from an extreme yet common case: file reading. Files can weigh gigabytes. Allocating all the data in a collection at every processing step would be a huge waste of memory. This is why we use sequences to process files by default.

As an example, let's analyze crimes in the city of Chicago. This city's public database of crimes committed since 2001³⁴ is accessible for free on the internet. This dataset currently weighs over 1.53 GB. Let's say our goal is to find how many crimes contain cannabis in their descriptions. This is how a naive solution using collection processing could look like:

```

// BAD SOLUTION, DO NOT USE COLLECTIONS FOR
// POSSIBLY BIG FILES
File("ChicagoCrimes.csv")
    .readLines() // returns List<String>
    .drop(1) // Drop labels
    .mapNotNull { it.split(",").getOrNull(6) }
    // Find description
    .filter { "CANNABIS" in it }
    .count()
    .let(::println)

```

This could produce on some machines `OutOfMemoryError`.



Exception in thread "main" java.lang.OutOfMemoryError: Java heap space

This could be expected. We create a collection, and then we have 3 intermediate processing steps, which means we have 4 collections in total. 3 of these contain the majority of this 1.53 GB data file, so in total, they consume more than 4.59 GB. This is a huge waste of memory. The correct implementation should involve

using a sequence, and we perform this using the `useLines` function, which always operates on a single line:

```
File("ChicagoCrimes.csv").useLines { lines ->
// The type of `lines` is Sequence<String>
    lines.drop(1) // Drop labels
        .mapNotNull { it.split(",").getOrNull(6) }
        // Find description
        .filter { "CANNABIS" in it }
        .count()
        .let { println(it) } // 318185
}
```

The second implementation is not only safer but also faster. Memory allocation and freeing it both take time. Using sequences for bigger files not only saves memory but also increases performance.

The fact that we create a new collection at every step is also a cost that manifests when dealing with collections with many elements. The difference between collections and sequence processing is that the processing of collections creates intermediate collections, unlike the sequence processing. However, this difference is not huge, mainly because intermediate temporary collections are created with the expected size; but when we add elements, we just place them in the next position. However, even cheap collection copying is still more expensive than avoiding copying at all. This is the main reason why we should **prefer to use Sequences for big collections with more than one processing step**.

By “big collections”, I mean either collections with tens of thousands of small elements or with a few huge (megabyte-sized) elements. These are not common situations, but they sometimes happen.

By one processing step, I mean more than a single function for collection processing. So, if you compare these two functions:

```
fun singleStepListProcessing(): List<Product> {
    return productsList.filter { it.bought }
}

fun singleStepSequenceProcessing(): List<Product> {
    return productsList.asSequence()
        .filter { it.bought }
        .toList()
}
```

You could notice that there is almost no difference in performance (actually, simple list processing is faster because its `filter` function is inlined). However, when you compare functions with more than one processing step (such as the functions below, which use `filter` and then `map`), the difference will be appreciable for bigger collections.

```
fun multipleStepsListProcessing(): List<ProductDto> {
    return productsList
        .filter { it.bought }
        .map { it.productDto() }
}

fun multipleStepsSequenceProcessing(): List<ProductDto> {
    return productsList.asSequence()
        .filter { it.bought }
        .map { it.productDto() }
        .toList()
}
```

When aren't sequences faster?

There are some operations where we don't profit from this sequence usage because we have to operate on the whole collection anyway. The `sorted` function is an example from Kotlin stdlib (currently it is the only example). It uses optimal implementation: it accumulates the `Sequence` into `List` and then uses `sort` from Java stdlib. The disadvantage is that this accumulation process takes some additional time compared to a `Collection` (although, if `Iterable` is not a `Collection` or an array, then the difference is not significant because it also has to be accumulated).

Whether or not `Sequence` should have methods such as `sorted` is controversial because sequences which have a method that requires all elements to calculate the next one are only partially lazy (evaluated when we need to get the first element), and they don't work on infinite sequences. `Sequence` was added because it is a popular function, and it is much easier to sort its values directly; however, Kotlin developers should remember about it, especially that it cannot be used with infinite sequences.

```
generateSequence(0) { it + 1 }.take(10).sorted().toList()
// [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
generateSequence(0) { it + 1 }.sorted().take(10).toList()
// Infinite time. Does not return.
```

The sorted function is a rare example of a processing step that is faster on Collection than on Sequence. Still, when we perform a few processing steps and then a single sorting function (or other function that needs to work on the whole collection), we could expect some performance improvements with sequence processing.

```
productsList.asSequence()
    .filter { it.bought }
    .map { it.price }
    .sorted()
    .take(10)
    .sum()
```

What about Java streams?

Java 8 introduced streams to allow collection processing. They perform and look similar to Kotlin sequences.

```
productsList.asSequence()
    .filter { it.bought }
    .map { it.price }
    .average()

productsList.stream()
    .filter { it.bought }
    .mapToDouble { it.price }
    .average()
    .orElse(0.0)
```

Java 8 streams are lazy and will be collected in the last (terminal) processing step. There are three significant differences between Java streams and Kotlin sequences:

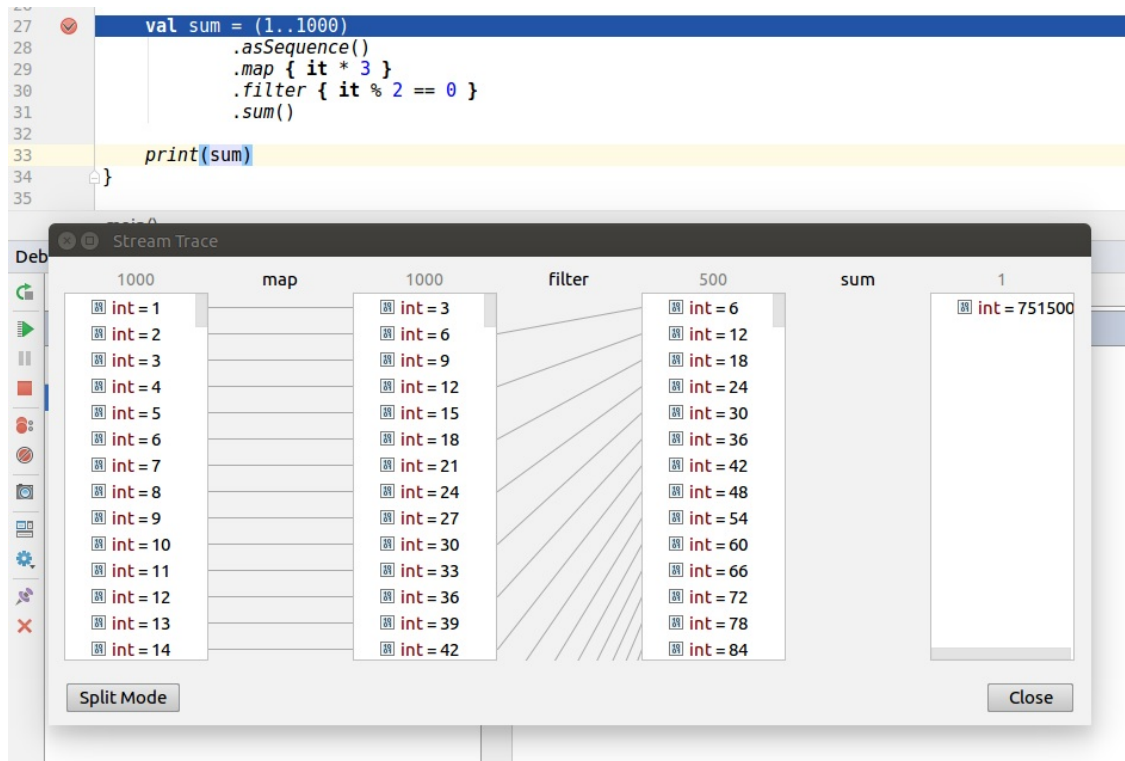
- Kotlin sequences have a lot of processing methods (because they are defined as extension functions), and they are generally easier to use (because Kotlin sequences were designed when Java streams were already being used; for instance, we can collect using `toList()` instead of `collect(Collectors.toList())`)
- Java stream processing can be started in parallel mode using a parallel function. This can give us a huge performance improvement in the context of a machine with multiple cores that are often unused (which is common nowadays). However, you should use this with caution because this feature has known pitfalls^{[35](#)}.

- Sequence is available on all Kotlin targets (Kotlin/JVM, Kotlin/JS, and Kotlin/Native) and in common modules, while Java streams require Java 8+ JVM.

In general, when we don't use parallel mode, it is hard to give a simple answer to whether Java streams or Kotlin sequences are more efficient. My suggestion is to only use Java streams rarely for computationally heavy processing where you can profit from the parallel mode. Otherwise, use Kotlin stdlib functions to have homogeneous and clean code that can be used on different platforms or on common modules.

Kotlin Sequence debugging

Both Kotlin Sequences and Java Streams have support in IntelliJ that helps us debug the flow of elements at every step. Java Streams require a plugin called “Java Stream Debugger”. Kotlin Sequences require a plugin named “Kotlin Sequence Debugger”, but this functionality is now integrated into the official Kotlin plugin. Here is a screen showing sequence processing at every step:



Summary

Collection and sequence processing are very similar and both support nearly the same processing methods. Yet, there are important differences between the two. Sequence processing is harder, as we generally keep elements in collections, therefore transforming a collection using sequence processing requires a transformation to a sequence and then back to a collection. Sequences are lazy, which brings some important advantages:

- They keep the natural order of operations.
- They perform the minimum number of operations.
- They can be infinite.
- They do not create intermediate collections at every step.

As a result, they are better for processing heavy objects or for bigger collections with more than one processing step. Sequences also have their own IDE debugger, which can help us visualize how elements are processed. Sequences are not designed to replace classic collection processing. You should use them only when there's a good reason, and you'll be rewarded with better performance and fewer memory problems.

Type Safe DSL Builders

There is a trend in programming: we like to move different kinds of definitions into the codebase. A well-known example is a build-tool configuration. It used to be standard practice to write such configurations in XML in build tools like Ant or Maven. Gradle, which can be considered a successor of Maven, defines its configuration in code. The `build.gradle` files that you might have seen in projects are just Groovy code:

```
// build.gradle
// Groovy
plugins {
    id 'java'
}

dependencies {
    implementation 'org.jetbrains.kotlin:kotlin-stdlib-jdk8'
    implementation "org.jetbrains.kotlinx:kotlinx-coroutines-core:1.6.0"
    testImplementation "io.mockk:mockk:1.12.1"
    testImplementation "org.junit.jupiter:junit-jupiter-api:5.8.2"
    testRuntimeOnly "org.junit.jupiter:junit-jupiter-engine:5.8.2"
}
```

Some dependencies are shortened to match book width.

Defining configurations in code makes working with them more convenient. First, this kind of environment is known to developers, so they know what they can and cannot do. It is possible to define helper functions, classes, use lambda expressions etc. However, Gradle was not completely satisfied with Groovy: it is too dynamic, suggestions practically don't work, and we often have no information about typos. These are the reasons why the new approach to define Gradle configurations is to use Kotlin:

```
// build.gradle.kts
// Kotlin
plugins {
    java
}

dependencies {
    implementation(kotlin("stdlib"))
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.6.0")
    testImplementation("io.mockk:mockk:1.12.1")
    testImplementation("org.junit.jupiter:junit-jupiter-api:5.8.2")
    testRuntimeOnly("org.junit.jupiter:junit-jupiter-engine:5.8.2")
}
```

In this chapter we will learn about the features that are used in the above code. When we write this code, at every point we can use concrete structures that were defined by the designer of this configuration API. This is why it is called a Domain Specific Language (DSL): creators define a small language that is specifically designed to describe something concrete using code, which in this case is a Gradle configuration.



Kotlin DSL is fully statically typed; so, at every point we are given suggestions of what we can do, and if you make a typo, it is immediately marked.

The motivation behind defining Domain-Specific Languages (DSLs) is to achieve fluent grammar when describing things and actions.

DSLs revolutionized how we define views on frontend applications. I believe that the biggest game-changer was React (a JavaScript library), which allowed us to define HTML in JavaScript. However, with Kotlin DSLs we can also implement React applications in Kotlin, and we can also define HTML for backend applications in Kotlin.

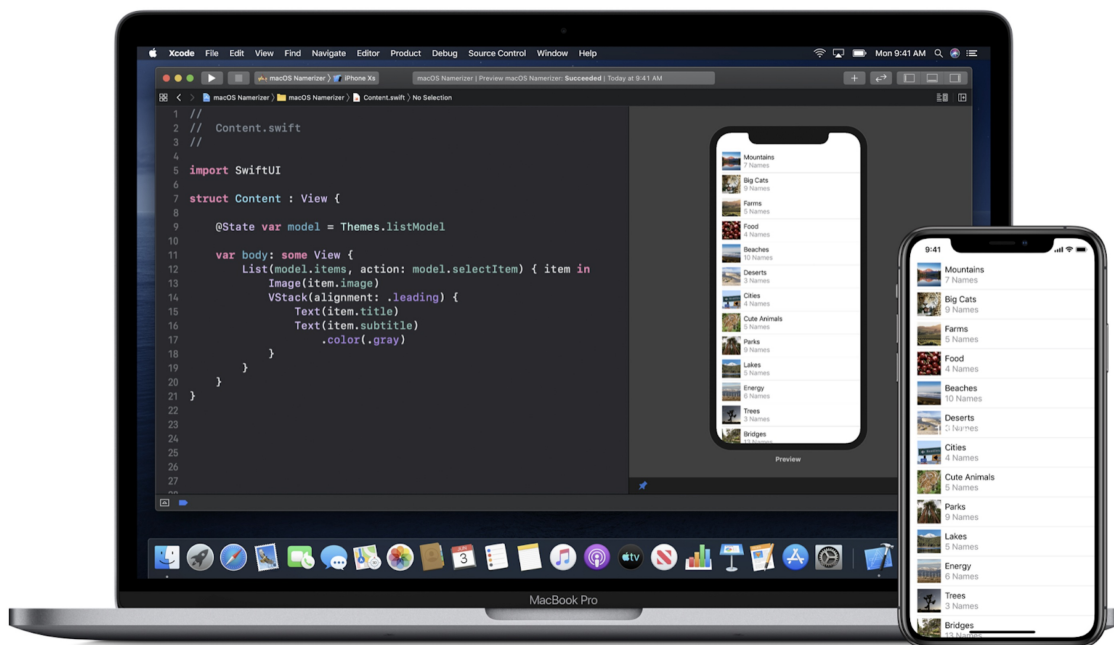
```
// Kotlin
body {
    div {
        a("https://kotlinlang.org") {
            target = ATarget.blank
            +"Main site"
        }
    }
}
```

```
}
+"Some content"
```

[Main site](#)
Some content

*HTML view generated from the above HTML
DSL.*

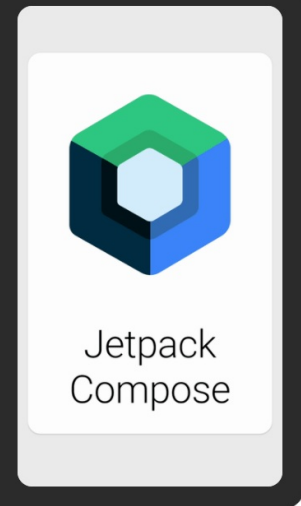
This approach also inspired other communities. At the time of writing this book, it is becoming standard practice to define iOS views using SwiftUI, which uses Swift DSL under its hood, and Android views are often defined using JetPack Compose, which uses Kotlin DSL^{[36](#)}.



```

@Composable
fun JetpackCompose() {
    Card {
        var expanded by remember { mutableStateOf(false) }
        Column(Modifier.clickable { expanded = !expanded }) {
            Image(painterResource(R.drawable.jetpack_compose))
            AnimatedVisibility(expanded) {
                Text(
                    text = "Jetpack Compose",
                    style = MaterialTheme.typography.h2,
                )
            }
        }
    }
}

```



The situation with desktop applications is similar. Here is a view defined using TornadoFX, which is built on top of JavaFX:

```

// Kotlin
class HelloWorld : View() {
    override val root = hbox {
        label("Hello world") {
            addClass(heading)
        }

        textfield {
            promptText = "Enter your name"
        }
    }
}

```



View from the above TornadoFX DSL

DSLs are also used on the backend. For example, Ktor framework API is based on Kotlin DSL. Thanks to that, endpoint definitions are simple and readable but also flexible and convenient to use.

```
fun Routing.api() {  
    route("news") {  
        get {  
            val newsData = NewsUseCase.getAcceptedNews()  
            call.respond(newsData)  
        }  
        get("propositions") {  
            requireSecret()  
            val newsData = NewsUseCase.getPropositions()  
            call.respond(newsData)  
        }  
    }  
    // ...  
}
```

DSL-based frameworks are also much more elastic than annotation-based ones. For instance, you can easily define several endpoints based on a list or map.

```
fun Routing.setupRedirect(redirect: Map<String, String>) {  
    for ((path, redirectTo) in redirect) {  
        get(path) {  
            call.respondRedirect(redirectTo)  
        }  
    }  
}
```

DSLs are considered highly readable, so more and more libraries use DSL-styled configurations instead of builders for their configurations.

```

@Configuration
class SecurityConfig: WebSecurityConfigurerAdapter() {
    override fun configure(http: HttpSecurity?) {
        http {
            authorizeRequests {
                authorize(anyRequest, authenticated)
            }
            formLogin {
                loginPage = "/log-in"
            }
        }
    }
}

```

Spring security can be configured with the Kotlin DSL.

DSLs are also used by some testing libraries. This is what an example test defined in Kotlin Test looks like:

```

class MyTests : StringSpec({
    "length should return size of string" {
        "hello".length shouldBe 5
    }
    "startsWith should test for a prefix" {
        "world" should startWith("wor")
    }
})

```

As you can see, DSLs are already widespread, and there are good reasons for this. They make it easy to define even complex and hierarchical data structures. Inside these DSLs, we can use everything that Kotlin offers, and we also have useful hints. It is likely that you have already used some Kotlin DSLs, but it is also important to know how to define them yourself. Even if you don't want to become a DSL creator, you'll become a better user.

A function type with a receiver

To understand how to make your own DSLs, it is important to understand the feature called *function type with a receiver*, which is a function type that

represents an extension function.

I believe that a good way to introduce a function type with a receiver is by starting with concepts we already know. In the *Anonymous functions* chapter, I explained that anonymous functions are defined like regular functions, but without names. This is also true for extension functions. The object that is produced by an *anonymous extension function* represents an extension function. Therefore, it can be called in a special way: on a receiver.

```
// Named extension function
fun String.myPlus1(other: String) = this + other

fun main() {
    println("A".myPlus1("B")) // AB

    // Anonymous extension function assigned to a variable
    val myPlus2 = fun String.(other: String) = this + other
    println(myPlus2.invoke("A", "B")) // AB
    println(myPlus2("A", "B")) // AB
    println("A".myPlus2("B")) // AB
}
```

So, we have an object that represents an extension function. It needs to have a type, but this type needs to be different from a type that represents a regular function. Yes, it needs to be a function type with a receiver.

We construct function types with a receiver the same way as regular function types, but they additionally define their receiver type:

- `User.() -> Unit` - a type representing an extension function on `User` that expects no arguments and returns nothing significant.
- `Int.(Int) -> Int` - a type representing an extension function on `Int` that expects a single argument of type `Int` and returns `Int`.
- `String.(String, String) -> String` - a function type representing an extension function on `String` that expects two arguments of type `String` and returns `String`.

The function stored in `myPlus2` is an extension function on `String`; it expects a single argument of type `String` and returns `String`, so its function type is `String.(String) -> String`.

```
fun main() {
    val myPlus2: String.(String) -> String =
        fun String.(other: String) = this + other
    println(myPlus2.invoke("A", "B")) // AB
}
```

```
println(myPlus2("A", "B")) // AB
println("A".myPlus2("B")) // AB
}
```

So, we know how to use anonymous extension functions, but now what we need is to define lambda expressions that represent extension functions. There is no special syntax for this. When a lambda expression is typed as a function type with receiver, it becomes a *lambda expression with a receiver*; as a result, it has an additional receiver inside its body (the `this` keyword).

```
fun main() {
    val myPlus3: String.(String) -> String = { other ->
        this + other
        // Inside, we can use receiver `this`,
        // that is of type `String`
    }
    // Here, there is no receiver, so `this` has no meaning
    println(myPlus3.invoke("A", "B")) // AB
    println(myPlus3("A", "B")) // AB
    println("A".myPlus3("B")) // AB
}
```

Simple DSL builders

The fact that lambda expressions with receivers change the meaning of `this` can help us introduce more convenient syntax to define some object properties. Imagine that you need to deal with classic *JavaBeans* objects: the initialized classes are empty, so we need to set all their properties using setters. These used to be quite popular in Java, and we can still find them in a variety of libraries. As an example, let's take a look at the following `Dialog` definition:

```
class Dialog {
    var title: String = ""
    var message: String = ""
    var okButtonText: String = ""
    var okButtonHandler: () -> Unit = {}
    var cancelButtonText: String = ""
    var cancelButtonHandler: () -> Unit = {}
    fun show() {
        /*...*/
    }
}

fun main() {
    val dialog = Dialog()
    dialog.title = "Some dialog"
    dialog.message = "Just accept it, ok?"
    dialog.okButtonText = "OK"
    dialog.okButtonHandler = { /*OK*/ }
    dialog.cancelButtonText = "Cancel"
}
```



```
    dialog.cancelButtonHandler = { /*Cancel*/ }  
    dialog.show()  
}
```

Referencing the dialog variable with every property we want to set is not very convenient. So, let's use a trick: if we use a lambda expression with a receiver of type Dialog, we can reference these properties implicitly because this can be hidden.

```
fun main() {  
    val dialog = Dialog()  
    val init: Dialog.() -> Unit = {  
        title = "Some dialog"  
        message = "Just accept it, ok?"  
        okButtonText = "OK"  
        okButtonHandler = { /*OK*/ }  
        cancelButtonText = "Cancel"  
        cancelButtonHandler = { /*Cancel*/ }  
    }  
    init.invoke(dialog)  
    dialog.show()  
}
```

The code above got a bit complicated, but we can extract the repetitive parts into a function, like showDialog.

```
fun showDialog(init: Dialog.() -> Unit) {  
    val dialog = Dialog()  
    init.invoke(dialog)  
    dialog.show()  
}  
  
fun main() {  
    showDialog {  
        title = "Some dialog"  
        message = "Just accept it, ok?"  
        okButtonText = "OK"  
        okButtonHandler = { /*OK*/ }  
        cancelButtonText = "Cancel"  
        cancelButtonHandler = { /*Cancel*/ }  
    }  
}
```

Now our function that shows a dialog is minimalistic and convenient. It is easy to understand how we set each property. We also have nice suggestions inside a function type with a receiver. This is our simplest DSL example.

Using apply

Instead of defining `showDialog` ourselves, we could use the generic `apply` function, which is an extension function on any type. It helps us create and call a function type with a receiver on any object we want.

```
// Simplified apply implementation
inline fun <T> T.apply(block: T.() -> Unit): T {
    this.block() // same as block.invoke(this)
    return this
}
```

In our case, we could just create a `Dialog`, apply all modifications, and then explicitly show it.

```
fun main() {
    Dialog().apply {
        title = "Some dialog"
        message = "Just accept it, ok?"
        okButtonText = "OK"
        okButtonHandler = { /*OK*/ }
        cancelButtonText = "Cancel"
        cancelButtonHandler = { /*Cancel*/ }
    }.show()
}
```

This is a better solution if showing a dialog is not repetitive code for us and we do not want to define the `showDialog` function. However, `apply` helps only in simple cases, and it is not enough for more complex multi-level object definitions.

Nevertheless, we will find `apply` useful for DSL definitions. We can simplify `showDialog` by using it to call `init`.

```
fun showDialog(init: Dialog.() -> Unit) {
    Dialog().apply(init).show()
}
```

Multi-level DSLs

Let's say that our `Dialog` has been refactored, and there is now a class that stores button properties:

```
class Dialog {
    var title: String = ""
    var message: String = ""
    var okButton: Button? = null
    var cancelButton: Button? = null

    fun show() {
```

```

        /*...*/
    }

    class Button {
        var message: String = ""
        var handler: () -> Unit = {}
    }
}

```

Now our showDialog is not enough because we need to create the buttons in the classic way:

```

fun main() {
    showDialog {
        title = "Some dialog"
        message = "Just accept it, ok?"
        okButton = Dialog.Button()
        okButton?.message = "OK"
        okButton?.handler = { /*OK*/ }
        cancelButton = Dialog.Button()
        cancelButton?.message = "Cancel"
        cancelButton?.handler = { /*Cancel*/ }
    }
}

```

However, we could apply the same trick as before, but this time to create buttons. We could make a small DSL for this.

```

fun makeButton(init: Dialog.Button.() -> Unit) {
    return Dialog.Button().apply(init)
}

fun main() {
    showDialog {
        title = "Some dialog"
        message = "Just accept it, ok?"
        okButton = makeButton {
            message = "OK"
            handler = { /*OK*/ }
        }
        cancelButton = makeButton {
            message = "Cancel"
            handler = { /*Cancel*/ }
        }
    }
}

```

This is better, but it's still not perfect. The user of our DSL needs to know that there is a makeButton function that is used to create a button. In general, we prefer to require users to remember as little as possible. Instead, we could make okButton and cancelButton methods inside Dialog to create buttons. Such functions are easily discoverable and their usage is really readable.

```

class Dialog {
    var title: String = ""
    var message: String = ""
    private var okButton: Button? = null
    private var cancelButton: Button? = null

    fun okButton(init: Button.() -> Unit) {
        okButton = Button().apply(init)
    }

    fun cancelButton(init: Button.() -> Unit) {
        cancelButton = Button().apply(init)
    }

    fun show() {
        /*...*/
    }

    class Button {
        var message: String = ""
        var handler: () -> Unit = {}
    }
}

fun showDialog(init: Dialog.() -> Unit) {
    Dialog().apply(init).show()
}

fun main() {
    showDialog {
        title = "Some dialog"
        message = "Just accept it, ok?"
        okButton {
            message = "OK"
            handler = { /*OK*/ }
        }
        cancelButton {
            message = "Cancel"
            handler = { /*Cancel*/ }
        }
    }
}

```

DslMarker

Our DSL builder for defining dialogs has one safety concern that we need to fix: by default, you can implicitly access elements from the outer receiver. In our example, this means that we can accidentally set the dialog title inside of okButton.

```

fun main() {
    showDialog {
        title = "Some dialog"
        message = "Just accept it, ok?"
        okButton {

```

```

        title = "OK" // This sets the dialog title!
        handler = { /*OK*/ }
    }
    cancelButton {
        message = "Cancel"
        handler = { /*Cancel*/ }
    }
}
}

```

This is an inconvenience because when you ask for suggestions inside `okButton`, elements will be suggested that should not be used. This also makes it easy to make a mistake.



To prevent these problems, we should use the `DslMarker` meta-annotation. A Meta-annotation is an annotation to an annotation class; so, to use `DslMarker`, we need to define our own annotation. In this case, we might call it `DialogDsl`. When we add this annotation before classes used in our DSL, it solves our safety problem³⁸. When we use it to annotate builder methods, it colors those functions' calls.

```

@DslMarker
annotation class DialogDsl

```

```

@DialogDsl
class Dialog {
    var title: String = ""
    var message: String = ""
    private var okButton: Button? = null
    private var cancelButton: Button? = null

    @DialogDsl
    fun okButton(init: Button.() -> Unit) {
        okButton = Button().apply(init)
    }

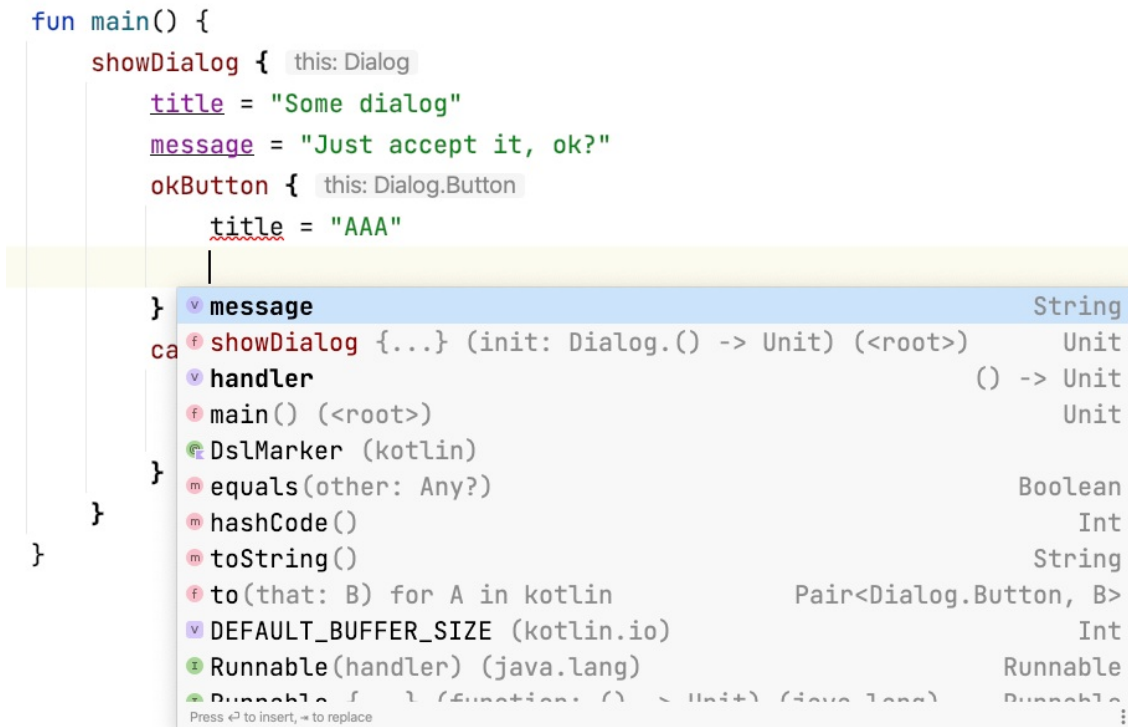
    @DialogDsl
    fun cancelButton(init: Button.() -> Unit) {
        cancelButton = Button().apply(init)
    }

    fun show() {
        /*...*/
    }

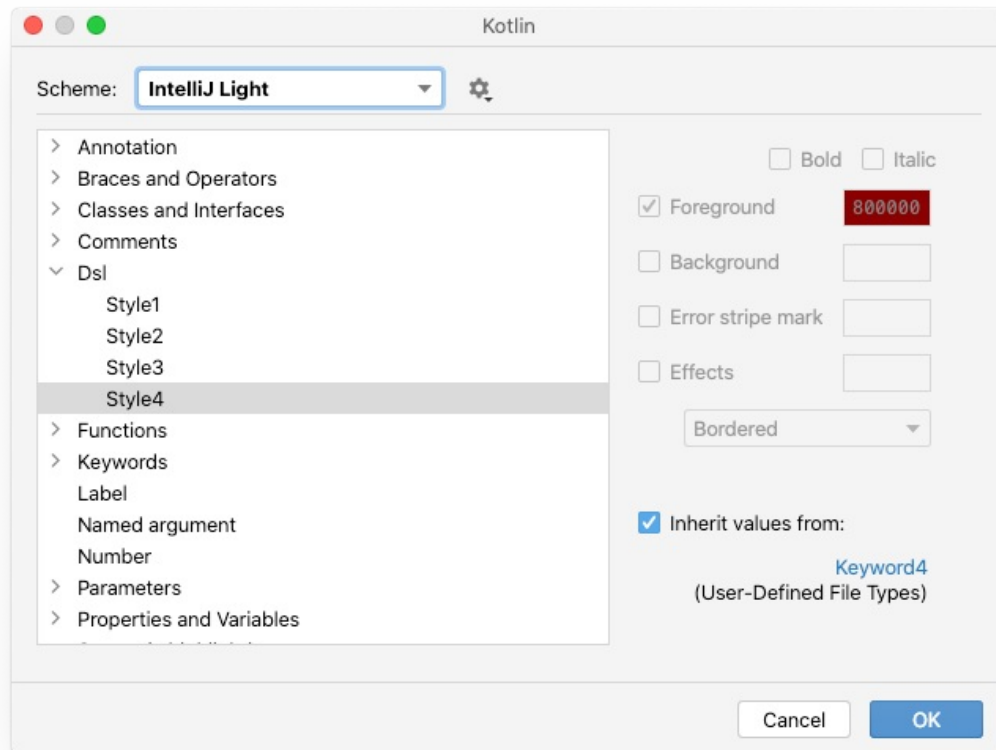
    @DialogDsl
    class Button {
        var message: String = ""
        var handler: () -> Unit = {}
    }
}

@DialogDsl
fun showDialog(init: Dialog.() -> Unit) {
    Dialog().apply(init).show()
}

```



As you might notice in the above image, DSL calls now have a different color (to me, it looks like burgundy). This color should be the same no matter which computer I start this code with. At the same time, DSLs can have one of four different colors that are specified in IntelliJ, and the style is chosen based on the hash of the DSL's annotation name. So, if you rename `DialogDs1` to something else, you will most likely change the color of this DSL function call.



The four possible styles for DSL elements can be customized in IntelliJ IDEA.

With `DslMarker`, we have a complete DSL example. Nearly all DSLs can be defined in the same way. To make sure we understand this completely, we will analyze a slightly more complicated example.

A more complex example

Previously, we built a DSL from bottom to top, but now we will go in the other direction and start with how we want our DSL to look. We will build a simple HTML DSL that defines some HTML with a header and a body with some text elements. In the end, we would like to support the following notation:

```
val html = html {  
    head {  
        title = "My websi" +  
            "te"  
        style("Some CSS1")  
        style("Some CSS2")  
    }  
}
```



```
}
body {
  h1("Title")
  h3("Subtitle 1")
  +"Some text 1"
  h3("Subtitle 2")
  +"Some text 2"
}
}
```

You can challenge yourself and try to implement it by yourself. I will start from the top, where the `html { ... }` is. What is that? This is a function call with a lambda expression that is used as an argument.

```
fun html(init: HtmlBuilder.() -> Unit): HtmlBuilder = TODO()
```

`head` and `body` only make sense inside `html`, so they need to be called on its receiver. We will define them inside `HtmlBuilder`. Since they have children, they will have receivers: `HeadBuilder` and (my favorite) `BodyBuilder`.

```
class HtmlBuilder {
  fun head(init: HeadBuilder.() -> Unit) {
    /*...*/
  }

  fun body(init: BodyBuilder.() -> Unit) {
    /*...*/
  }
}
```

Inside `head`, we can specify the title using a setter. So, `HeadBuilder` should have a `title` property. It also needs a function `style` in order to specify a style.

```
class HeadBuilder {
  var title: String = ""

  fun style(body: String) {
    /*...*/
  }
}
```

The situation is similar with `body`, which needs `h1` and `h3` methods. But what is `+"Some text 1"`? This is the unary plus operator on `String`^{[39](#)}. It's strange, but we need it. A plain value would not work because we need a function call to add a value to a builder. This is why it's become so common to use the `unaryPlus` operator in such cases.

```

class BodyBuilder {
    fun h1(text: String) {
        /*...*/
    }

    fun h3(text: String) {
        /*...*/
    }

    operator fun String.unaryPlus() {
        /*...*/
    }
}

```

With all these elements, our DSL definition shows no compilation errors; however, it's not yet functional because the functions are still empty. We need them to store all the values somewhere. For the sake of simplicity, I will store everything in the builder we just defined.

In HeadBuilder, I just need to store the defined styles. We will use a list.

```

class HeadBuilder {
    var title: String = ""
    private var styles: List<String> = emptyList()

    fun style(body: String) {
        styles += body
    }
}

```

In BodyBuilder, we need to keep the elements in order, so I will store them in a list, and I will use a dedicated classes to represent each view element type.

```

class BodyBuilder {
    private var elements: List<BodyElement> = emptyList()

    fun h1(text: String) {
        this.elements += H1(text)
    }

    fun h3(text: String) {
        this.elements += H3(text)
    }

    operator fun String.unaryPlus() {
        elements += Text(this)
    }
}

sealed interface BodyElement
data class H1(val text: String) : BodyElement
data class H3(val text: String) : BodyElement
data class Text(val text: String) : BodyElement

```

In head and body, we need to do the same as we previously did in `makeButton`. There are typically three steps:

1. Create an empty builder.
2. Fill it with data using the `init` function.
3. Store it somewhere.

So, head could be implemented like this:

```
fun head(init: HeadBuilder.() -> Unit) {  
    val head = HeadBuilder()  
    init.invoke(head)  
    // or init(head)  
    // or head.init()  
    this.head = head  
}
```

This can be simplified with `apply`. In head and body we store data in `HtmlBuilder`. In `html` we need to return the builder.

```
fun html(init: HtmlBuilder.() -> Unit): HtmlBuilder {  
    return HtmlBuilder().apply(init)  
}  
  
class HtmlBuilder {  
    private var head: HeadBuilder? = null  
    private var body: BodyBuilder? = null  
  
    fun head(init: HeadBuilder.() -> Unit) {  
        this.head = HeadBuilder().apply(init)  
    }  
  
    fun body(init: BodyBuilder.() -> Unit) {  
        this.body = BodyBuilder().apply(init)  
    }  
}
```

Now our builders collect all the data defined in the DSL. We can just parse it and make HTML text. Here is a complete example in which the `DslMarker` and `toString` functions present our HTML as text.

```
// DSL definition  
@DslMarker  
annotation class HtmlDsl  
  
@HtmlDsl  
fun html(init: HtmlBuilder.() -> Unit): HtmlBuilder {  
    return HtmlBuilder().apply(init)  
}
```

```

@HtmlDsl
class HtmlBuilder {
    private var head: HeadBuilder? = null
    private var body: BodyBuilder? = null

    @HtmlDsl
    fun head(init: HeadBuilder.() -> Unit) {
        this.head = HeadBuilder().apply(init)
    }

    @HtmlDsl
    fun body(init: BodyBuilder.() -> Unit) {
        this.body = BodyBuilder().apply(init)
    }

    override fun toString(): String =
        listOfNotNull(head, body)
            .joinToString(
                separator = "",
                prefix = "<html>\n",
                postfix = "</html>",
                transform = { "$it\n" }
            )
}

@HtmlDsl
class HeadBuilder {
    var title: String = ""
    private var cssList: List<String> = emptyList()

    @HtmlDsl
    fun css(body: String) {
        cssList += body
    }

    override fun toString(): String {
        val css = cssList.joinToString(separator = "") {
            "<style>$it</style>\n"
        }
        return "<head>\n<title>$title</title>\n$css</head>"
    }
}

@HtmlDsl
class BodyBuilder {
    private var elements: List<BodyElement> = emptyList()

    @HtmlDsl
    fun h1(text: String) {
        this.elements += H1(text)
    }

    @HtmlDsl
    fun h3(text: String) {
        this.elements += H3(text)
    }

    operator fun String.unaryPlus() {
        elements += Text(this)
    }
}

```

```

    }

    override fun toString(): String {
        val body = elements.joinToString(separator = "\n")
        return "<body>\n$body\n</body>"
    }
}

sealed interface BodyElement
data class H1(val text: String) : BodyElement {
    override fun toString(): String = "<h1>$text</h1>"
}

data class H3(val text: String) : BodyElement {
    override fun toString(): String = "<h3>$text</h3>"
}

data class Text(val text: String) : BodyElement {
    override fun toString(): String = text
}

// DSL usage
val html = html {
    head {
        title = "My website"
        css("Some CSS1")
        css("Some CSS2")
    }
    body {
        h1("Title")
        h3("Subtitle 1")
        +"Some text 1"
        h3("Subtitle 2")
        +"Some text 2"
    }
}

fun main() {
    println(html)
}
/*
<html>
<head>
<title>My website</title>
<style>Some CSS1</style>
<style>Some CSS2</style>
</head>
<body>
<h1>Title</h1>
<h3>Subtitle 1</h3>
Some text 1
<h3>Subtitle 2</h3>
Some text 2
</body>
</html>
*/

```

When should we use DSLs?

DSLs give us a way to define information. DSLs can be used to express any kind of information you want, but it is never clear to users how exactly this information will be later used. In Jetpack Compose, Anko, TornadoFX or HTML DSL, we trust that the view will be correctly built based on our definitions, but it is often hard to track exactly how this happens. DSLs are hard to debug, and their usage might confuse developers who are not used to them. How they are defined can be a cost - in both developer confusion and performance. DSLs are overkill when we can use other simpler features instead. However, they are really useful when we need to express:

- complicated data structures,
- hierarchical structures,
- a huge amount of data.

I remember a project that needed AD campaigns configuration. It initially defined them in a YAML file, but later they transformed it into a DSL. They did that to use code to define rules for when ads should be shown. As a benefit, they gave users better suggestions and flexibility. I could see sets of campaigns defined in a for-loop. YAML files shine for simple configurations, but DSLs have much more to offer for more complex cases.

Everything can be expressed without a DSL-like structure by using builders or just constructors. **DSLs are about boilerplate elimination of such structures.** You should consider using a DSL when you see repeatable boilerplate code and there are no simpler Kotlin features that can help.

Summary

A Domain Specific Language is a structure that defines a special language inside a language. Kotlin has features that allow us to make type-safe, readable, and easy-to-use DSLs, which can simplify creating complex objects or hierarchies like HTML code or configurations. On the other hand, DSL implementations might be confusing or difficult for new developers, and they are hard to define. This is why they should only be used when they offer real value. This is also why they are also preferably defined in libraries rather than in applications. It is not easy to make a good DSL, but a well-defined DSL can make our project much better.

Scope functions

There is a group of minimalistic but useful inline functions from the standard library called *scope functions*. This group typically includes `let`, `apply`, `also`, `run` and `with`. Some developers also include `takeIf` and `takeUnless` in this group. They are all extensions on any generic type^{[40](#)}. All scope functions are just a few lines long. Let's discuss their usages and how they work, starting with the functions I find most useful.

let

```
// `let` implementation without contract
inline fun <T, R> T.let(block: (T) -> R): R = block(this)
```

`let` is a very simple function, yet it is used in many Kotlin idioms. It can be compared to the `map` function but for a single object: it transforms an object using a lambda expression.

```
fun main() {
    println(listOf("a", "b", "c").map { it.uppercase() })
    // [A, B, C]
    println("a".let { it.uppercase() }) // A
}
```

Let's see its common use cases.

Mapping a single object

To understand how `let` is used, let's imagine that you need to read a zip file with buffering, unpack it, and read an object from the result. On JVM, we use input streams for such operations. We first create a `FileInputStream` to read a file, and then we decorate it with classes that add the capabilities we need.

```
val fis = FileInputStream("someFile.gz")
val bis = BufferedInputStream(fis)
val gis = ZipInputStream(bis)
val ois = ObjectInputStream(gis)
val someObject = ois.readObject()
```

This pattern is not very readable because we create plenty of variables that are used only once. We can easily make a mistake, for instance by using an incorrect

variable at any step. How can we improve it? By using the `let` function! We can first create `FileInputStream`, and then decorate it using `let`:

```
val someObject = FileInputStream("someFile.gz")
    .let { BufferedInputStream(it) }
    .let { ZipInputStream(it) }
    .let { ObjectInputStream(it) }
    .readObject()
```

If you prefer, you can also use constructor references^{[41](#)}:

```
val someObject = FileInputStream("someFile.gz")
    .let(::BufferedInputStream)
    .let(::ZipInputStream)
    .let(::ObjectInputStream)
    .readObject()
```

Using `let`, we can form a nice flow of how an element is transformed. What is more, if a nullability is introduced at any step, we can use `let` conditionally with a safe call. To see this in practice, let's imagine that we are implementing a service that, based on a user token, responds with this user's active courses.

```
class CoursesService(
    private val userRepository: UserRepository,
    private val coursesRepository: CoursesRepository,
    private val userCoursesFactory: UserCoursesFactory,
) {
    // Imperative approach, without let
    fun getActiveCourses(token: String): UserCourses? {
        val user = userRepository.getUser(token)
        ?: return null
        val activeCourses = coursesRepository
            .getActiveCourses(user.id) ?: return null
        return userCoursesFactory.produce(activeCourses)
    }

    // Functional approach, using let
    fun getActiveCourses(token: String): UserCourses? =
        userRepository.getUser(token)
            ?.let { coursesRepository.getActiveCourses(it.id) }
            ?.let { userCoursesFactory::produce }
}
```

In these cases, `let` is not necessary, but it's very convenient. I see similar usage quite often, especially on backend applications. It makes our functions form a nice flow of data, and it lets us easily control the scope of each variable. It also has downsides, such as the fact that debugging is harder, so you need to decide yourself whether to use this approach in your applications.

The problem with member extension functions

At this point, it is worth mentioning that there is an ongoing discussion about transforming objects from one class to another. Let's say that we need to transform from `UserCreationRequest` to `UserDto`. The typical Kotlin way is to define a `toUserDto` or `toDomain` method (either a member function or an extension function).

```
class UserCreationRequest(  
    val id: String,  
    val name: String,  
    val surname: String,  
)  
  
class UserDto(  
    val userId: String,  
    val firstName: String,  
    val lastName: String,  
)  
  
fun UserCreationRequest.toUserDto() = UserDto(  
    userId = this.id,  
    firstName = this.name,  
    lastName = this.surname,  
)
```

The problem arises when the transformation function needs to use some external services. It needs to be defined in a class, and defining member extension functions is an anti-pattern^{[42](#)}.

```
class UserCreationRequest(  
    val name: String,  
    val surname: String,  
)  
  
class UserDto(  
    val userId: String,  
    val firstName: String,  
    val lastName: String,  
)  
  
class UserCreationService(  
    private val userRepository: UserRepository,  
    private val idGenerator: IdGenerator,  
) {  
    fun addUser(request: UserCreationRequest): User =  
        request.toUserDto()  
            .also { userRepository.addUser(it) }  
            .toUser()  
  
    // Anti-pattern!  
    private fun UserCreationRequest.toUserDto() = UserDto(  

```

```

        userId = idGenerator.generate(),
        firstName = this.name,
        lastName = this.surname,
    )
}

```

A good solution to this problem is defining transformation functions as regular functions in such cases, and if we want to call them “on an object”, just use `let`.

```

class UserCreationRequest(
    val name: String,
    val surname: String,
)

class UserDto(
    val userId: String,
    val firstName: String,
    val lastName: String,
)

class UserCreationService(
    private val userRepository: UserRepository,
    private val idGenerator: IdGenerator,
) {
    fun addUser(request: UserCreationRequest): User =
        request.let { createUserDto(it) }
        // or request.let(::createUserDto)
        .also { userRepository.addUser(it) }
        .toUser()

    private fun createUserDto(request: UserCreationRequest) =
        UserDto(
            userId = idGenerator.generate(),
            firstName = request.name,
            lastName = request.surname,
        )
}

```

This approach works just as well when object creation is extracted into a class, like `UserDtoFactory`.

```

class UserCreationService(
    private val userRepository: UserRepository,
    private val userDtoFactory: UserDtoFactory,
) {
    fun addUser(request: UserCreationRequest): User =
        request.let { userDtoFactory.produce(it) }
        .also { userRepository.addUser(it) }
        .toUser()

    // or
    // fun addUser(request: UserCreationRequest): User =
    //     request.let(userDtoFactory::produce)
    //     .also(userRepository::addUser)
}

```

```
//      .toUser()  
}
```

Moving an operation to the end of processing

The second typical use case for `let` is when we want to move an operation to the end of processing. Let's get back to our example, where we were reading an object from a zip file, but this time we will assume that we need to do something with that object in the end. For simplification, we might be printing it. Again, we face the same problem: we either need to introduce a variable or wrap the processing with a misplaced print call.

```
// Not good, not terrible  
val someObject = FileInputStream("/someFile.gz")  
    .let(::BufferedInputStream)  
    .let(::ZipInputStream)  
    .let(::ObjectInputStream)  
    .readObject()  
println(someObject)  
  
// Terrible  
print(  
    FileInputStream("/someFile.gz")  
        .let(::BufferedInputStream)  
        .let(::ZipInputStream)  
        .let(::ObjectInputStream)  
        .readObject()  
)
```

The solution to this problem is to use `let` (or another scope function) to invoke `print` “on the result”.

```
FileInputStream("/someFile.gz")  
    .let(::BufferedInputStream)  
    .let(::ZipInputStream)  
    .let(::ObjectInputStream)  
    .readObject()  
    .let(::print)
```

Some developers will argue that in such cases one should use `also` instead of `let`. The reasoning is that `let` is a transformation function and should therefore have no side effects, while `also` is dedicated to use for side effects. On the other hand, using `let` in such cases is popular.

This approach allows us to use safe-calls and call operations only on non-null objects.

```
FileInputStream("/someFile.gz")
    .let(::BufferedInputStream)
    .let(::ZipInputStream)
    .let(::ObjectInputStream)
    .readObject()
    ?.let(::print)
```

Dealing with nullability

The `let` function (and nearly all other scope functions) is called on an object, so it can be called with a safe call. We've already seen a few examples of how this capability helped us in the previous use cases. But it goes even further: `let` is often called just to help with nullability. To see this, let's consider the following example, where we want to print the user name if the user is not `null`. Smart casting does not work for variables because they can be modified by another thread. The easiest solution uses `let`.

```
class User(val name: String)

var user: User? = null

fun showUserNameIfPresent() {
    // will not work, because cannot smart-cast a property
    // if (user != null) {
    //     println(user.name)
    // }

    // works
    // val u = user
    // if (u != null) {
    //     println(u.name)
    // }

    // perfect
    user?.let { println(it.name) }
}
```

In this solution, if `user` is `null`, `let` is not called (due to the safe call used), and nothing happens. If `user` is not-`null`, `let` is called, so it calls `println` with the user name. This solution is fully thread-safe even in extreme cases: if `user` is not `null` during the safe call, and it then changes to `null` straight after that, printing the name will work fine because it is the reference to the user that was used at the time of the nullability check.

Some developers will again argue that in such cases one should use `also` instead of `let`; again, using `let` for null checks is popular.

These are the key cases where `let` is used. As you can see, it is pretty useful but there are other scope functions with similar characteristics. Let's see these, starting from the one mentioned a few times already: `also`.

`also`

```
// `also` implementation without contract
inline fun <T> T.also(block: (T) -> Unit): T {
    block(this)
    return this
}
```

We have mentioned the use of `also` already, so let's discuss it. It is pretty similar to `let`, but instead of returning the result of its lambda expression, it returns the object it is invoked on. So, if `let` is like `map` for a single object, then `also` can be considered an `onEach` for a single object, as `also` returns the object as it is.

`also` is used to invoke an operation on an object. Such operations typically include some side effects. We've used it already to add a user to our database.

```
fun addUser(request: UserCreationRequest): User =
    request.toUserDto()
        .also { userRepository.addUser(it) }
        .toUser()
```

It can be also used for all kinds of additional operations, like printing logs or storing a value in a cache.

```
fun addUser(request: UserCreationRequest): User =
    request.toUserDto()
        .also { userRepository.addUser(it) }
        .also { log("User created: $it") }
        .toUser()

class CachingDatabaseFactory(
    private val databaseFactory: DatabaseFactory,
) : DatabaseFactory {
    private var cache: Database? = null

    override fun createDatabase(): Database = cache
        ?: databaseFactory.createDatabase()
            .also { cache = it }
}
```

As mentioned already, `also` can also be used instead of `let` to unpack a nullable object or move an operation to the end.

```
class User(val name: String)

var user: User? = null

fun showUserNameIfPresent() {
    user?.also { println(it.name) }
}

fun readAndPrint() {
    FileInputStream("/someFile.gz")
        .let(::BufferedInputStream)
        .let(::ZipInputStream)
        .let(::ObjectInputStream)
        .readObject()
        ?.also(::print)
}
```

takeIf and takeUnless

```
// `takeIf` implementation without contract
inline fun <T> T.takeIf(predicate: (T) -> Boolean): T? {
    return if (predicate(this)) this else null
}

// `takeUnless` implementation without contract
inline fun <T> T.takeUnless(predicate: (T) -> Boolean): T? {
    return if (!predicate(this)) this else null
}
```

We already know that `let` is like a `map` for a single object. We know that `also` is like an `onEach` for a single object. So, now it's time to learn about `takeIf` and `takeUnless`, which are like `filter` and `filterNot` for a single object.

Depending on what their predicates return, these functions either return the object they were invoked on, or `null`. `takeIf` returns an untouched object if the predicate returned `true`, and it returns `null` if the predicate returned `false`. `takeUnless` is like `takeIf` with a reversed predicate result (so `takeUnless(pred)` is like `takeIf { !pred(it) }`).

We use these functions to filter out incorrect objects. For instance, if you want to read a file only if it exists.

```
val lines = File("SomeFile")
    .takeIf { it.exists() }
    ?.readLines()
```

We use such checks for safety. For example, if a file does not exist, `readLines` throws an exception. Replacing incorrect objects with `null` helps us handle them

safely. It also helps us drop incorrect results.

```
class UserCreationService(  
    private val userRepository: UserRepository,  
) {  
    fun readUser(token: String): User? =  
        userRepository.findUser(token)  
            .takeIf { it.isValid() }  
            ?.toUser()  
}
```

apply

```
// `apply` implementation without contract  
inline fun <T> T.apply(block: T.() -> Unit): T {  
    block()  
    return this  
}
```

Moving into a slightly different kind of scope function, it's time to present `apply`, which we already used in the *DSL* chapter. It works like `also` in that it is called on an object and it returns it, but it introduces an essential change: its parameter is not a regular function type but a function type with a receiver.

This means that if you take `also` and replace it with `apply`, and you replace the argument (typically `it`) with a receiver (`this`) inside the lambda, the resulting code will be the same as before. However, this small change is actually really important. As we learned in the *DSL* chapter, changing receivers can be both a big convenience and a big danger. This is why we should not change receivers thoughtlessly, and we should restrict `apply` to concrete use cases. These use cases mainly include setting up an object after its creation and defining DSL function definitions.

```
fun createDialog() = Dialog().apply {  
    title = "Some dialog"  
    message = "Just accept it, ok?"  
    // ...  
}  
  
fun showUsers(users: List<User>) {  
    listView.apply {  
        adapter = UsersListAdapter(users)  
        layoutManager = LinearLayoutManager(context)  
    }  
}
```

The dangers of careless receiver overloading

The `this` receiver can be used implicitly, which is both convenient and potentially dangerous. It is not a good situation when we don't know which receiver is being used. In some languages, like JavaScript, this is a common source of mistakes. In Kotlin, we have more control over the receiver, but we can still easily fool ourselves. To see an example, try to guess what the result of the following snippet will be:

```
class Node(val name: String) {

    fun makeChild(childName: String) =
        create("$name.$childName")
        .apply { print("Created $name") }

    fun create(name: String): Node? = Node(name)
}

fun main() {
    val node = Node("parent")
    node.makeChild("child")
}
```

The intuitive answer is “Created child”, but the actual answer is “Created parent”. Why? Notice that the `create` function declares a nullable result type, so the receiver inside `apply` is `Node?`. Can you call `name` on `Node?` type? No, you need to unpack it first. However, Kotlin will automatically (without any warning) use the outer scope, and that is why “Created parent” will be printed. We fooled ourselves. The solution is to not create unnecessary receivers. This is not a case in which we should use `apply`: it is a clear case for `also`, for which Kotlin would force us to use the argument value safely if we used it.

```
class Node(val name: String) {

    fun makeChild(childName: String) =
        create("$name.$childName")
        .also { print("Created ${it?.name}") }

    fun create(name: String): Node? = Node(name)
}

fun main() {
    val node = Node("parent")
    node.makeChild("child") // Created child
}
```

with

```
// `with` implementation without contract
inline fun <T, R> with(receiver: T, block: T.() -> R): R =
    receiver.block()
```

As you can see, changing a receiver is not a small deal, so it is good to make it visible. `apply` is perfect for object initialization; for most other cases, a very popular option is `with`. **We use `with` to explicitly turn an argument into a receiver.**

In contrast to other scope functions, `with` is a top-level function whose first argument is used as its lambda expression receiver. This makes the new receiver definition really visible.

Typical use cases for `with` include explicit scope changing in Kotlin Coroutines, or specifying multiple assertions on a single object in tests.

```
// explicit scope changing in Kotlin Coroutines
val scope = CoroutineScope(SupervisorJob())
with(scope) {
    launch {
        // ...
    }
    launch {
        // ...
    }
}

// unit-test assertions
with(user) {
    assertEquals(aName, name)
    assertEquals(aSurname, surname)
    assertEquals(aWebsite, socialMedia?.websiteUrl)
    assertEquals(aTwitter, socialMedia?.twitter)
    assertEquals(aLinkedIn, socialMedia?.linkedin)
    assertEquals(aGithub, socialMedia?.github)
}
```

`with` returns the result of its block argument, so it can be used as a transformation function; however, this fact is rarely used, and I would suggest using `with` as if it is returning `Unit`.

run

```
// `run` implementation without contract
inline fun <R> run(block: () -> R): R = block()

// `run` implementation without contract
inline fun <T, R> T.run(block: T.() -> R): R = block()
```

We have already encountered a top-level `run` function in the *Lambda expressions* chapter. It just invokes a lambda expression. Its only advantage over an immediately invoked lambda expression (`{ /* ... */ }()`) is that it is inline. A

plain `run` function is used to form a scope. This is not a common need, but it can be useful from time to time.

```
val locationWatcher = run {  
    val positionListener = createPositionListener()  
    val streetListener = createStreetListener()  
    LocationWatcher(positionListener, streetListener)  
}
```

Another variant of the `run` function is invoked on an object. Such an object becomes a receiver inside the `run` lambda expression. However, I do not know any good use cases for this function. Some developers use `run` for certain use cases, but nowadays, I rarely see `run` used in commercial projects. Personally, I avoid using it^{[43](#)}.

Using scope functions

In this chapter, we have learned about many small but useful functions, called **scope functions**. Most of them have clear use cases. Some compete with each other for use cases (especially `let` and `apply`, or `apply` and `with`). Nevertheless, knowing all these functions well and using them in suitable situations is a recipe for nicer and cleaner code. Just please use them only where they make sense; don't use them just to use them.

A simplified comparison between key scope functions is presented in the following table:

Returns Reference to receiver	Receiver	Results of lambda
it	also	let
this	apply	run/with

Context receivers

Context receivers were added in Kotlin 1.6.20 and do not work in earlier versions. What is more, to enable this experimental feature in that version, one needs to add the “-Xcontext-receivers” compiler argument.

There are two kinds of problems that extension functions help us solve. The first one is quite intuitive: extending types with additional methods. This is basically what extension functions are designed for. So, for instance, if you need the `capitalize` method on `String` or the `product` method on `Iterable<Int>`, nothing is lost as you can always add these methods using an extension function.

```
fun String.capitalize() = this
    .replaceFirstChar(Char::uppercase)

fun Iterable<Int>.product() = this
    .fold(1, Int::times)

fun main() {
    println("alex".capitalize()) // Alex
    println("this is text".capitalize()) // This is text
    println((1..5).product()) // 120
    println(listOf(1, 3, 5).product()) // 15
}
```

The second kind of use case is less obvious but also quite common. We turn functions into extensions to explicitly pass a context of their use. Let’s take a look at a few examples.

Consider a situation in which you use Kotlin HTML DSL, and you want to extract some structures into a function. We might use the DSL we defined in the *Type Safe DSL Builders* chapter and define a `standardHead` function that sets up a standard head. Such a function needs a reference to `HtmlBuilder`, which we might provide as an extension receiver.

```
fun HtmlBuilder.standardHead() {
    head {
        title = "My website"
        css("Some CSS1")
        css("Some CSS2")
    }
}

val html = html {
    standardHead()
}
```

```
body {  
    h1("Title")  
    h3("Subtitle 1")  
    +"Some text 1"  
    h3("Subtitle 2")  
    +"Some text 2"  
}  
}
```

Defining an extension function in a case like this is very popular because it is very convenient. However, this is not what extension functions were initially designed for: we do not intend to call `standardHead` on an object of type `HtmlBuilder`. Instead, we want it to be used where there is a receiver of type `HtmlBuilder`. An extension in such a use case is used to receive a context. We should prefer a dedicated feature for just receiving a context. Why? Let's consider the essential extension function problems with this use case.

Extension function problems

Extension functions were designed to define new methods to call on objects, so they do not work well when used to receive a context. Here are the most important problems:

- extension functions are limited to a single receiver,
- using an extension receiver to pass a context gives a false impression of this function's meaning and how it should be called,
- an extension function can only be called on a receiver object.

Let's discuss these problems in detail.

Extension functions are limited to a single receiver. This makes a lot of sense when we define extension functions as methods to call on objects, but not when we want to use them to pass a receiver.

For example, when we use Kotlin Coroutines, we often want to launch a flow on a coroutine scope[12_1]. A scope is often used as a receiver, but the function used to launch it is already an extension on `Flow<T>`, so it cannot also be an extension on `CoroutineScope`. As a result we have the `launchIn` function, which expects `CoroutineScope` as a regular argument and is often called as `launchIn(this)`.

```
import kotlinx.coroutines.coroutineScope
import kotlinx.coroutines.flow.*

fun <T> Flow<T>.launchIn(scope: CoroutineScope): Job =
    scope.launch { collect() }

suspend fun main(): Unit = coroutineScope {
    flowOf(1, 2, 3)
        .onEach { print(it) }
        .launchIn(this)
}
```

Using an extension receiver to pass a context gives a false impression of this function's meaning and how it should be called. To understand this, consider the `sendNotification` function, which sends a notification to a user. Its additional functionality is displaying info using a logger. Let's say that in our application we make our classes implement `LoggerContext` to be able to use a logger implicitly. When we call `sendNotification`, we need to pass this `LoggingContext` somehow, and the most convenient way is as a receiver. So, we define `sendNotification` as an extension function on `LoggerContext`. However, this is a very poor design choice because it suggests that `sendNotification` is a method on `LoggingContext`, which is not true.

```
interface LoggingContext {
    val logger: Logger
}

fun LoggingContext.sendNotification(
    notification: NotificationData
) {
    logger.info("Sending notification $notification")
    notificationSender.send(notification)
}
```

An extension function can only be called on objects, which is precisely why extension functions were invented, but this is not great when we want to use extension functions to pass a receiver implicitly. Consider `standardHead` from the example above. We want to use it as a part of HTML DSL, but we do not want to allow it to be called on an object of type `HtmlBuilder`

```
// Do
html {
    standardHead()
}

// Don't
builder.standardHead()

// Will do
```

```
with(receiver) {  
    standardHead()  
}
```

To address all these problems, Kotlin introduced a feature called *context receivers*.

Introducing context receivers

Kotlin 1.6.20 introduced a new feature that is dedicated to passing implicit receivers into functions. This feature is called **context receivers** and it addresses all the aforementioned issues. How do we use it? For any function, we can specify the context receiver types inside brackets after the context keyword. Such functions have receivers of specified types, and these functions need to be called in the scope where all the specified receivers are.

```
class Foo {  
    fun foo() {  
        print("Foo")  
    }  
}  
  
context(Foo)  
fun callFoo() {  
    foo()  
}  
  
fun main() {  
    with(Foo()) {  
        callFoo()  
    }  
}
```

Importantly, a context receiver function call expects an implicit receiver, so such functions cannot be called on an object of receiver type.

```
fun main() {  
    Foo().callFoo() // ERROR  
}
```

When you want to use an explicit context receiver, you always need to specify a label after this with a type that specifies which receiver you want to use.

```
context(Foo)  
fun callFoo() {  
    this@Foo.foo() // OK  
    this.foo() // ERROR, this is not defined  
}
```

Context receivers can specify multiple receiver types. For example, in the code below, the `callFooBoo` function expects both `Foo` and `Boo` receiver types.

```
class Foo {
    fun foo() {
        print("Foo")
    }
}

class Boo {
    fun boo() {
        println("Boo")
    }
}

context(Foo, Boo)
fun callFooBoo() {
    foo()
    boo()
}

context(Foo, Boo)
fun callFooBoo2() {
    callFooBoo()
}

fun main() {
    with(Foo()) {
        with(Boo()) {
            callFooBoo() // FooBoo
            callFooBoo2() // FooBoo
        }
    }
    with(Boo()) {
        with(Foo()) {
            callFooBoo() // FooBoo
            callFooBoo2() // FooBoo
        }
    }
}
```

A receiver is anything that this represents. It might be an extension function receiver, a lambda expression receiver, or a dispatch receiver (the enclosing class for methods and properties). One receiver can be used for multiple expected types. For example, in the code below, inside the method `call` in `FooBoo`, we use a dispatch receiver for both `Foo` and `Boo` types.

```
package fgfds

interface Foo {
    fun foo() {
        print("Foo")
    }
}
```



```

interface Boo {
    fun boo() {
        println("Boo")
    }
}

context(Foo, Boo)
fun callFooBoo() {
    foo()
    boo()
}

class FooBoo : Foo, Boo {
    fun call() {
        callFooBoo()
    }
}

fun main() {
    val fooBoo = FooBoo()
    fooBoo.call() // FooBoo
}

```

Use cases

Now, let's see how context receivers address the aforementioned issues. We could use a context receiver to define that `standardHead` needs to be called on `HtmlBuilder`. This way should be preferred over using an extension function.

```

context(HtmlBuilder)
fun standardHead() {
    head {
        title = "My website"
        css("Some CSS1")
        css("Some CSS2")
    }
}

```

Context receivers are a better choice for most functions that should be used on a DSL and DSL definitions.

The function that is used to launch a flow can also benefit from context receiver functionality. We could define a `launchFlow` extension function on `Flow<T>` with the `CoroutineScope` context receiver. Such a function needs to be called on a flow in a scope where `CoroutineScope` is a receiver.

```

import kotlinx.coroutines.coroutineScope
import kotlinx.coroutines.flow.*

context(CoroutineScope)
fun <T> Flow<T>.launchFlow(): Job =

```

```
        this@CoroutineScope.launch { collect() }

suspend fun main(): Unit = coroutineScope {
    flowOf(1, 2, 3)
        .onEach { print(it) }
        .launchFlow()
}
```

Now, consider the `sendNotification` function, which needed `LoggingContext`, but we did not want it to be defined as an extension function. We could provide `LoggingContext` using a context receiver.

```
context(LoggingContext)
fun sendNotification(notification: NotificationData) {
    logger.info("Sending notification $notification")
    notificationSender.send(notification)
}
```

Now let's see some other examples. Consider an external DSL builder where you can add items using the `addItem` method.

```
fun myChristmasLetter() = christmasLetter {
    title = "My presents list"
    addItem("Cookie")
    addItem("Drawing kit")
    addItem("Poi set")
}
```

Let's say that you want to extend this builder to make it possible to define items using the unary plus operator on `String` instead:

```
fun myChristmasLetter() = christmasLetter {
    title = "My presents list"
    +"Cookie"
    +"Drawing kit"
    +"Poi set"
}
```

To do that, we need to define a `unaryPlus` operator function which is an extension on `String`. However, we also need a receiver that will let us add elements using the `addItem` function. To do that, we can use a context receiver.

```
context(ChristmasLetterBuilder)
operator fun String.unaryPlus() {
    addItem(this)
}
```

A popular Android example of using a context receiver is defining dp size (density-independent pixels) in code. This is a standard way of describing width or height. The problem is that dp size depends on a view because it depends on display density. The solution is that the dp extension property might have a context receiver of View type. Then, such a property can quickly and conveniently be used as a part of view builders.

```
context(View)
val Float.dp get() = this * resources.displayMetrics.density

context(View)
val Int.dp get() = this.toFloat().dp
```

As you can see, there are many cases in which context receiver functionality is useful, but remember that most of them are related to DSL builders.

Classes with context receivers

A context receiver can also be used for classes; in practice, this means that a receiver is expected when we call the constructor of a class with a context receiver, and it is then stored in an additional property.

```
package sdfgv

class ApplicationConfig(
    val name: String,
) {
    fun start() {
        print("Start application")
    }
}

context(ApplicationConfig)
class ApplicationControl(
    val applicationName: String = this@ApplicationConfig.name
) {
    fun start() {
        print("Using control: ")
        this@ApplicationConfig.start()
    }
}

fun main() {
    with(ApplicationConfig("AppName")) {
        val control = ApplicationControl()
        println(control.applicationName) // AppName
        control.start() // Using control: Start application
    }
}
```

This feature is experimental, and might be removed in the future versions of Kotlin.

Concerns

Like every good feature, context receivers can also be used poorly, which could lead to code that is more complicated or less safe than it needs to be. We should use this feature only where it makes sense, and using too many receivers in our code is not good for readability. Implicit function calls are not as clear as explicit ones. There are also risks of name collisions. Receivers are not as visible as arguments. Using implicit receivers too often can make code confusing for other developers. I suggest not using context receivers if they often need wrapping function calls using scope functions, like with.

```
// Don't do this
context(
    LoggerContext,
    NotificationSenderProvider, // not a context
    NotificationsRepository // not a context
) // it might hard to call such a function
suspend fun sendNotifications() {
    log("Sending notifications")
    val notifications = getUnsentNotifications() // unclear
    val sender = create() // unclear
    for (n in notifications) {
        sender.send(n)
    }
    log("Notifications sent")
}

class NotificationsController(
    notificationSenderProvider: NotificationSenderProvider,
    notificationsRepository: NotificationsRepository
) : Logger() {
    @Post("send")
    suspend fun send() {
        with(notificationSenderProvider) { // avoid such calls
            with(notificationsRepository) { //avoid such calls
                sendNotifications()
            }
        }
    }
}
```

In general, my suggestions are:

- When there is no good reason to use a context receiver, prefer using a regular argument.

- When it is unclear from which receiver a method comes, consider using an argument instead of the receiver or use this receiver explicitly[12_2].

```
// Don't do that
context(LoggerContext)
suspend fun sendNotifications(
    notificationSenderProvider: NotificationSenderProvider,
    notificationsRepository: NotificationsRepository
) {
    log("Sending notifications")
    val notifications = notificationsRepository
        .getUnsentNotifications()
    val sender = notificationSenderProvider.create()
    for (n in notifications) {
        sender.send(n)
    }
    log("Notifications sent")
}

class NotificationsController(
    notificationSenderProvider: NotificationSenderProvider,
    notificationsRepository: NotificationsRepository
) : Logger() {
    @Post("send")
    suspend fun send() {
        sendNotifications(
            notificationSenderProvider,
            notificationsRepository
        )
    }
}
```

Summary

Kotlin introduced a new prototype feature called context receivers to address situations in which we want to pass receivers into functions or classes implicitly. Until now, we used extension functions for this, but their biggest issues were:

- extension functions are limited to a single receiver,
- using an extension receiver to pass a context gives a false impression of this function's meaning and how it should be called,
- an extension function can only be called on a receiver object.

Context receivers solve all these problems and are very convenient. I'm looking forward to them becoming a stable feature that I can use in my projects.

[12_1]: More about this in the *Kotlin Coroutines: Deep Dive* book.

[12_2]: See *Effective Kotlin, Item 15: Consider referencing receivers explicitly*.

A birds-eye view of Arrow

In this book, I have concentrated on Kotlin features inspired by innovations from the functional programming community. Kotlin supports many of these, but plenty of tools, techniques, and concepts are still left. Since there are many Functional Programming enthusiasts in the Kotlin community, some of them took matters into their own hands and made libraries that extend this support and allow programs to be written in a more functional style. Among these libraries, there is a group that is clearly the most popular and influential: Arrow (website `arrow-kt.io`), which is a set of libraries and compiler plug-ins with the common goal of making functional-style programming in Kotlin both easier and more productive.

I decided that this book must present at least a birds-eye view of Arrow, so I asked its maintainers to give you the best explanation directly from the source. Now I am happy to present this chapter, which presents essential Arrow features; it is written by Alejandro Serrano Mena, Raúl Raja Martínez, and Simon Vergauwen - Arrow maintainers and co-creators whose contribution to Arrow is astonishing.

In this chapter, we focus on Arrow Core and Arrow Optics, leaving aside Arrow Fx (a library that builds upon the coroutine support in Kotlin) and Arrow Analysis (which introduces new forms of static analysis).

Functions and Arrow Core

This part was written by Alejandro Serrano Mena, with support from Simon Vergauwen and Raúl Raja Martínez.

Let's begin with the Core library, which focuses on making functional programming shine in Kotlin. To use it, you need to add `io.arrow-kt:arrow-core` as a dependency in your project. At the time of writing, the library is in the 1.1.x series, with 2.0 being planned and worked on.

Being a library that targets functional programming, Arrow Core includes several extensions for function types, in the `arrow.core` package. The first one

is compose, which creates a new function by executing two functions, one after another:

```
val squaredPlusOne: (Int) -> Int =  
  { x: Int -> x * 2 } compose { it + 1 }
```

The function above is equivalent to `{ x: Int -> (x + 1) * 2 }`. The composition of functions works from right to left. This is often surprising at first because we read code from left to right. However, this can simplify complex chains of functions, especially when using function references, whereas the corresponding version with explicit parameters requires nesting.

```
people.filter(  
  Boolean::not compose ::goodString compose Person::name  
)  
  
// instead of  
people.filter { !goodString(it.name) }
```

This way of writing functions only works when they take *exactly* one parameter. But let's say that we want to replace our reference to `goodString` with a different check. In particular, we want to check whether the string starts with a given prefix. To do so, we want to use the `isPrefixOf` function, which takes such a prefix as an argument.

```
fun String.isPrefixOf(s: String) = s.startsWith(this)
```

If we replace `::goodString` with `String::isPrefixOf`, the compiler rightly complains. It's expecting a function with a single argument, but `isPrefixOf` has two (the receiver and the argument `s`). We could create a lambda that gives the first argument, but another solution is to use one of the helper functions in Arrow Core.

```
(String::isPrefixOf).partially1("FP")
```

This is an example of *partial application*, i.e., creating a function by providing fewer arguments than required to another function. Here we are providing one fewer argument to `isPrefixOf`. You may have noticed the 1 at the end of `partially1`. Arrow Core includes functions to partially apply not only one but up to 22 arguments at once.

Memoization

There's a function that is typically discussed when introducing recursive functions: Fibonacci numbers. These numbers form a sequence, 0, 1, 1, 2, 3, 5, 8, ..., in which a given element is the sum of the two elements that precede it (except for the initial values 0 and 1). This is an example of a function whose stack may grow wildly, even for small arguments, so Kotlin recommends using the `DeepRecursiveFunction` constructor to define it:

```
val fibonacci = DeepRecursiveFunction<Int, Int> { x ->
    when {
        x < 0 -> 0
        x == 1 -> 1
        else -> callRecursive(x - 1) + callRecursive(x - 2)
    }
}
```

This way, we prevent the stack from overflowing. However, notice that `Fibonacci` is not defined as a `fun` but as a `val`, so we prefer to have an actual bridge function that starts the recursive computation.

```
fun fib(x: Int) = fibonacci(x)
```

Now the function no longer causes a stack overflow, but we have another problem: we are wasting loads of time computing the same values over and over. Imagine, for example, we want `fib(4)`, which requires both `fib(3)` and `fib(2)`. But the computation of `fib(3)` also requires `fib(2)`! Since this function is pure, we know that both calls to `fib(2)` return the same value. For these scenarios, we can apply the technique of *memoization*, i.e., caching intermediate values to avoid recomputations. Arrow Core contains a specific function called `memoize`, which takes care of creating and updating this cache, so all we need to do is:

```
fun fibM(x: Int) = ::fib.memoize()(x)
```

In this case, by the time we get to the second call to `fib(2)`, the entire sequence at that point has been computed and cached. We go from an exponential blowup to a linear function.

Testing higher-order functions

The last piece of functionality we describe in this section relates not to using functions but to *testing* them. Many people in the FP community use property-based testing instead of bare unit testing: the idea is that instead of checking particular input/output pairs, you execute a function with random inputs and

check that the output satisfies some properties. For example, if you test an ordering function, you need to check that the elements in the output are equal to the elements of the input. One important part of a property-based testing framework like Kotest, is the set of *generators*. Generators are responsible for creating random values, and we want these generated values to have a nice distribution across the entire domain and to provide common corner cases. Think about a generator for `Int`: values close to zero and close to the overflow and underflow points tend to break functions that don't account for these corner cases. Kotest comes with a big battery of generators in the `Arb` object, but there's no support for generating functions. This means you cannot test higher-order functions, so you generally need to resort to good ol' unit testing in these cases; however, if you bring the `kotest-property-arrow` library into your project, this limitation is gone.

```
val gen = Arb.functionAToB<Int, Int>(Arb.int())
```

Now you can use this generator to test the behavior of functions like `map`.

Error Handling

This part was written by Simon Vergauwen, with support from Alejandro Serrano Mena and Raúl Raja Martínez.

When writing code in a functional style, we typically want our function signatures to be as accurate as possible. We don't wish to have errors represented as exceptions; instead, we reflect these errors as part of the return type of a function. Composing functions that return types with errors is more complex.

One of the most common exceptions in Java is `NullPointerException`, and Kotlin has an elegant solution: nullable types! Kotlin allows us to model the *absence* of a value through nullable types.

For example, Java offers `Integer.parseInt`, which can unexpectedly throw `NumberFormatException`. Still, Kotlin has `String.toIntOrNull`, which returns `Int?` as a result type and produces `null` when a `String` can't coerce to an `Int`.

Kotlin doesn't have checked exceptions, so there is no way for a function to signal to the caller that it needs to be wrapped in try-catch. When using nullable types, we can force the user to handle possible null values or failures.

Working with nullable types

Let's take a simple example that reads a value from the environment and results in an optional String value. In the function below, the exception from `System.getenv` is swallowed and flattened into `null`.

```
/** read value from environment,
 * or null if failed or not present */
fun envOrNull(name: String): String? =
    runCatching { System.getenv(name) }.getOrNull()
```

Now we can use this function to read values from our environment and build a simple example of combining nullable functions to load a data class `Config(val port: Int)`. Within Java, the most common way to deal with `null` is to use `if(x != null)`, so let's explore that first.

```
fun configOrNull(): Config? {
    val envOrNull = envOrNull("port")
    return if (envOrNull != null) {
        val portOrNull = envOrNull.toIntOrNull()
        if (portOrNull != null) Config(portOrNull) else null
    } else null
}
```

The simple example is already considerably complex and contains some repetition. Luckily, the Kotlin compiler has smart-casted the values to non-null inside the branch of each `if` statement, thus ensuring you can safely access them as non-nullable values.

Kotlin offers much nicer ways of working with nullable types, such as `?.` and scoping functions like `let`.

The same code above can be expressed as:

```
fun config2(): MyConfig? =
    envOrNull("port")?.toIntOrNull()?.let(::Config)
```

The above snippet is more Kotlin idiomatic and easier to read. Sadly, this syntax only works for nullable types; other types such as `Result` or `Either` cannot benefit from the special `?` syntax.

There are two improvements we could make to the code above:

1. Unify APIs to work with errors and nullable types.
2. Swallow all exceptions from `System.getenv` into `null`.

To solve the first issue, we can leverage Arrow's DSLs. A DSL is a *Domain Specific Language* or an API that is specific to working with a particular domain. Arrow offers DSLs based on continuations that offer a unified API for working with all error types.

First, rewrite our above example using the nullable Arrow DSL. The `nullable.eager` offers us a DSL with `bind`, which allows us to unwrap `Int?` to `Int`.

```
import arrow.core.continuations.nullable

fun config3(): Config? = nullable.eager {
    val env = envOrNull("port").bind()
    val port = env.toIntOrNull().bind()
    Config(port)
}
```

In Arrow 1.x.x there is `nullable.eager { }` for *non-suspend* code, and `nullable { }` for *suspend* code. In Arrow 2.x.x this will become simply `nullable { }` for both *suspend* & *non-suspend* code.

If we encounter a `null` value when unwrapping `Int?` to `Int` using `bind`, then the `nullable.eager { }` DSL will immediately return `null` without running the rest of the code in the lambda. Using `.bind` is an easier alternative to applying the Elvis operator on each check and short-circuiting the lambda with an early return:

```
fun add(a: String, b: String): Int? {
    val x = a.toIntOrNull() ?: return null
    val y = b.toIntOrNull() ?: return null
    return x + y
}
```

To prevent swallowing any exceptions from `System.getenv`, we can use `runCatching` and `Result` from the Kotlin Standard Library.

Working with Result

`Result<A>` is a special type in Kotlin that we can use to model the result of an operation that may succeed or may result in an exception. To more accurately model our previous operation `envOrNull` of reading a value from the environment, we use `Result` to model the failure of `System.getenv`. Additionally, the environment variable might not be present, so the function should return `Result<String?>` to also model the potential absence of the environment variable.

Our previous `envOrNull` can leverage `Result` as the return type:

```
fun envOrNull(name: String): Result<String?> = runCatching {
    System.getenv(name)
}
```

The `envOrNull` function defined above now correctly models the failure of `System.getenv` and the potential absence of our environment variable. Now, we need to deal with nullable types inside the context of `Result`. Luckily, the Arrow DSL offers a DSL for `Result` that allows us to work with `Result` in the same way as we did for the nullable types above.

To ensure that our environment variable is present, the Arrow DSL offers `ensureNotNull`, which checks if the passed value `envOrNull` is **not** null and smart-casts it. If `ensureNotNull` encounters a null value, it returns a `Result.failure` with the passed exception. In this case, we return `Result.failure(IllegalArgumentException("Required port value was null."))` when encountering null.

Finally, we must transform our `String` into an `Int`. The most convenient way of doing this inside the `Result` context is using `toInt`, which throws a `NumberFormatException` if the passed value is not a valid `Int`. When using `toInt`, we can use `runCatching` to safely turn it into `Result<Int>`.

```
import arrow.core.continuations.result

fun config4(): Result<Config> = result.eager {
    val envOrNull = envOrNull("port").bind()
    ensureNotNull(envOrNull) {
        IllegalStateException("Required port value was null")
    }
    val port = runCatching { envOrNull.toInt() }.bind()
    Config(port)
}
```

The example above used Kotlin's `Result` type to model the different failures to load the configuration:

- Any exceptions thrown from `System.getenv` using `SecurityException` or `Throwable`
- The absence of the environment variable using `IllegalStateException`
- The failure of `toInt` using `NumberFormatException`

If the API you are interfacing with throws exceptions, `Result` might be the best way to model your use case. If you are designing a library or application, you may want to control your error types, and these types do not need to be part of the `Throwable` or exception hierarchies.

It doesn't make sense to use `Result` for every error type you want to model. With `Either`, we can model the different failures to load the configuration in a more expressive and better-typed way without depending on exceptions or `Result`.

Working with Either

Before we dive into solving our problem with `Either`, let's first take a quick look at the `Either` type itself.

`Either<E, A>` models the result of a computation that might fail with an error of type `E` or success of type `A`. It's a sealed class, and the `Left` and `Right` subtypes accordingly represent the Error and Success cases.

```
sealed class Either<out E, out A> {  
    data class Left<E>(val value: E) : Either<E, Nothing>()  
    data class Right<A>(val value: A) : Either<Nothing, A>()  
}
```

When modeling our errors with `Either`, we can use any type to represent failures arising from loading our `Config`.

In our `Result` example, we used the following exceptions to model our errors:

- `SecurityException/Throwable` when accessing the environment variable
- `IllegalStateException` when the environment variable is not present
- `NumberFormatException` when the environment variable is present but is not a valid `Int`

In this new example based on Either, we can instead model our errors with a sealed type ConfigError.

```
sealed interface ConfigError
data class SystemError(val underlying: Throwable)
object PortNotAvailable : ConfigError
data class InvalidPort(val port: String) : ConfigError
```

ConfigError is a sealed interface that represents all the different kinds of errors that can occur when loading. During the loading of our configuration, an unexpected system error could occur, such as java.lang.SecurityException. The SystemError type represents this. When the environment variable is absent, we should return the PortNotAvailable type; when the environment variable is present but is not a valid Int, we should return an InvalidPort type.

This new error encoding based on a sealed hierarchy changes our previous example to:

```
import arrow.core.continuations.either

fun config5(): Either<ConfigError, Config> = either.eager {
    val envOrNull = Either.catch { System.getenv("port") }
        .mapLeft(::SecurityError)
        .bind()
    ensureNotNull(envOrNull) { PortNotAvailable }
    val port = ensureNotNull(envOrNull.toIntOrNull()) {
        InvalidPort(env)
    }
    Config(port)
}
```

The above example uses Either.catch to catch any exception thrown by System.getenv; it then _map_s them to a SecurityError using mapLeft before calling bind. If we had not mapped our error from Either<Throwable, String?> to Either<SecurityError, String?>, we would not have been able to call bind because our Either<ConfigError, Config> context can only handle errors of type ConfigError. Finally, we use ensureNotNull again to check if the environment variable is present. We also rely on ensureNotNull for the result of the toIntOrNull call.

Our original sample has improved so as to **not** swallow any exceptions and return all errors in a typed manner.

A final improvement we can still make to the function that loads our configuration is to ensure that the Port is valid. So, we check if the value lies

between 0 and 65535; if not, we return our existing error type `InvalidPort`.

```
import arrow.core.continuations.either

private val VALID_PORT = 0..65536

fun config5(): Either<ConfigError, Config> = either.eager {
    val envOrNull = Either.catch { System.getenv("port") }
        .mapLeft(::SecurityError)
        .bind()
    val env = ensureNotNull(envOrNull) { PortNotAvailable }
    val port = ensureNotNull(env.toIntOrNull()) {
        InvalidPort(env)
    }
    ensure(port in VALID_PORT) { InvalidPort(env) }
    Config(port)
}
```

In the examples above, we've learned that we can have all flavors of error handling with nullable types, `Result` or `Either`. We use nullable types when a value can be absent, or we don't have any useful error information; we use `Result` when the operations may fail with an exception; and we use `Either` when we want to control custom error types that are not exceptions.

Data Immutability with Arrow Optics

This part was written by Alejandro Serrano Mena, with support from Simon Vergauwen and Raúl Raja Martínez.

When working on a functional programming-inspired codebase, you often want to limit the number of side effects a function can perform. Of these, mutability is one of the main offenders: a function that depends on a mutable variable may potentially change its behavior between two runs, even if the arguments provided are exactly the same between these two runs. Making this rule more concrete in Kotlin leads to a style which:

- Prefers `val` over `var`, even to the point of forbidding `var` entirely.
- Models the application domain using data classes without methods, instead of using object-oriented techniques in which classes hold both data and behavior.

Here's one example of how persons and addresses are modeled in this fashion:

```
data class Address(  
    val zipcode: String,  
    val country: String  
)  
data class Person(  
    val name: String,  
    val age: Int,  
    val address: Address  
)
```

In fact, the design of data classes in Kotlin complements functional programming very well, thus making it much easier to err on the side of immutability. When using data classes, constructors and fields are defined in one go; no boilerplate is required, as in Java⁴⁴. Another prime example of this is the copy method, which allows us to create a new version of a value based on another one, where we only change a few of the fields.

```
fun Person.happyBirthday(): Person =  
    copy(age = age + 1)
```

This nice syntax falls short, however, when the transformation affects nested fields. For example, let's say we want to normalize the way countries are spelled out within Person. The code is by no means pretty.

```
fun Person.normalizeCountry(): Person =  
    copy(  
        address = address  
            .copy(country = address.country.capitalize())  
    )
```

Arrow Optics provides a solution to this problem as part of the more general problem of transforming immutable data with nice syntax. Two libraries working together give Arrow Optics its power: there's the basic `io.arrow-kt:arrow-optics` library, and there's also the `io.arrow-kt:arrow-optics-ksp-plugin` compiler plug-in, which automates some of the boilerplate required by the former. The plug-in is built using the Kotlin Symbol Processing API (KSP)⁴⁵. Once the plug-in is ready, you only need to sprinkle some `@optics` annotations⁴⁶ in your code to let the fun begin.

```
@optics  
data class Address(  
    val zipcode: String,  
    val country: String  
) {  
    companion object  
}
```



```
@optics
data class Person(
    val name: String,
    val age: Int,
    val address: Address
) {
    companion object
}
```

Under the hood, the compiler plug-in generates *lenses*, which are a particular kind of *optics*. A lens is nothing more than a combination of a getter and a setter; however, in contrast to them, you use the name of the field before the element to be queried or modified. These lenses are generated as part of the companion object of the class the `@optics` annotation is applied to, so you can find them under the class name. The code below shows an implementation of `happyBirthday` using lenses.

```
fun Person.happyBirthday(): Person {
    val currentAge = Person.age.get(this)
    return Person.age.set(this, currentAge + 1)
}
```

Note that the `set` function, regardless of its name, works as a copy method for a particular field: it generates a new version of the given value. This simplest use of lenses already brings some benefits. For example, the pattern for setting a new value of a field based on the previous value (as we are doing here for `age`) has been abstracted into the `modify` method. Kotlin's syntax for trailing lambdas allows for a very concise and readable implementation of `happyBirthday` in a single line.

```
fun Person.happyBirthday(): Person =
    Person.age.modify(this) { it + 1 }
```

Let's go back to our original problem of modifying nested fields in immutable objects without dying under a pile of copy methods. The trick is to *compose* lenses to create a new lens that focuses on the nested element. The setter (or the modifier) in this new lens changes exactly what we need and takes care of keeping the rest of the fields unchanged.

```
fun Person.normalizeCountry(): Person =
    (Person.address compose Address.country).modify(this) {
        it.capitalize()
    }
```

Accessing nested fields is such a common operation that the Arrow Optics developers have also decided to generate additional declarations to simplify this scenario. In particular, starting with an initial lens, you can compose automatically with a lens in the nested type by using a dot, as you would do with actual fields. This means you can write the preceding example as follows:

```
fun Person.normalizeCountry(): Person =
    (Person.address.country).modify(this) { it.capitalize() }
```

Optics are a big family whose ultimate goal is to make immutable data transformation easier. Up to this point, we've talked about lenses, which focus just on a single field, but the other important member of this family is traversals. Traversals make it possible to apply a transformation over several elements at once, so they are very useful for manipulating collections. As a concrete example, let's define a new data class which holds information about every person born on a single day; this could be interesting if we're sending a promotional code to people to celebrate their birthdays.

```
@optics
data class BirthdayResult(
    val day: LocalDate,
    val people: List<Person>
) {
    companion object
}
```

How do we change the age field for all of them? We not only need nested copy methods; we must also be careful that people is a list, so transformation occurs using map.

```
fun BirthdayResult.happyBirthday(): BirthdayResult =
    copy(people = people.map { it.copy(age = it.age + 1) })
```

The same transformation can be defined by composing several optics and then applying a single modify function, as before. The traversal required for this job lives in the Every class, which includes optics for the most commonly used collection types in Kotlin.

```
fun BirthdayResult.happyBirthday2(): BirthdayResult =
    (BirthdayResult.people
    compose Every.list()
    compose Person.age)
    .modify(this) { it + 1 }
```

We would like to stress that the biggest benefit of using optics is the *uniformity* of their API. Only two operations, `compose` and `modify`, were needed to define nested transformations of immutable data. Although getting used to this style of programming takes a bit of time, being acquainted with optics is definitely useful in the longer term.

Notes

- 1** In his presentation at Kotlin Dev Days 2022, Andrey Breslav claimed that he had asked Martin Odersky (the creator of Scala) about what makes a language functional, and he answered that every statement is an expression. A statement is a single command that a programmer expresses in a programming language, typically a single line of code. An expression is something that returns a value.[↵](#)
- 2** More about types in *Kotlin Essentials*, *Typing system* chapter.[↵](#)
- 3** A method is a function associated with a class; it is called on an object, so both member and extension functions are methods.[↵](#)
- 4** `Unit` is an object with a single value that can be used within generic types. A function with the return type `Unit` is equivalent to a Java method that declares `void`.[↵](#)
- 5** Those who have read the *Typing system* chapter from *Kotlin Essentials* might have guessed why I describe `Unit` as “nothing significant” instead of “nothing”. Functions in Kotlin can indeed return nothing; in such cases, they declare `Nothing` as a result type, but this has a very different meaning than `Unit`.[↵](#)
- 6** More about operators in *Kotlin Essentials*, *Operators* chapter.[↵](#)
- 7** The example was proposed by Endre Deak.[↵](#)
- 8** Protecting ourselves from type misuse is better described in *Effective Kotlin*, *Item 49: Consider using inline value classes*.[↵](#)
- 9** More about destructuring in *Kotlin Essentials*, *Data modifier* chapter.[↵](#)
- 10** Best practices regarding naming arguments are explained in *Effective Kotlin*, *Item 17: Consider naming arguments*. The named argument convention is explained in *Kotlin Essentials*, *Functions* chapter.[↵](#)

- 11** Also, the above algorithm is poorly implemented. It should instead use `sumOf` function, which we will present later in this book.↵
- 12** Top-level function is a function defined outside a class, so in a file.↵
- 13** More about reflection in *Advanced Kotlin, Reflection* chapter.↵
- 14** For this, the reference needs to be immediately typed as a function type.↵
- 15** `String::isNullOrBlank` also works because `String` is a subtype of `String?`; however, this doesn't make much sense because its function type is `(String) -> Boolean`, so it does not accept `null` and behaves like `String::isBlank`.↵
- 16** More about object expressions and object declarations in *Kotlin Essentials, Objects* chapter.↵
- 17** See *Effective Kotlin, Item 33: Consider factory functions instead of secondary constructors*.↵
- 18** For details, see [KEEP](https://kt.academy/l/keep-bound-ref), link: kt.academy/l/keep-bound-ref↵
- 19** A type parameter is a placeholder for a type, so it is typically T , T_1 , T_2 , R etc. A type argument is an actual type that is used when we call a generic function. In the `printTypeName<Int>()` call, the type `Int` is used as a type argument.↵
- 20** There is [an influential paper from 1991](#) **Functional Programming with Bananas, Lenses, Envelopes and Barbed Wire** that pushed the idea of common recursion schemes (`map`, `fold`, etc.) to separate the “what” from the “how” of processing using functional algebra.↵
- 21** Lisp is one of the oldest programming languages still in widespread use today. Often known as the father of all functional programming languages. Today, the best-known general-purpose Lisp dialects are Clojure, Common Lisp, and Scheme.↵
- 22** In this chapter, I will use the term “collection processing” as shorthand for “functional-style collection processing”.↵

23 In this chapter, by aliases we will mean functions with exactly the same meaning.[↵](#)

24 Destructuring is creating multiple variables based on a single value. This concept is explained in the book *Kotlin Essentials*.[↵](#)

25 To learn more about this, search under the term “vacuous truth”.[↵](#)

26 The `mapValues` function is a function on `Map` that transforms all values according to the transformation function.[↵](#)

27 I described using `groupBy` in *Effective Kotlin, Item 53: Consider using groupBy instead of groupBy*.[↵](#)

28 I hope it is clear, that `List` and `Set` are iterables, because they implement `Iterable` interface. [↵](#)

29 This optimization is better explained in *Effective Kotlin, Item 52: Consider associating elements to a map*.[↵](#)

30 The second difference is that a set does not necessarily keep elements in order.[↵](#)

31 Here is a note about a historical background, written by Owen Griffiths: Originally, this is supported in pure functional languages such as Haskell where it is called “list fusion” and transforms together compose-able function calls that would result in extra memory allocations for improved efficiency. In Clojure, a JVM Lisp, these are known as Transducers - in other words: - to move across.[↵](#)

32 Unless the sequence is constrained-once. For instance, the function `useLines` that reads lines from a file line-by-line can be used only once, and then it closes a connection to this file.[↵](#)

33 This sequence is generated using a coroutine. This is better explained in the book *Kotlin Coroutines: Deep Dive*.[↵](#)

34 You can find this database under the link kt.academy/1/chicago-crime-data[↵](#)

35 The problems come from the common join-fork thread pool they use, which allows one process to block another. There's also a problem with the fact that single-element processing blocks other elements. To read more about this, see the article *Think Twice Before Using Java 8 Parallel Streams* by Lukas Kreca, you can find under the link kt.academy/l/java8-streams↵

36 Jetpack Compose looks a bit different than a typical Kotlin DSL because some of its elements are added under the hood by the compiler plugin, and this process is based on annotations.↵

37 A dialog (as Wikipedia explains) is a graphical control element in the form of a small window that communicates information to the user and prompts for a response.↵

38 Concretely, when it is used as a receiver in a function type with a receiver, then it can only be used implicitly when it is the most inner receiver (so when it is an outer receiver, it needs to be used with an explicit `this@label`).↵

39 Operators are better described in *Kotlin Essentials*, *Operators* chapter.↵

40 Except for `with`, which is not an extension function.↵

41 Constructor references were explained in the chapter *Function references*. ↵

42 For details, see *Effective Kotlin*, *Item 46: Avoid member extensions*.↵

43 Email me if you have some good use cases where you think that `run` clearly fits better than the other scope functions. My email is marcinmoskala@gmail.com.↵

44 Projects like Lombok, which automatizes the generation of “dummy” getters, setters, and equality functions, show that this pattern is really widespread.↵

45 Please check the instructions on how to enable it for your particular project set-up (at the time of writing, there are important differences depending on whether you need Multiplatform support or not).↵

46 For technical reasons, a companion object (even if empty) is required for the plug-in to work.↵